

POP-32

**บอร์ดไมโครคอนโทรลเลอร์
เพื่อการเรียนรู้และพัฒนาโครงงาน
ระบบสมองกลฝังตัวและหุ่นยนต์**

นคร ภัคดีชาติ

วรพจน์ กรแก้ววัฒนกุล

ชัยวัฒน์ ลิ้มพรจิตรวิไล

Innovative Experiment Co.,Ltd



POP-32 บอร์ดไมโครคอนโทรลเลอร์เพื่อการเรียนรู้และพัฒนาโครงงานระบบสมองกลฝังตัวและหุ่นยนต์

สงวนลิขสิทธิ์ตาม พ.ร.บ. ลิขสิทธิ์ พ.ศ. 2537

ห้ามการลอกเลียนไม่ว่าส่วนหนึ่งส่วนใดของหนังสือเล่มนี้ นอกจากจะได้รับอนุญาต

ใครควรใช้หนังสือเล่มนี้

1. นักเรียน นิสิต นักศึกษา และบุคคลทั่วไปที่มีความสนใจในการนำไมโครคอนโทรลเลอร์ไปประยุกต์ใช้ในการทดลองเกี่ยวกับการทำงานของระบบอัตโนมัติ หรือสนใจในการเรียนรู้และทดลองไมโครคอนโทรลเลอร์ในแนวทางใหม่ที่ใช้กิจกรรมโครงงานวิทยาศาสตร์ประยุกต์เป็นสื่อ
2. สถาบันการศึกษา โรงเรียนในระดับมัธยมศึกษาที่มีการเรียนการสอนในวิชาตามกลุ่มสาระวิชาวิทยาศาสตร์ เทคโนโลยี วิศวกรรม และคณิตศาสตร์ (STEM)
5. วิทยาลัยและมหาวิทยาลัยที่มีการเปิดการเรียนการสอนวิชาอิเล็กทรอนิกส์หรือภาควิชาวิศวกรรมอิเล็กทรอนิกส์และคอมพิวเตอร์
4. คณาจารย์ที่มีความต้องการศึกษา และเตรียมการเรียนการสอนวิชาตามกลุ่มสาระวิชาวิทยาศาสตร์ เทคโนโลยี วิศวกรรม และคณิตศาสตร์ (STEM), ไมโครคอนโทรลเลอร์ รวมถึงวิทยาศาสตร์ประยุกต์ที่ต้องการบูรณาการความรู้ทางอิเล็กทรอนิกส์-ไมโครคอนโทรลเลอร์-การเขียนโปรแกรมคอมพิวเตอร์-การทดลองทางวิทยาศาสตร์ในระดับมัธยมศึกษา อาชีวศึกษา และปริญญาตรี

ดำเนินการจัดพิมพ์และจำหน่ายโดย

บริษัท อินโนเวทีฟ เอ็กเพอริเมนต์ จำกัด

108 ซ.สุขุมวิท 101/2 ถ.สุขุมวิท แขวงบางนา เขตบางนา กรุงเทพฯ 10260

โทรศัพท์ 0-2747-7001-4

โทรสาร 0-2747-7005

รายละเอียดที่ปรากฏในหนังสือเล่มนี้ได้ผ่านการตรวจทานอย่างละเอียดและถี่ถ้วน เพื่อให้มีความสมบูรณ์และถูกต้องมากที่สุดภายใต้เงื่อนไขและเวลาที่พื้มีก่อนการจัดพิมพ์เผยแพร่ ความเสียหายอันอาจเกิดจากการนำข้อมูลในหนังสือเล่มนี้ไปใช้ ทางบริษัท อินโนเวทีฟ เอ็กเพอริเมนต์ จำกัด มิได้มีภาระในการรับผิดชอบแต่ประการใด ความผิดพลาดคลาดเคลื่อนที่อาจมีและได้รับการจัดพิมพ์เผยแพร่ออกไปนั้น ทางบริษัทฯ จะพยายามชี้แจงและแก้ไขในการจัดพิมพ์ครั้งต่อไป

การเรียนรู้ไมโครคอนโทรลเลอร์กับ STEM ศึกษา

ในการเรียนรู้วิทยาการสมัยใหม่มีแนวคิดหนึ่งที่ได้รับการยอมรับอย่างกว้างขวาง นั่นคือ แนวคิดด้าน STEM ศึกษา หรือ STEM Education ประกอบด้วย วิทยาศาสตร์ (Science), เทคโนโลยี (Technology), วิศวกรรม (Engineering) และคณิตศาสตร์ (Mathematic) เพื่อให้ผู้เรียนได้ศึกษากระบวนการทางวิทยาศาสตร์และเทคโนโลยีสมัยใหม่ที่จะต้องอาศัยทักษะพื้นฐานสำคัญทั้ง 4 ด้านมาบูรณาการกัน เพื่อก่อให้เกิดความรู้ ความเข้าใจในเรื่องนั้นๆ อย่างถูกต้อง

ไมโครคอนโทรลเลอร์ (microcontroller) และระบบสมองกลฝังตัว (embedded system) ถูกนำมาเป็นส่วนหนึ่งของกระบวนการ STEM ศึกษาในระดับมัธยมศึกษา เนื่องจากในสังคมสมัยใหม่มีอุปกรณ์เครื่องใช้จำนวนมากในชีวิตประจำวันที่มีวงจรและอุปกรณ์อิเล็กทรอนิกส์เข้าไปเกี่ยวข้อง การเรียนรู้ของนักเรียนเพื่อทำความเข้าใจและต่อยอดไปสู่การพัฒนาเป็นโครงงานวิทยาศาสตร์สมัยใหม่ที่มีความเกี่ยวข้องกับระบบควบคุมอัตโนมัติ อันมีไมโครคอนโทรลเลอร์เป็นส่วนหนึ่งของหัวใจหลักจึงเป็นเป้าหมายหนึ่งที่แสดงถึงความสัมฤทธิ์ผลของการเรียนรู้ STEM ศึกษา

บอร์ดไมโครคอนโทรลเลอร์ POP-32 (จาก INEX : www.inex.co.th) และซอฟต์แวร์ระบบเปิด (open source) ที่ใช้ในการพัฒนาโปรแกรมควบคุมด้วยภาษา C/C++ ที่ชื่อ Arduino (www.arduino.cc) เป็นหนึ่งในทางเลือกเพื่อสนับสนุนการเรียนรู้ไมโครคอนโทรลเลอร์อย่างง่าย ก่อให้เกิดกระบวนการคิดเพื่อพัฒนาโครงงานวิทยาศาสตร์สมัยใหม่ที่มีความเกี่ยวข้องกับระบบควบคุมอัตโนมัติขนาดเล็ก เพื่อให้ผู้เรียนสามารถบรรลุจุดประสงค์ของการเรียนรู้ STEM ศึกษา ได้อย่างมีประสิทธิภาพ

การเรียนรู้ไมโครคอนโทรลเลอร์ด้วยบอร์ด POP-32 และซอฟต์แวร์ Arduino ช่วยให้ผู้เรียนและครู-อาจารย์สามารถเขียนโปรแกรมควบคุมอุปกรณ์ฮาร์ดแวร์ด้วยภาษา C/C++ ได้ไม่ยาก เข้าใจถึงความสัมพันธ์และการทำงานร่วมกันระหว่างการเชื่อมต่อทางฮาร์ดแวร์และกระบวนการคิดทางซอฟต์แวร์ มีการอ่านค่าเข้ามาในระบบ ทำการประมวลผล คิด วิเคราะห์ แล้วส่งคำสั่งหรือสั่งการให้ส่งค่าออกไปยังอุปกรณ์ภายนอก เพื่อควบคุมการทำงานหรือแสดงผล ซึ่งนั่นก็คือ หลักการเบื้องต้นของการพัฒนาโครงงานวิทยาศาสตร์ประยุกต์สมัยใหม่ที่สอดคล้องกับแนวคิด STEM ศึกษา

แนวคิด STEM ศึกษาได้จำกัดขอบเขตของผู้เรียนไว้ในระดับมัธยมศึกษาเท่านั้น ในระดับอาชีวศึกษาหรือระดับปริญญาตรีช่วงต้นก็สามารถเรียนรู้และนำแนวคิด STEM ศึกษาไปต่อยอดได้ อาจกล่าวให้เข้าใจง่าย ๆ ว่า STEM ศึกษาคือ สารความรู้ด้านพื้นฐานทางวิศวกรรมก็ได้ เพราะมิได้จำกัดเฉพาะด้านไฟฟ้า อิเล็กทรอนิกส์ คอมพิวเตอร์ เท่านั้น STEM มีฐานความรู้ที่กว้างไปถึงความรู้ทางกลศาสตร์ ทักษะการใช้เครื่องมือ การคำนวณ สถิติ เคมี ชีววิทยา ดังนั้นการเรียนรู้ไมโครคอนโทรลเลอร์และฝึกหัดการเขียนโปรแกรมด้วยภาษา C/C++ จึงนำไปประยุกต์ใช้ให้เข้ากับองค์ความรู้พื้นฐานต่างๆ ได้อย่างลงตัว และตอบโจทย์การพัฒนากระบวนการคิดได้เป็นที่ประจักษ์

ชัยวัฒน์ ลิ้มพรจิตรวิไล

บรรณาธิการ

คำชี้แจงจากคณะผู้เขียน/เรียบเรียง

การนำเสนอข้อมูลเกี่ยวกับข้อมูลทางเทคนิคและเทคโนโลยีในหนังสือเล่มนี้เกิดจากความต้องการที่จะอธิบายกระบวนการและหลักการทำงานของอุปกรณ์ในภาพรวมด้วยถ้อยคำที่ง่ายเพื่อสร้างความเข้าใจแก่ผู้อ่าน ดังนั้นการแปลคำศัพท์ทางเทคนิคหลายๆ คำอาจไม่ตรงตามข้อบัญญัติของราชบัณฑิตยสถาน และมีหลายๆ คำที่ยังไม่มีการบัญญัติอย่างเป็นทางการ คณะผู้เขียนจึงขออนุญาตบัญญัติศัพท์ขึ้นมาใช้ในการอธิบาย โดยมีข้อจำกัดเพื่ออ้างอิงในหนังสือเล่มนี้เท่านั้น

สาเหตุหลักของข้อชี้แจงนี้มาจากการรวบรวมข้อมูลของอุปกรณ์ในระบบสมองกลฝังตัวและเทคโนโลยีหุ่นยนต์สำหรับการศึกษาเพื่อนำมาเรียบเรียงเป็นภาษาไทยนั้นทำได้ไม่ง่ายนัก ทางคณะผู้เขียนต้องทำการรวบรวมและทดลองเพื่อให้แน่ใจว่า ความเข้าใจในกระบวนการทำงานต่างๆ นั้นมีความคลาดเคลื่อนน้อยที่สุด

เมื่อต้องการเรียบเรียงออกมาเป็นภาษาไทย ศัพท์ทางเทคนิคหลายคำมีความหมายที่ทับซ้อนกันมาก การบัญญัติศัพท์จึงเกิดจากการปฏิบัติจริงร่วมกับความหมายทางภาษาศาสตร์ ดังนั้น หากมีความคลาดเคลื่อนหรือผิดพลาดเกิดขึ้น ทางคณะผู้เขียนขออภัยและหากได้รับคำอธิบายหรือชี้แนะจากท่านผู้รู้จะได้ทำการชี้แจงและปรับปรุงข้อผิดพลาดที่อาจมีเหล่านั้นโดยเร็วที่สุด

ทั้งนี้เพื่อให้การพัฒนาสื่อทางวิชาการ โดยเฉพาะอย่างยิ่งกับความรู้ของเทคโนโลยีสมัยใหม่สามารถดำเนินไปได้อย่างต่อเนื่อง ภายใต้การมีส่วนร่วมของผู้รู้ในทุกภาคส่วน

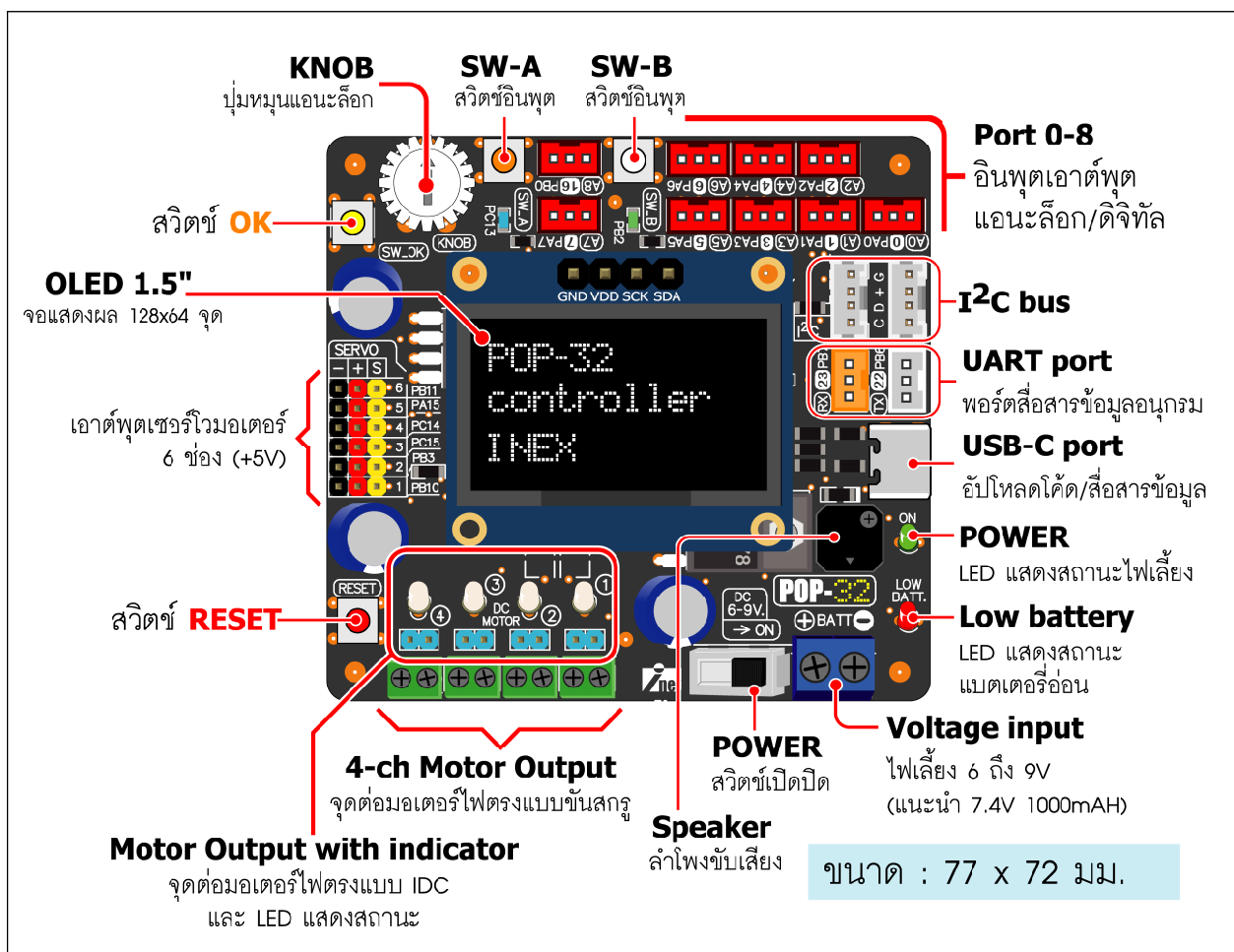
สารบัญ

บทที่ 1	แนะนำบอร์ด POP-32	7
บทที่ 2	แนะนำและติดตั้งซอฟต์แวร์ Arduino IDE สำหรับบอร์ด POP-32	11
บทที่ 3	โครงสร้างโปรแกรมของ Arduino	29
บทที่ 4	ฟังก์ชันพื้นฐานของ Arduino และตัวอย่างคำสั่งสำหรับการพัฒนาโปรแกรม.....	63
บทที่ 5	ไลบรารีสำหรับการพัฒนาโปรแกรมของบอร์ด POP-32	89

บทที่ 1

คุณสมบัติของบอร์ด POP-32

POP-32 เป็นบอร์ดควบคุมที่ใช้ไมโครคอนโทรลเลอร์ 32 บิต เบอร์ **STM32F103CBT6** ของ STMicroelectronics (www.st.com) มีวงจรเชื่อมต่อพอร์ต USB เพื่อใช้ในการสื่อสารข้อมูลและดาวน์โหลดโปรแกรมได้ในตัว โดยไม่ต้องใช้สายสัญญาณหรืออุปกรณ์แปลงสัญญาณใดๆ เพิ่มเติม จึงทำให้การใช้งานง่ายและสะดวกมาก รวมถึง **POP-32** ได้เลือกใช้ฮาร์ดแวร์และซอฟต์แวร์จากโครงการไมโครคอนโทรลเลอร์ระบบเปิด (โอเพ่นซอร์ส : open source) ที่ชื่อ **Arduino** (www.arduino.cc) มาปรับปรุงต่อ โดยมีไลบรารีฟังก์ชันภาษาซีสำหรับติดต่อกับฮาร์ดแวร์จำนวนมากไว้ให้ ทำให้เขียนโปรแกรมสั่งงานอุปกรณ์ต่างๆ ได้ง่าย โดยไม่ต้องศึกษาลงไปในรายละเอียดของไมโครคอนโทรลเลอร์มากนัก



รูปที่ 1-1 ส่วนประกอบของบอร์ด POP-32

8 • POP-32 บอร์ดไมโครคอนโทรลเลอร์เพื่อการเรียนรู้และพัฒนาโครงงานระบบสมองกลฝังตัว

ส่วนประกอบทั้งหมดของบอร์ด **POP-32** แสดงในรูปที่ 1-1 มีคุณสมบัติโดยสรุปดังนี้

- ใช้ไมโครคอนโทรลเลอร์ขนาด 32 บิตเบอร์ STM32F103CBT6 มีหน่วยความจำแฟลช 128KB โปรแกรมใหม่ได้ 10,000 ครั้ง มีหน่วยความจำข้อมูลแรม 20KB สัญญาณนาฬิกา 20MHz จากเซรามิกเรโซเนเตอร์
- จุดต่อพอร์ตแบบ JST 3 ขา 11 จุดสำหรับต่ออุปกรณ์ตรวจจับและอุปกรณ์ต่อพ่วง
- มี LED แสดงสถานะไฟเลี้ยง, แจ้งเตือนแบตเตอรี่อ่อน และสถานะการเชื่อมต่อพอร์ต USB
- มีสวิตช์ RESET
- มีจุดต่อพอร์ต USB สำหรับดาวน์โหลดโปรแกรมและสื่อสารข้อมูลกับคอมพิวเตอร์
- มีจุดต่อไฟเลี้ยงผ่านทางจุดต่อสายแบบขันสกรู รับไฟเลี้ยง 6 ถึง 9V มีสวิตช์เปิด-ปิด เพื่อตัดต่อไฟเลี้ยง
- ใช้กับแบตเตอรี่ลิเทียมโพลิเมอร์ได้สูงสุด 2 เซล (7.4V สูงสุดไม่เกิน 8.4V)
- มีวงจรควบคุมไฟเลี้ยง 5.3V เพื่อจ่ายให้กับไมโครคอนโทรลเลอร์, จอแสดงผล OLED และจุดต่อพอร์ตอินพุตเอาต์พุตหลัก
- จุดต่อพอร์ตอินพุตเอาต์พุตดิจิทัลหรืออะนาล็อก 9 ช่อง คือ A0 ถึง A8 (ตรงกับขา PA0 ถึง PA7 และ PB0) รองรับการทำงานเป็นขาอินพุตรับสัญญาณอินเทอร์รัปต์จากภายนอก
- จุดต่อพอร์ตดิจิทัลรองรับระบบบัส I²C 2 ชุดคือ จุดต่อ SDA และ SCL ต่อพ่วงกัน โดยใช้คอนเน็กเตอร์แบบ PH4 จัดหาแบบ GROVE
- มีจุดต่อพอร์ตสื่อสารข้อมูลอนุกรม UART 1 ชุดคือ จุดต่อขาพอร์ต PB7 (RxD) และ PB6 (TxD)
- มีวงจรขับเคลื่อนมอเตอร์ไฟตรง 4 ช่อง พร้อม LED แสดงสถานะการทำงาน ใช้จุดต่อแบบคอนเน็กเตอร์ IDC 2 ขาและแบบเทอร์มินอลบล็อก 2 ขาต่อช่อง รองรับมอเตอร์ไฟตรง 3 ถึง 9V มีความสามารถขับเคลื่อนกระแสไฟฟ้าได้ต่อเนื่อง 1.5A ต่อช่อง สูงสุดไม่เกิน 2A มีวงจรจำกัดกระแสไฟฟ้าเกินเพื่อป้องกันความเสียหายที่อาจเกิดขึ้นกับตัวบอร์ด
- มีจุดต่อขาพอร์ตของไมโครคอนโทรลเลอร์สำหรับขับเซอร์โวมอเตอร์ 6 ช่องคือ จุดต่อ SERVO1 (PB10), SERVO2 (PB3), SERVO3 (PC15), SERVO4 (PC14), SERVO5 (PA15) และ SERVO6 (PB11)

- มีลำโพง piezo สำหรับขับเสียง โดยต่อกับขาพอร์ต PB5
- มีโมดูลแสดงผลแบบ OLED ขนาด 1.5 นิ้ว ความละเอียด 128 x 64 จุด แสดงภาพกราฟิกลายเส้นและพื้นสี แสดงผลเป็นตัวอักษรขนาดปกติ (5 x 7 จุด) ได้ 21 ตัวอักษร 8 บรรทัด (21 x 8) ติดต่อผ่านบัส I²C
- มีสวิทช์กดติดปล่อยดับพร้อมใช้งาน 3 จุด
 1. **ปุ่ม OK** (ปุ่มสีเหลือง) ต่อร่วมกับตัวต้านทานปรับค่าได้ (KNOB) ซึ่งเชื่อมต่อไปยังขาพอร์ต PB1 ทำให้อ่านค่าสัญญาณดิจิทัลและอะนาล็อกได้ในขาพอร์ตเดียวกัน
 2. **สวิทช์ A** (ปุ่มสีส้ม) ต่อกับขาพอร์ต PC13 พร้อมมีตัวต้านทานต่อพูลอัพ และต่อร่วมกับ LED สีฟ้าเพื่อแสดงสถานะลอจิก
 5. **สวิทช์ B** (ปุ่มสีขาว) ต่อกับขาพอร์ต PB2 พร้อมมีตัวต้านทานต่อพูลอัพ และต่อร่วมกับ LED สีเขียวเพื่อแสดงสถานะลอจิก
- มีวงจรตัวต้านทานปรับค่าได้ **KNOB** พร้อมปุ่มปรับเพื่อใช้ในการทดสอบวงจรแปลงสัญญาณแอนะล็อกเป็นดิจิทัลบนบอร์ด ต่อกับขาพอร์ต PB1 โดยต่อร่วมใช้งานกับสวิทช์กด **OK**



บทที่ 2

การติดตั้งซอฟต์แวร์ Arduino IDE

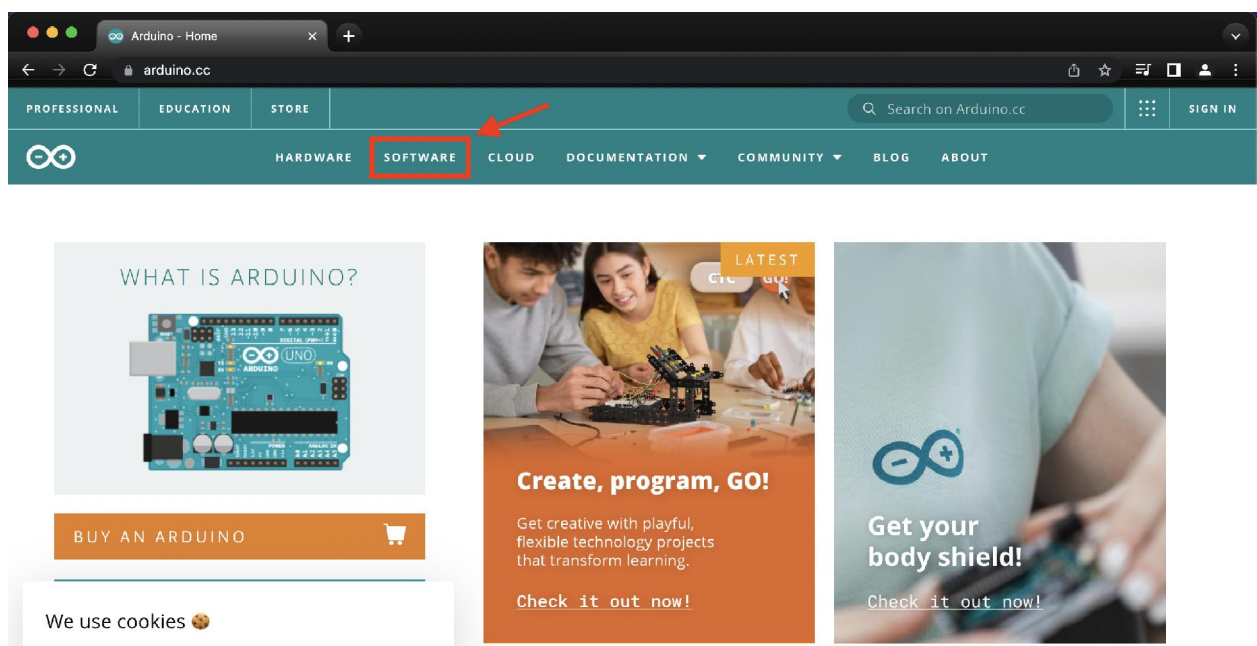
การพัฒนาโปรแกรมให้กับบอร์ด **POP-32** ในที่นี้เลือกใช้โปรแกรมภาษา C/C++ โดยใช้แพลตฟอร์มระบบเปิดที่ชื่อ Arduino ซอฟต์แวร์หลักคือ Arduino IDE ที่จัดการทั้งกระบวนการจบด้วยโปรแกรมเพียงตัวเดียว ตั้งแต่มีส่วนของการสร้างโค้ดภาษา C/C++ มีไลบรารีมาตรฐาน ตัวแปลภาษา C/C++ หรือคอมไพเลอร์ ลิงเกอร์ และส่วนของการอัปโหลดโค้ดไปเขียนลงในหน่วยความจำโปรแกรมของไมโครคอนโทรลเลอร์ ในบทนี้นำเสนอขั้นตอนการติดตั้งโปรแกรม Arduino ไปจนถึงการทดสอบใช้งานเบื้องต้น

2.1 การติดตั้งซอฟต์แวร์ Arduino IDE บนระบบปฏิบัติการ Windows

2.1.1 การติดตั้ง Arduino IDE และไดรเวอร์ USB

มีขั้นตอนดังนี้

(1) เชื่อมต่อคอมพิวเตอร์เข้ากับเครือข่ายอินเทอร์เน็ต จากนั้นเปิดเว็บเบราว์เซอร์ไปยังเว็บไซต์ Arduino ที่ <https://www.arduino.cc> จากนั้นคลิกที่หัวข้อ SOFTWARE ตามรูปที่ 2-1



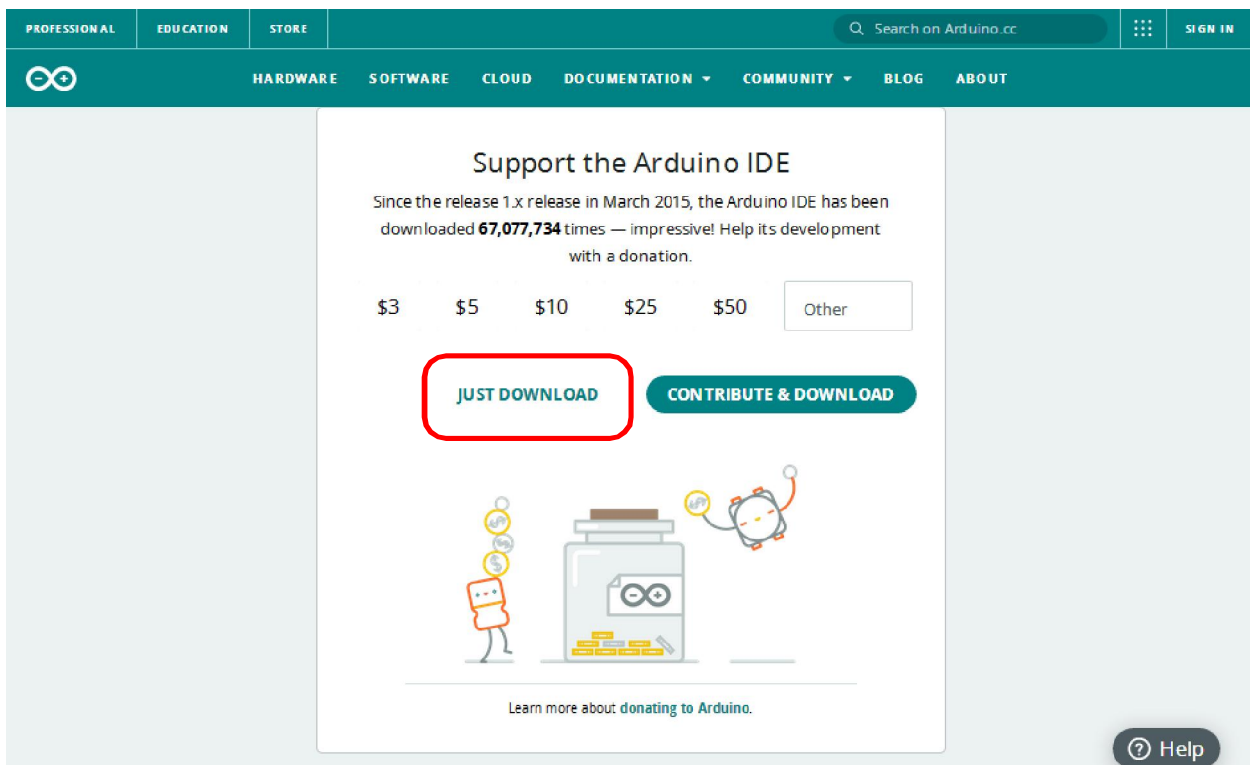
รูปที่ 2-1 หน้าเว็บของ Arduino



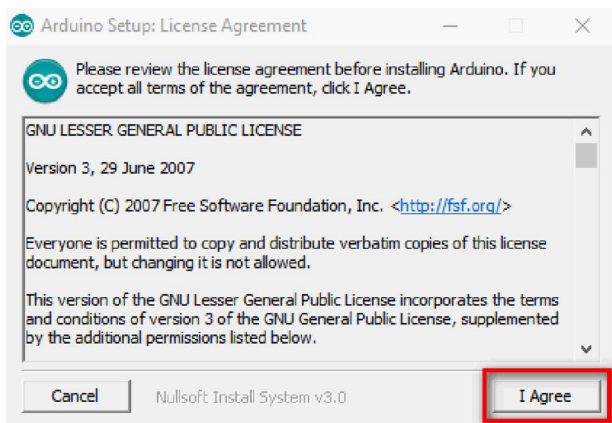
รูปที่ 2-2 แสดงการเลือกดาวน์โหลดไฟล์ติดตั้งซอฟต์แวร์ Arduino IDE เวอร์ชัน 1.8.X

(2) เลื่อนแถบลงมาด้านล่าง ค้นหาหัวข้อ **Legacy IDE** เลือกรายการ **Windows Win 7 and Newer** เพื่อดาวน์โหลดไฟล์ติดตั้งซอฟต์แวร์ Arduino IDE เวอร์ชัน 1.8.X ตามรูปที่ 2-2

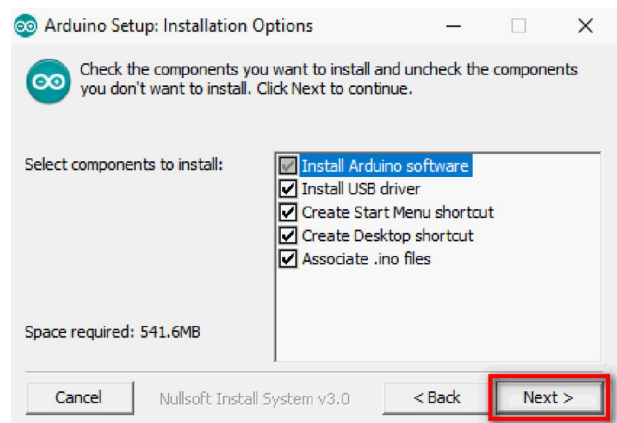
(3) จากนั้นจะปรากฏหน้าต่างตามรูปที่ 2-3 คลิกที่ปุ่ม **JUST DOWNLOAD** เพื่อดาวน์โหลดไฟล์ติดตั้ง



รูปที่ 2-3 แสดงปุ่ม JUST DOWNLOAD สำหรับดาวน์โหลดซอฟต์แวร์ Arduino IDE



รูปที่ 2-4 แสดงหน้าต่างแจ้งยอมรับลิขสิทธิ์ก่อน
เริ่มต้นการติดตั้งโปรแกรม Arduino IDE



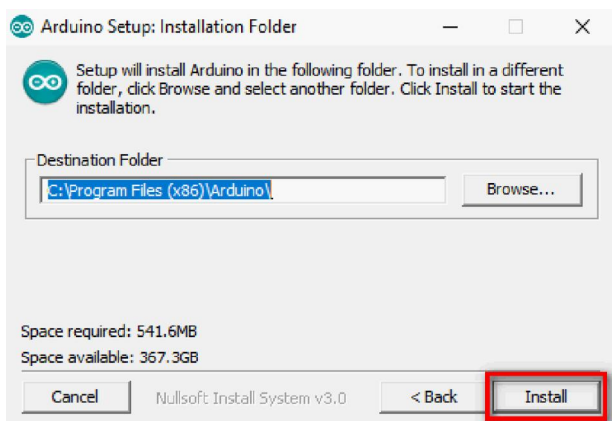
รูปที่ 2-5 หน้าต่างเลือกส่วนประกอบของโปรแกรม
Arduino IDE ที่ต้องการติดตั้ง

(4) ทำการดับเบิลคลิกที่ไฟล์ติดตั้งซอฟต์แวร์ Arduino IDE จะปรากฏหน้าต่างสำหรับการเริ่มต้นการติดตั้งโปรแกรม Arduino IDE ตามรูปที่ 2-4

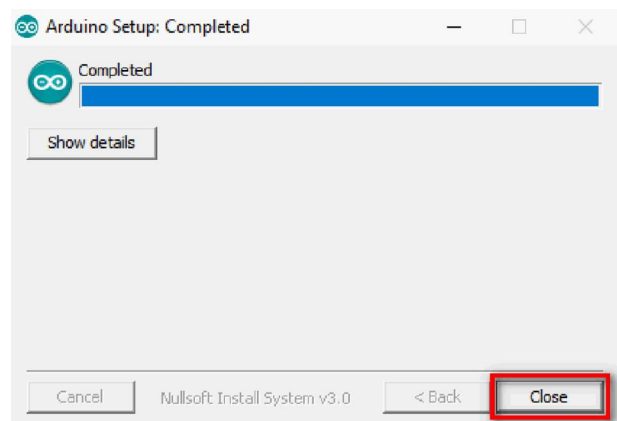
(5) จากนั้นจะปรากฏหน้าต่างเลือกส่วนประกอบของโปรแกรมที่ต้องการติดตั้งแสดงขึ้นมาตามรูปที่ 2-5 คลิกปุ่ม Next

(6) หน้าต่างเลือกโฟลเดอร์สำหรับติดตั้งโปรแกรมดังรูปที่ 2-6 ปรากฏขึ้นมา คลิกปุ่ม Install เพื่อเริ่มต้นการติดตั้งโปรแกรม Arduino

(7) จากนั้นรอนกระทำการติดตั้งโปรแกรมเสร็จสมบูรณ์ ตามรูปที่ 2-7 คลิกปุ่ม Close เพื่อปิดหน้าต่างนี้



รูปที่ 2-6 หน้าต่างสำหรับเลือกโฟลเดอร์ติดตั้ง
โปรแกรม



รูปที่ 2-7 แสดงการติดตั้งโปรแกรม Arduino IDE
เสร็จสมบูรณ์

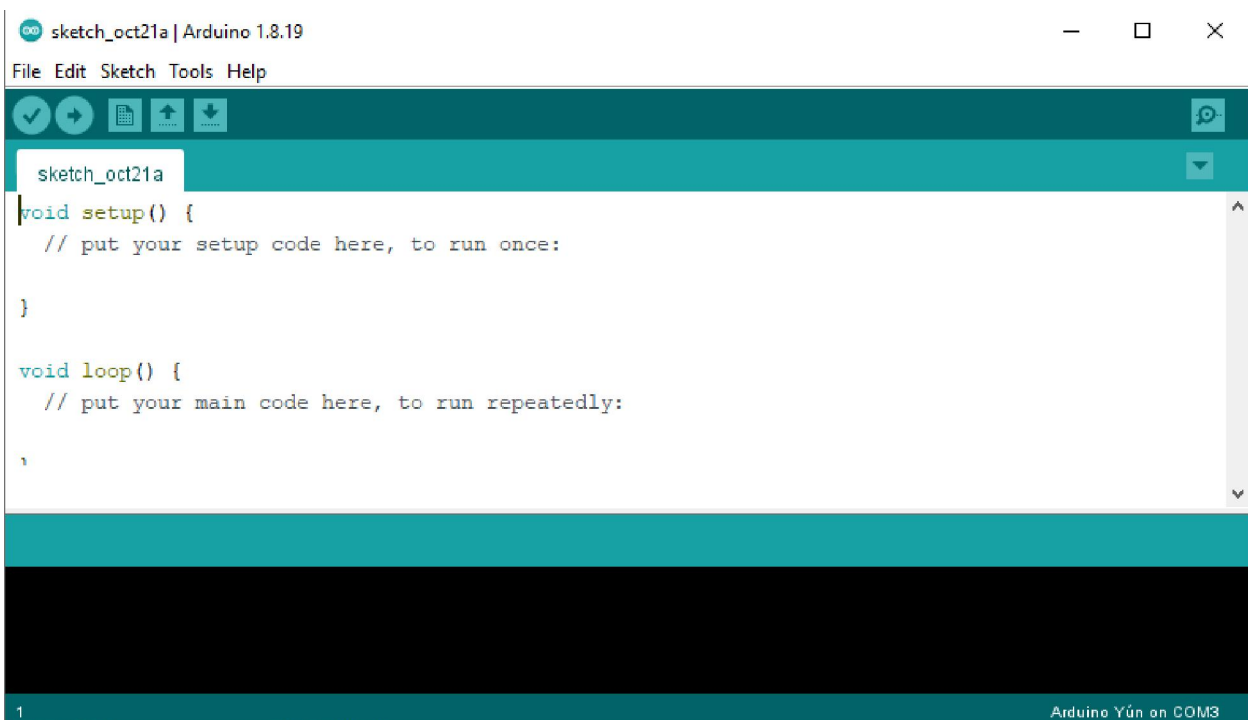
2.1.2 การติดตั้งฮาร์ดแวร์และไลบรารีสำหรับใช้งานบอร์ด POP-32

เมื่อติดตั้งโปรแกรม Arduino IDE เรียบร้อยแล้ว ลำดับถัดไปคือ การทำให้โปรแกรม Arduino IDE ทำงานกับบอร์ด **POP-32** ได้ด้วยการเพิ่มข้อมูลทางฮาร์ดแวร์และติดตั้งไลบรารีสำหรับการพัฒนาโปรแกรมด้วยภาษา C/C++ ให้กับ Arduino IDE รวมถึงทำให้เครื่องมือในการอัปโหลดโปรแกรมของ Arduino IDE สามารถทำการอัปโหลดโค้ดมายังบอร์ด **POP-32** ได้

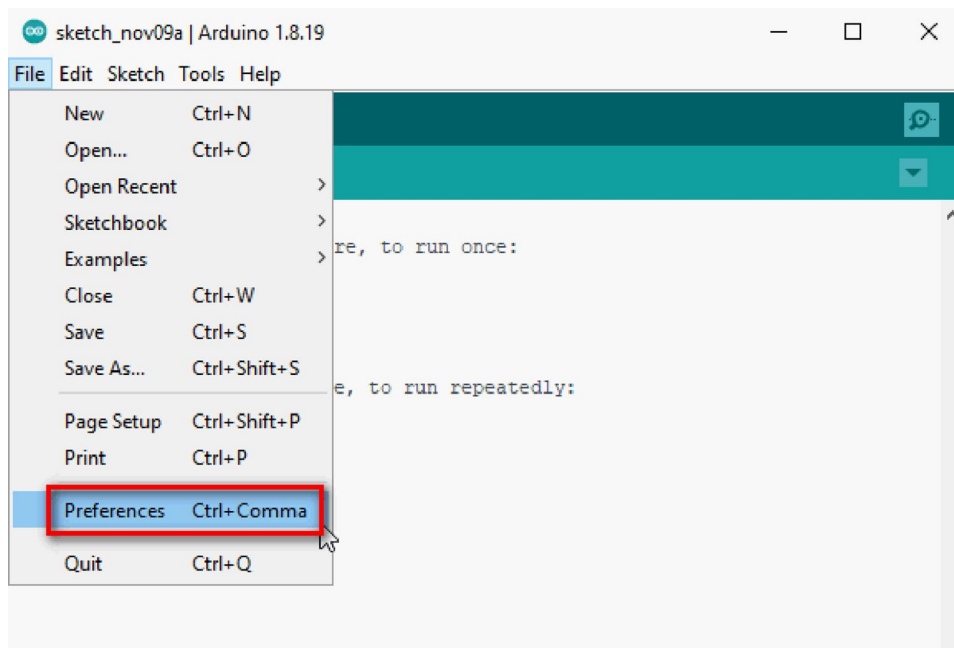
มีขั้นตอนดังนี้

- (1) เปิดโปรแกรม Arduino IDE ปรากฏหน้าต่างหลักของโปรแกรม **Arduino IDE** ดังรูปที่ 2-8
- (2) เลือกเมนู **File > Preferences...** ตามรูปที่ 2-9
- (3) หน้าต่าง **Preferences** ปรากฏขึ้นมาตามรูปที่ 2-10 ทำการตั้งค่าดังนี้
 - คลิกที่รายการ **Verify code after upload** เพื่อนำเครื่องหมายถูกออก
 - คลิกที่รายการ **Check for updates on startup** เพื่อนำเครื่องหมายถูกออก
 - ที่รายการ **Additional Boards Manager URLs:** กำหนดค่าเป็น

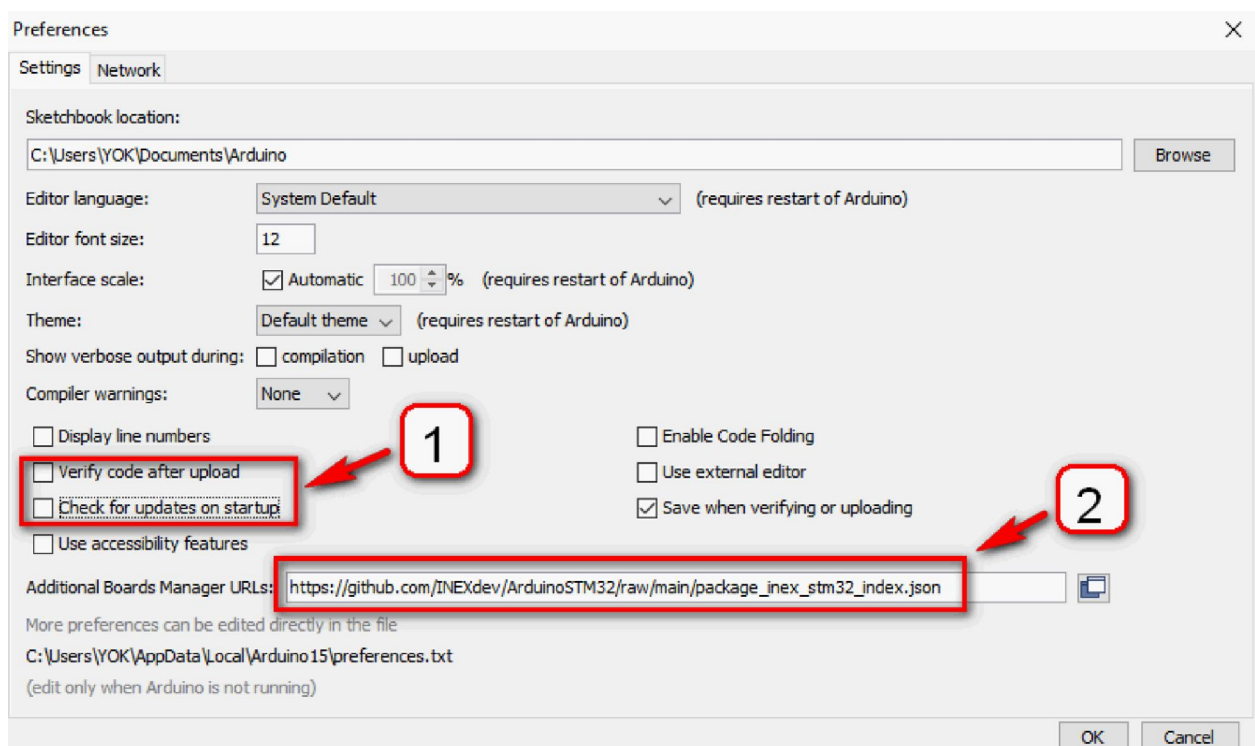
https://github.com/INEXdev/ArduinoSTM32/raw/main/package_inex_stm32_index.json จากนั้นคลิกปุ่ม **OK** เพื่อยืนยันการตั้งค่า



รูปที่ 2-8 หน้าต่างหลักของโปรแกรม Arduino IDE



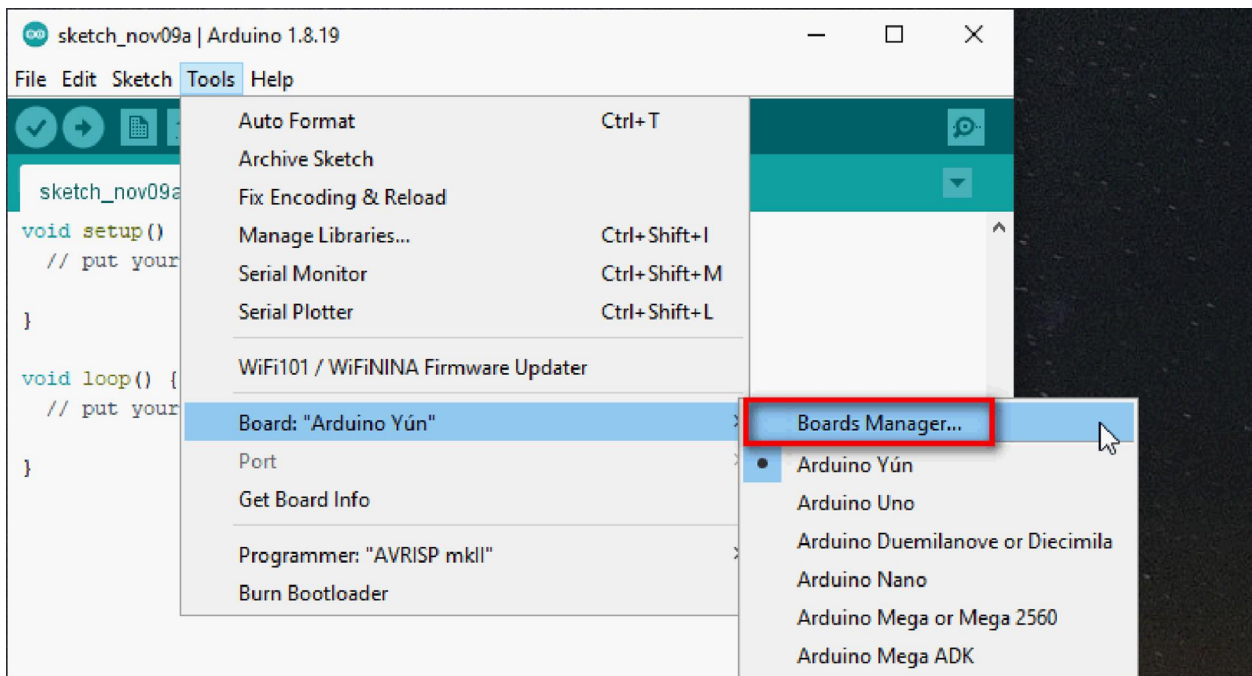
รูปที่ 2-9 การเลือกเปิดหน้าต่าง Preferences ของโปรแกรม Arduino IDE



รูปที่ 2-10 แสดงการตั้งค่าหน้าต่าง Preferences เพื่อเตรียมการติดตั้งข้อมูลของบอร์ด POP-32

(4) เลือกเมนู **Tools > Board:xxx > Boards Manager...** ตามรูปที่ 2-11

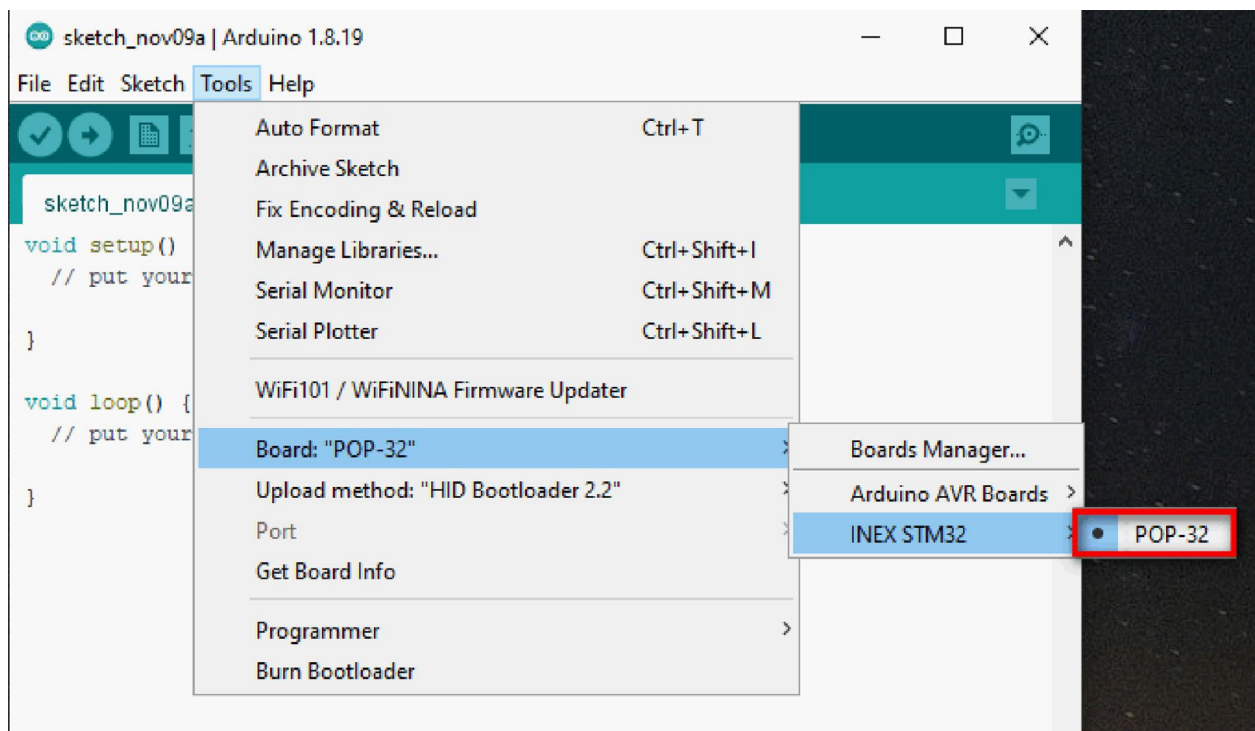
(5) หน้าต่าง **Boards Manager** ปรากฏขึ้นมาตามรูปที่ 2-12 ให้พิมพ์ค้นหาด้วยคำว่า **INEX** จะพบรายการตัวติดตั้งข้อมูลทางฮาร์ดแวร์ชื่อ **INEX_STM32** ซึ่งมีข้อมูลของบอร์ด **POP-32** รวมอยู่ด้วย จากนั้นคลิกปุ่ม **Install** เพื่อทำการติดตั้ง



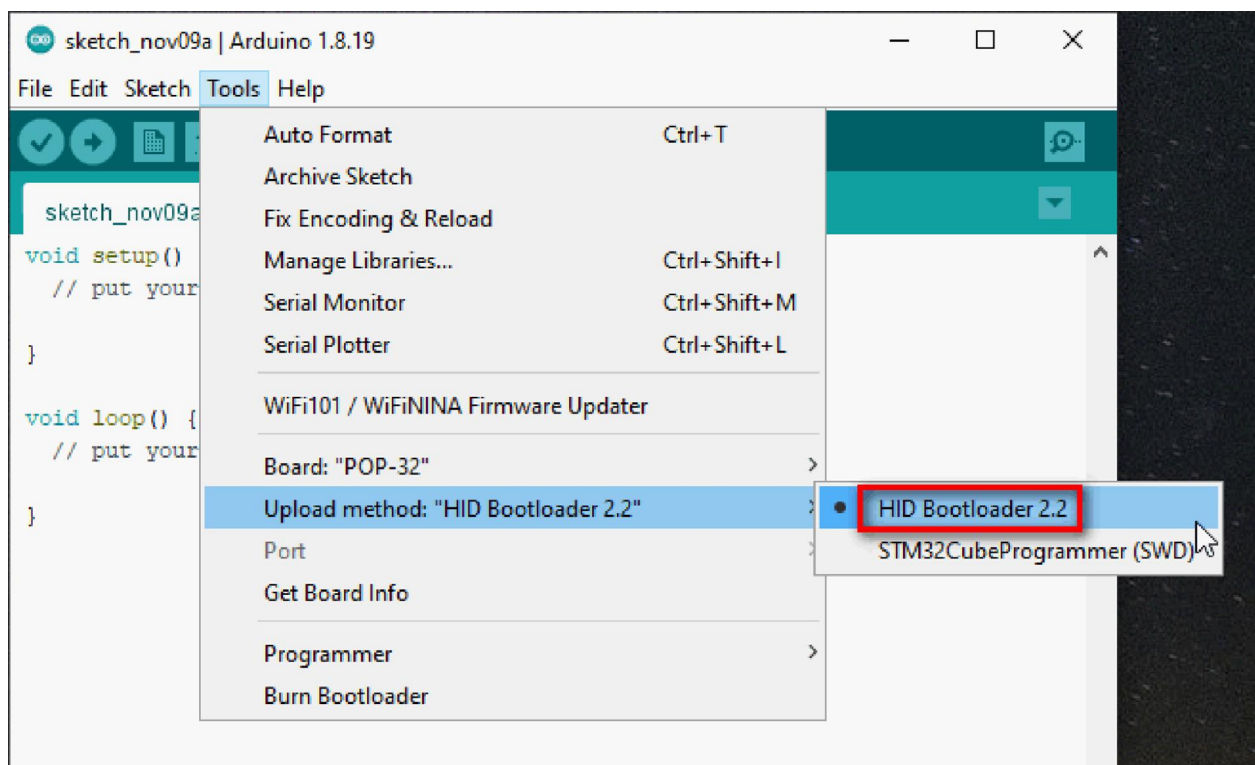
รูปที่ 2-11 แสดงการเลือกเมนู **Tools>Board:xxx>Boards Manager...**



รูปที่ 2-12 แสดงหน้าต่าง **Boards Manager** สำหรับติดตั้งไลบรารีและข้อมูลทางฮาร์ดแวร์ของ **INEX_STM32**



รูปที่ 2-13 แสดงการเลือกบอร์ด **POP-32**



รูปที่ 2-14 แสดงการเลือกวิธีการอัปโหลดโค้ดเป็น **HID Bootloader 2.2**

(6) จากนั้นเข้าสู่กระบวนการติดตั้งไลบรารี รอจนกระทั่งการติดตั้งเสร็จสมบูรณ์

ขั้นตอนถัดไปเป็นทดสอบอัปโหลดโปรแกรมไปยังบอร์ด **POP-32** เพื่อยืนยันว่าการติดตั้งข้อมูลฮาร์ดแวร์และไลบรารีของบอร์ด **POP-32** ให้กับโปรแกรม Arduino IDE ถูกต้องและพร้อมใช้งาน

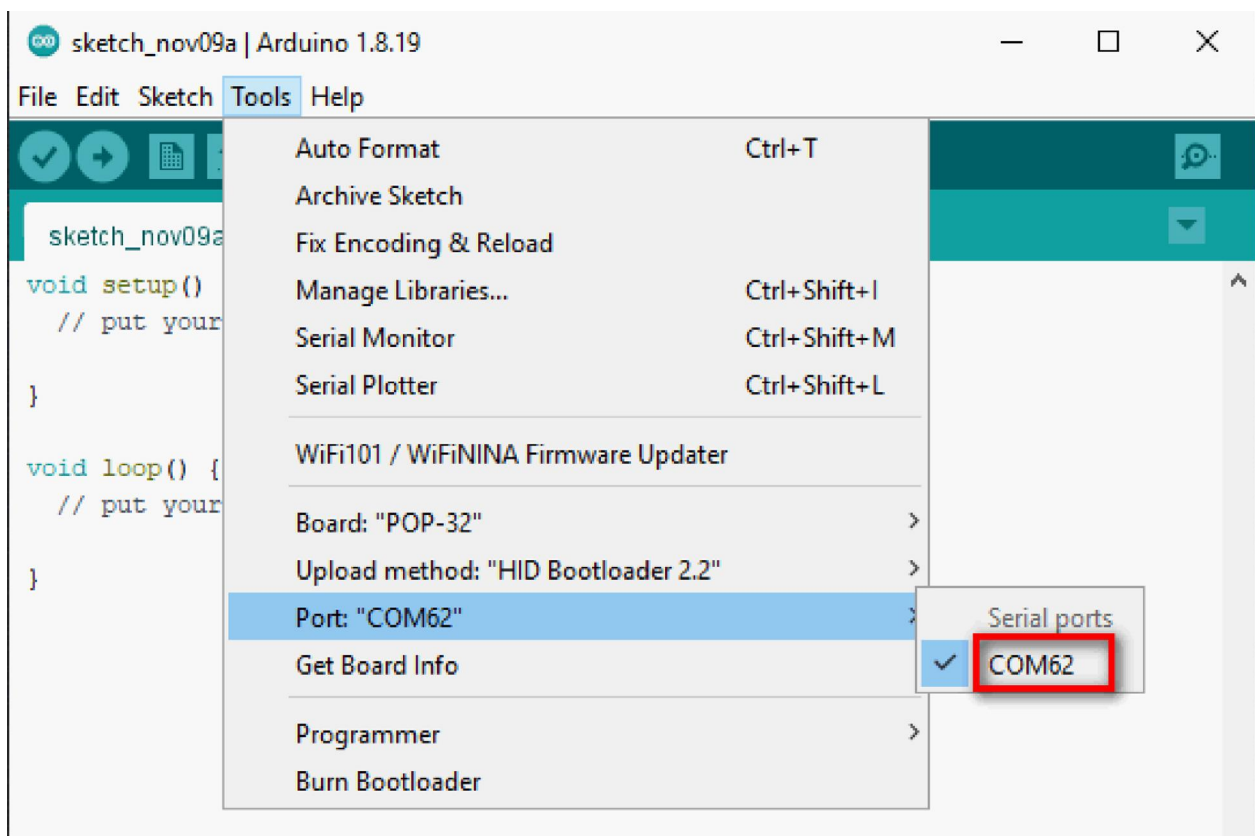
(7) ต่อสาย USB เพื่อเชื่อมต่อบอร์ด **POP-32** กับคอมพิวเตอร์

(8) จ่ายไฟเลี้ยงให้บอร์ด **POP-32** และเปิดสวิตช์ **POWER**

(9) ที่หน้าต่างหลักของโปรแกรม Arduino IDE เลือกเมนู **Tools > Board:xxx > INEX STM32 > POP-32** ตามรูปที่ 2-13

(10) เลือกวิธีการอัปโหลดโค้ด โดยเลือกเมนู **Tools > Upload method: “HID Bootloader 2.2” > HID Bootloader 2.2** ตามรูปที่ 2-14

(11) เลือกพอร์ตอนุกรมสำหรับการสื่อสารข้อมูลกับโปรแกรม Arduino IDE โดยเลือกเมนู **Tools > Port: “xxx” > XXXXX** ตามรูปที่ 2-15 โดยในส่วนนี้หมายเลขพอร์ตเชื่อมต่อพอร์ต USB ระบบปฏิบัติการจะเป็นตัวกำหนดให้ ในที่นี้คือ **COM62**



รูปที่ 2-15 แสดงการเลือกพอร์ตอนุกรมที่เชื่อมต่อผ่านพอร์ต USB ของบอร์ด **POP-32**

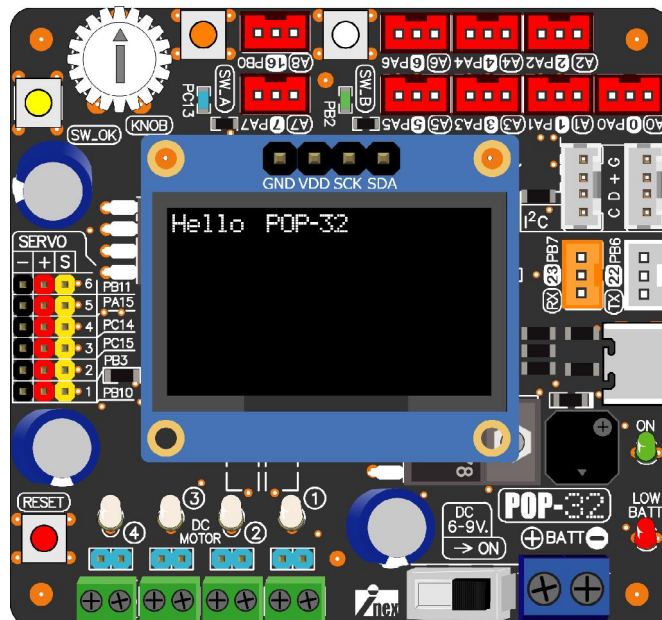
```
#include <POP32.h>
void setup()
{
  oled.text(0,0,"Hello POP-32");
  oled.show();
}
void loop()
{}
```

โปรแกรมที่ 2-1 โค้ดภาษา C/C++ สำหรับทดสอบการแสดงผลข้อความที่จอแสดงผลของบอร์ด **POP-32**

(12) กลับมายังหน้าต่างหลักของโปรแกรม Arduino IDE พิมพ์โค้ดตามโปรแกรมที่ 2-1 สำหรับทดสอบการทำงาน จากนั้นคลิกปุ่ม **Upload**

(13) เมื่อการอัปโหลดโปรแกรมสิ้นสุดลง บอร์ด **POP-32** จะทำงานทันที

*บอร์ด **POP-32** แสดงข้อความ Hello POP-32 ที่จอแสดงผล OLED*

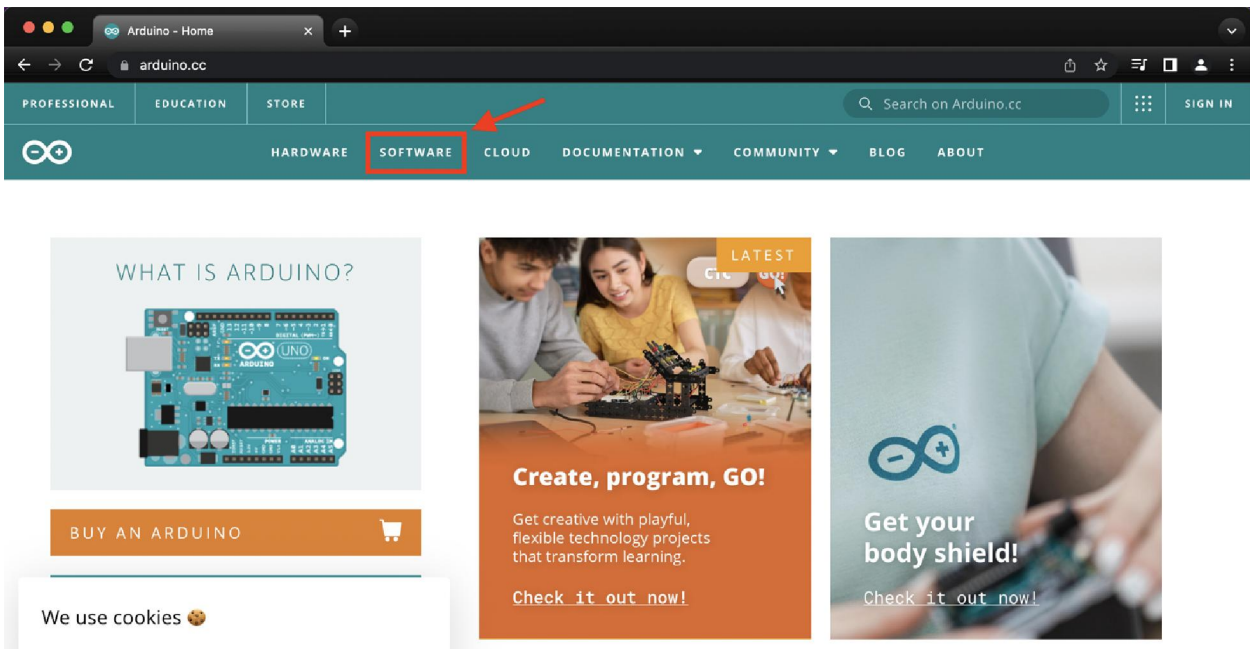


2.2 การติดตั้งซอฟต์แวร์ Arduino IDE บนระบบปฏิบัติการ MAC OS X

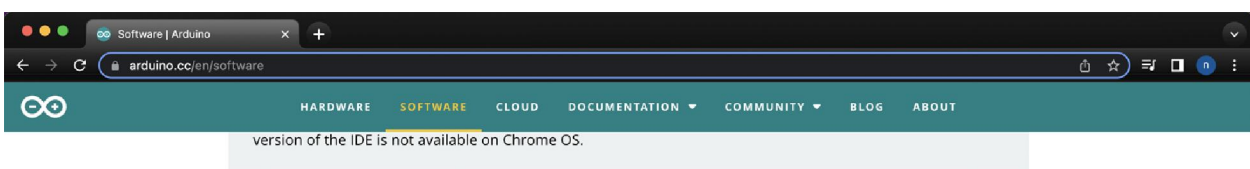
2.2.1 การติดตั้ง Arduino IDE

มีขั้นตอนดังนี้

(1) เชื่อมต่อคอมพิวเตอร์ MAC เข้ากับเครือข่ายอินเทอร์เน็ต จากนั้นเปิดเว็บเบราว์เซอร์ไปยังเว็บไซต์ **Arduino** ที่ <https://www.arduino.cc> จากนั้นคลิกที่หัวข้อ **SOFTWARE** ตามรูปที่ 2-16



รูปที่ 2-16 หน้าเว็บของ Arduino ผ่านเว็บเบราว์เซอร์บนคอมพิวเตอร์ MAC



Legacy IDE (1.8.X)



Arduino IDE 1.8.19

The open-source Arduino Software (IDE) makes it easy to write code and upload it to the board. This software can be used with any Arduino board.

Refer to the [Getting Started](#) page for Installation instructions.

SOURCE CODE

Active development of the Arduino software is [hosted by GitHub](#). See the instructions for [building the code](#). Latest release source code archives are available [here](#). The archives are PGP-signed so they can be verified using [this](#) gpg key.

DOWNLOAD OPTIONS

Windows Win 7 and newer

Windows ZIP file

Windows app Win 8.1 or 10 [Get](#)

Linux 32 bits

Linux 64 bits

Linux ARM 32 bits

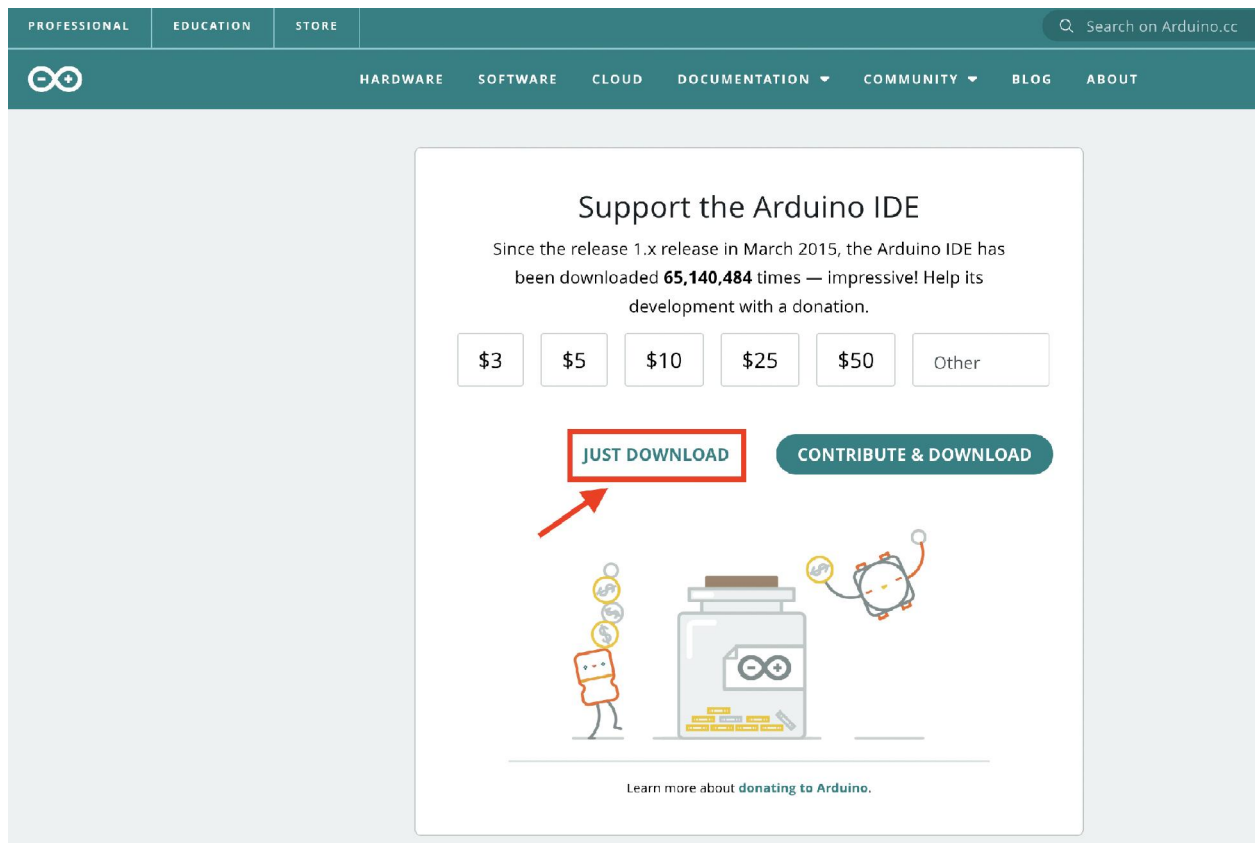
Linux ARM 64 bits

Mac OS X 10.10 or newer →

Release Notes

Checksums (sha512)

รูปที่ 2-17 แสดงลิงก์ดาวน์โหลดไฟล์ติดตั้งซอฟต์แวร์ Arduino IDE สำหรับระบบ ปฏิบัติการ MAC OS X

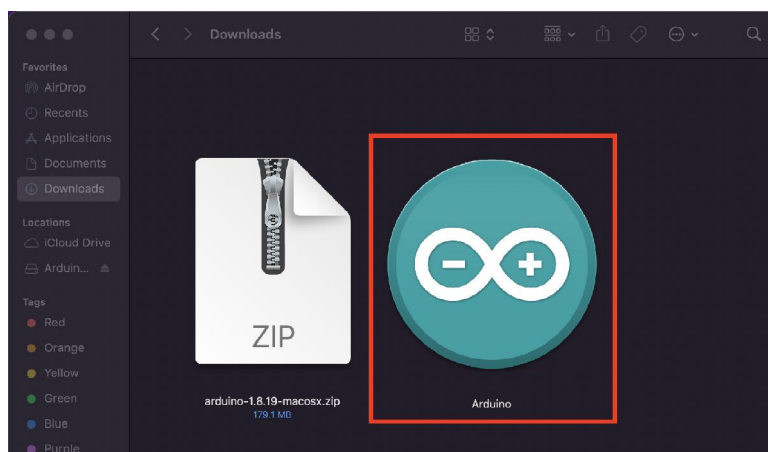


รูปที่ 2-18 แสดงปุ่ม **JUST DOWNLOAD** สำหรับดาวน์โหลดซอฟต์แวร์ **Arduino IDE**

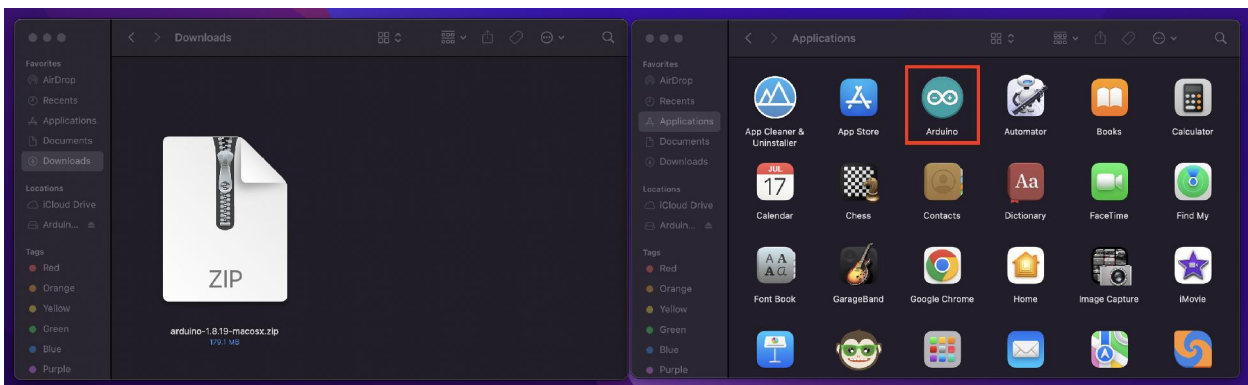
(2) เลื่อนแถบลงมาด้านล่าง ค้นหาหัวข้อ **Legacy IDE** จะพบลิงก์สำหรับดาวน์โหลดไฟล์ติดตั้งซอฟต์แวร์ **Arduino IDE** เวอร์ชัน 1.8.X สำหรับ **MAC OS X 10.10** ตามรูปที่ 2-17

(3) จากนั้นจะปรากฏหน้าต่างตามรูปที่ 2-18 คลิกที่ปุ่ม **JUST DOWNLOAD** เพื่อดาวน์โหลดไฟล์ติดตั้ง จะได้ไฟล์ **arduino-1.8.19-macosx.zip**

(4) ดับเบิลคลิกที่ไฟล์ .zip เพื่อแตกไฟล์ จากนั้นแอปพลิเคชัน **Arduino** จะถูกสร้างขึ้นตามรูปที่ 2-19



รูปที่ 2-19 แสดงแอปพลิเคชัน **Arduino** สำหรับใช้งาน



รูปที่ 2-20 แสดงแอปพลิเคชัน Arduino ที่ถูกย้ายไปจัดเก็บในส่วนของ Applications

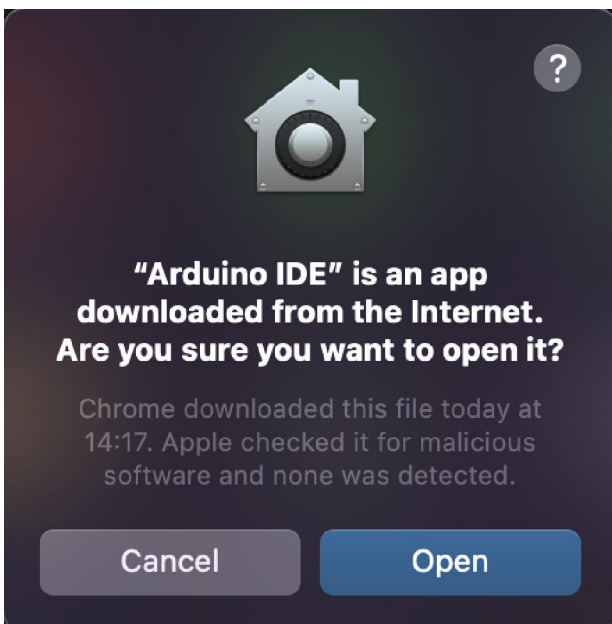
(5) เพื่อความเป็นระเบียบหมวดหมู่ของแอปพลิเคชัน แนะนำให้ย้ายแอปพลิเคชัน Arduino ไปไว้ในส่วนของ **Applications** ของคอมพิวเตอร์ MAC ตามรูปที่ 2-20 จากนั้นเรียกใช้งาน Arduino IDE

2.2.2 การติดตั้งฮาร์ดแวร์และไลบรารีสำหรับใช้งานบอร์ด POP-32

เมื่อติดตั้งโปรแกรม Arduino IDE เรียบร้อยแล้ว ลำดับถัดไปคือ การทำให้โปรแกรม Arduino IDE ทำงานกับบอร์ด POP-32 ได้ด้วยการเพิ่มข้อมูลทางฮาร์ดแวร์และติดตั้งไลบรารีสำหรับการพัฒนาโปรแกรมด้วยภาษา C/C++ ให้กับ Arduino IDE รวมถึงทำให้เครื่องมือในการอัปโหลดโปรแกรมของ Arduino IDE สามารถทำการอัปโหลดโค้ดมายังบอร์ด **POP-32** ได้

มีขั้นตอนดังนี้

(1) เปิดโปรแกรม Arduino IDE ขึ้นมา โดยในครั้งแรกสุดหลังจากการติดตั้งโปรแกรม ในกรณีที่ปรากฏหน้าต่างยืนยันการเปิดใช้งานตามรูปที่ 2-21 ให้คลิกปุ่ม **Open**



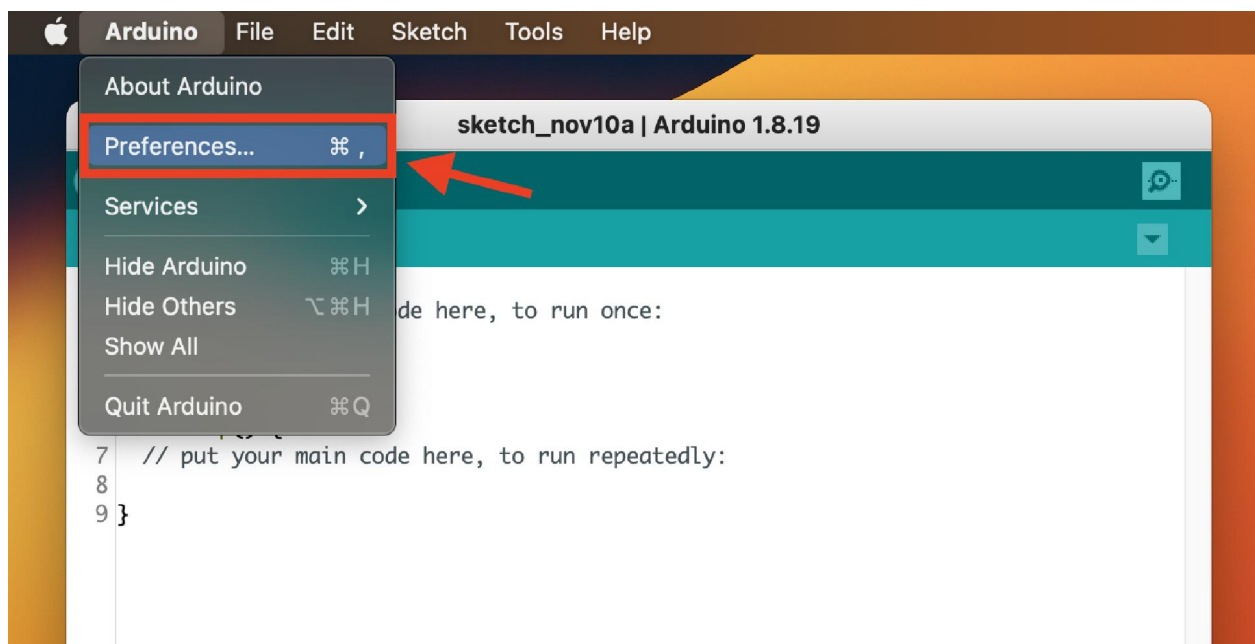
รูปที่ 2-21 แสดงหน้าต่างยืนยันการเปิดใช้งานโปรแกรม Arduino ในครั้งแรกหลังการติดตั้งใช้งาน



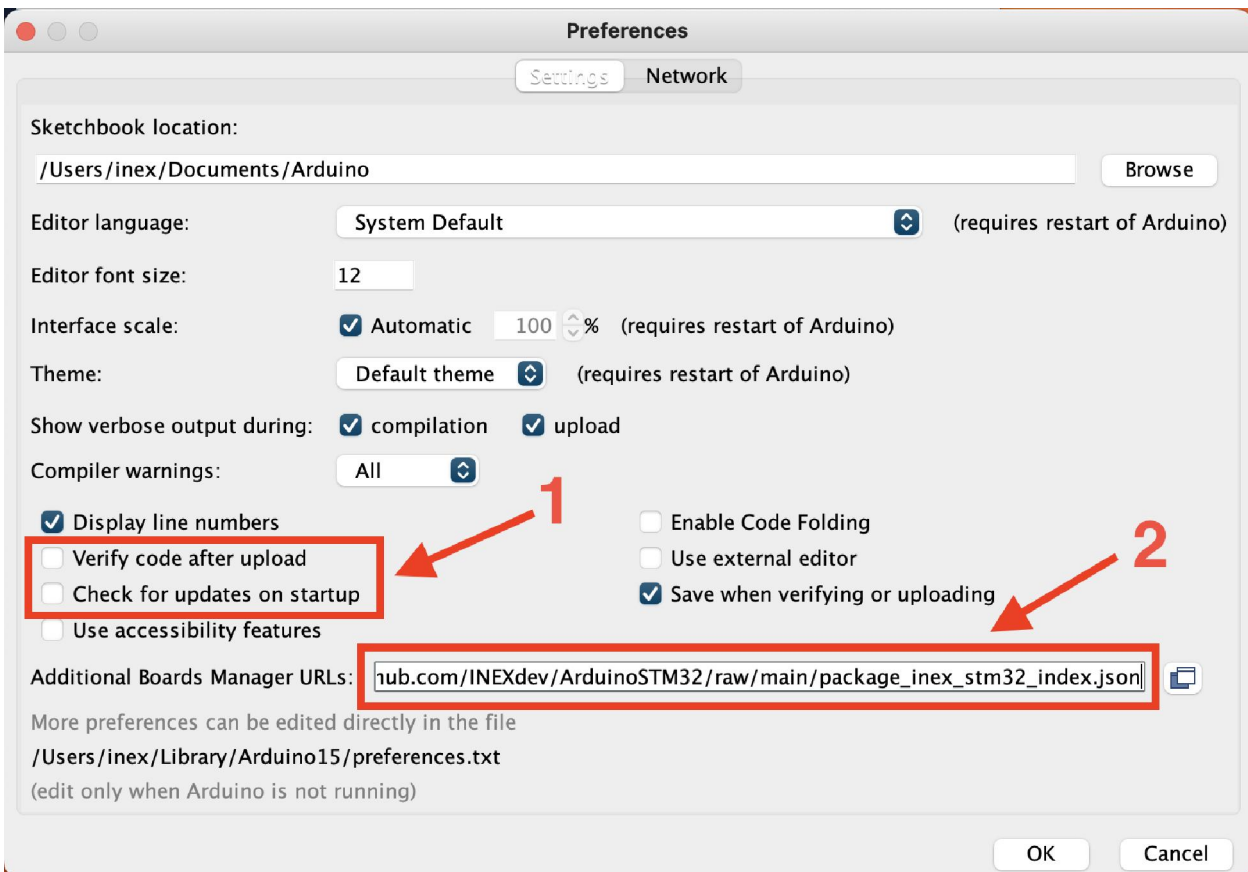
รูปที่ 2-22 แสดงหน้าต่างหลักของโปรแกรม Arduino IDE

(2) หน้าต่างหลักของโปรแกรม Arduino IDE จะปรากฏขึ้นมาพร้อมใช้งานตามรูปที่ 2-22

(3) ในขณะที่โปรแกรม Arduino IDE แยกดีฟที่แถบเมนูด้านบนให้คลิกเลือกเมนู **Arduino > Preferences...**ตามรูปที่ 2-23



รูปที่ 2-23 แสดงการเลือกเมนู Preferences ของโปรแกรม ARduino IDE บนระบบปฏิบัติการ MAC OS X



รูปที่ 2-24 แสดงการตั้งค่าหน้าต่าง Preferences เพื่อเตรียมการติดตั้งข้อมูลของบอร์ด POP-32

(4) หน้าต่าง Preferences จะปรากฏขึ้นมาตามรูปที่ 2-24 ให้ทำการตั้งค่าดังนี้

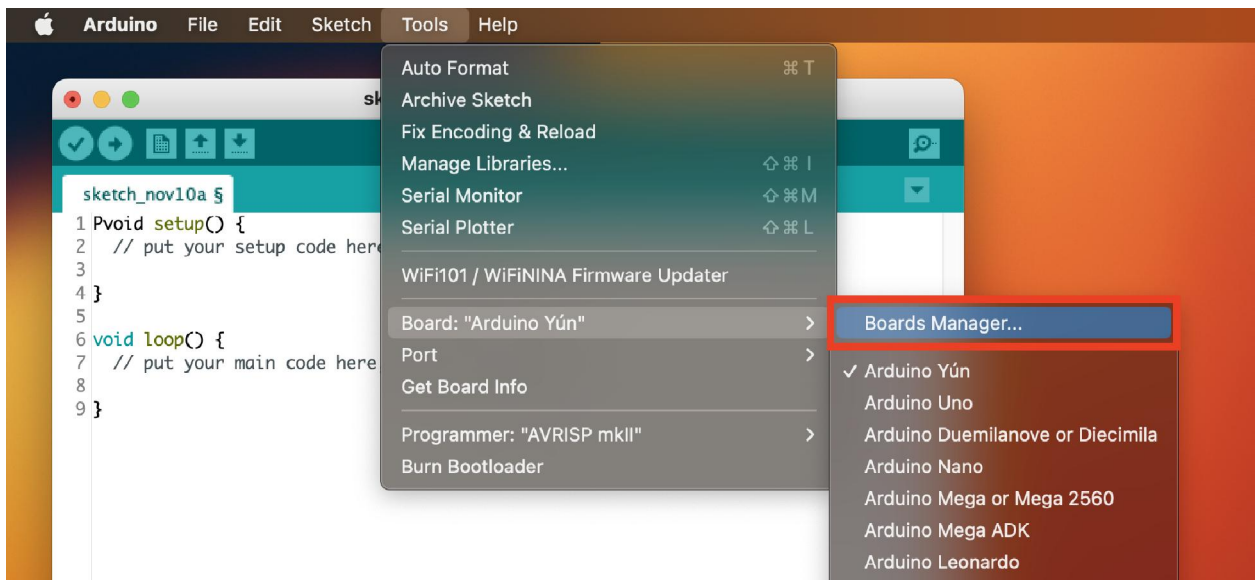
- คลิกที่รายการ **Verify code after upload** เพื่อนำเครื่องหมายถูกออก
- คลิกที่รายการ **Check for updates on startup** เพื่อนำเครื่องหมายถูกออก
- ที่รายการ **Additional Boards Manager URLs:** กำหนดค่าเป็น

`https://github.com/INEXdev/ArduinoSTM32/raw/main/package_inex_stm32_index.json`

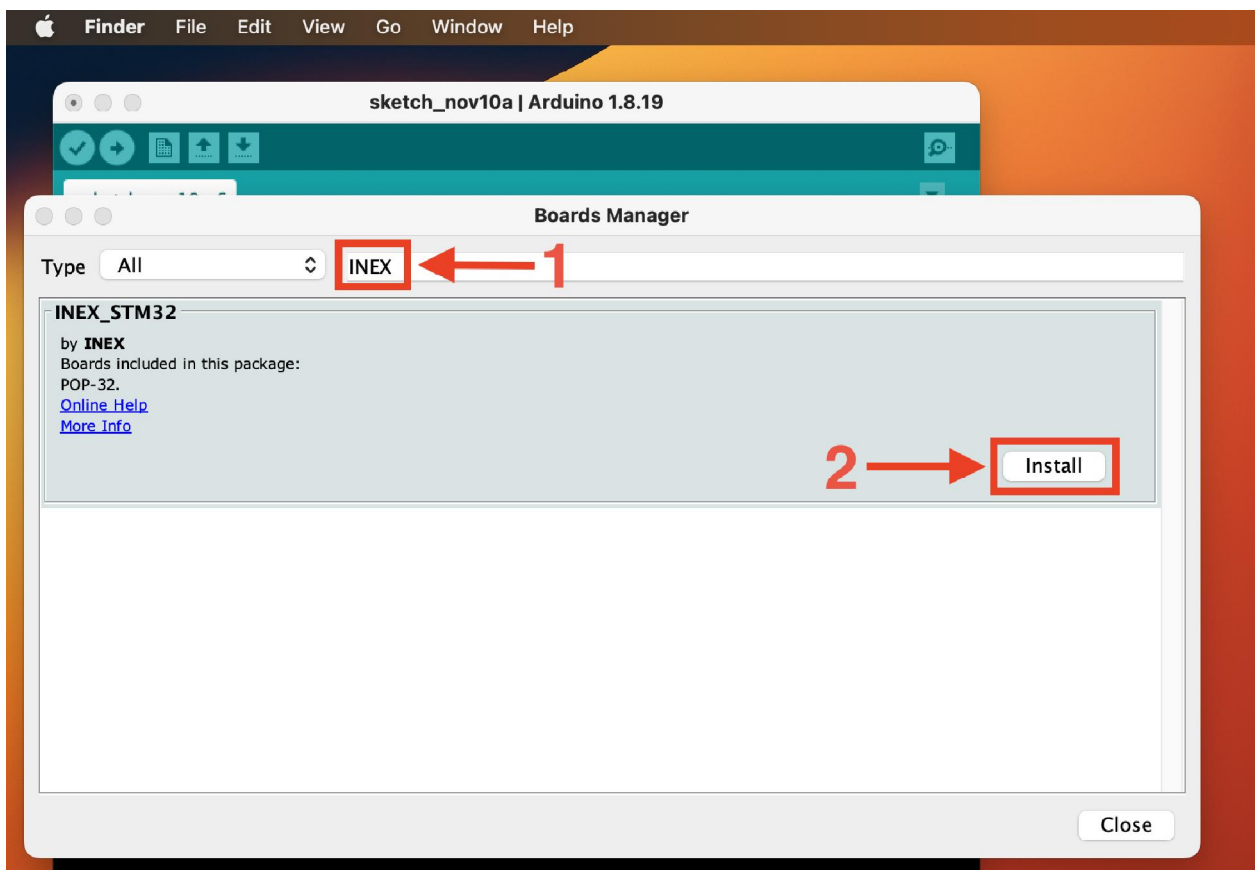
จากนั้นคลิกปุ่ม **OK** เพื่อยืนยันการตั้งค่า

(5) เลือกเมนู **Tools > Board:xxx > Boards Manager...** ตามรูปที่ 2-25

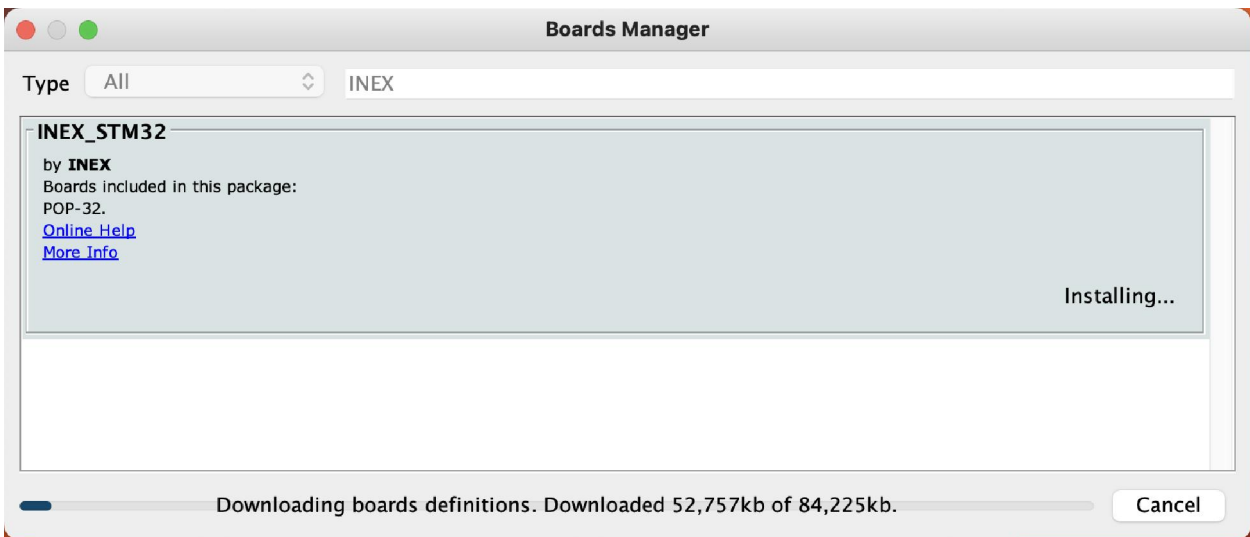
(6) หน้าต่าง **Boards Manager** ปรากฏขึ้นตามรูปที่ 2-26 ให้พิมพ์ค้นหาด้วยคำว่า **INEX** จะพบรายการตัวติดตั้งข้อมูลทางฮาร์ดแวร์ชื่อ **INEX_STM32** ซึ่งมีข้อมูลของบอร์ด **POP-32** รวมอยู่ด้วย จากนั้นคลิกปุ่ม **Install** เพื่อทำการติดตั้ง



รูปที่ 2-25 แสดงการเลือกเปิด Boards Manager...



รูปที่ 2-26 แสดงหน้าต่าง Boards Manager สำหรับติดตั้งไลบรารีและข้อมูลทางฮาร์ดแวร์ของ INEX_STM32



รูปที่ 2-27 แสดงกระบวนการติดตั้งไลบรารีในกลุ่ม INEX_STM32

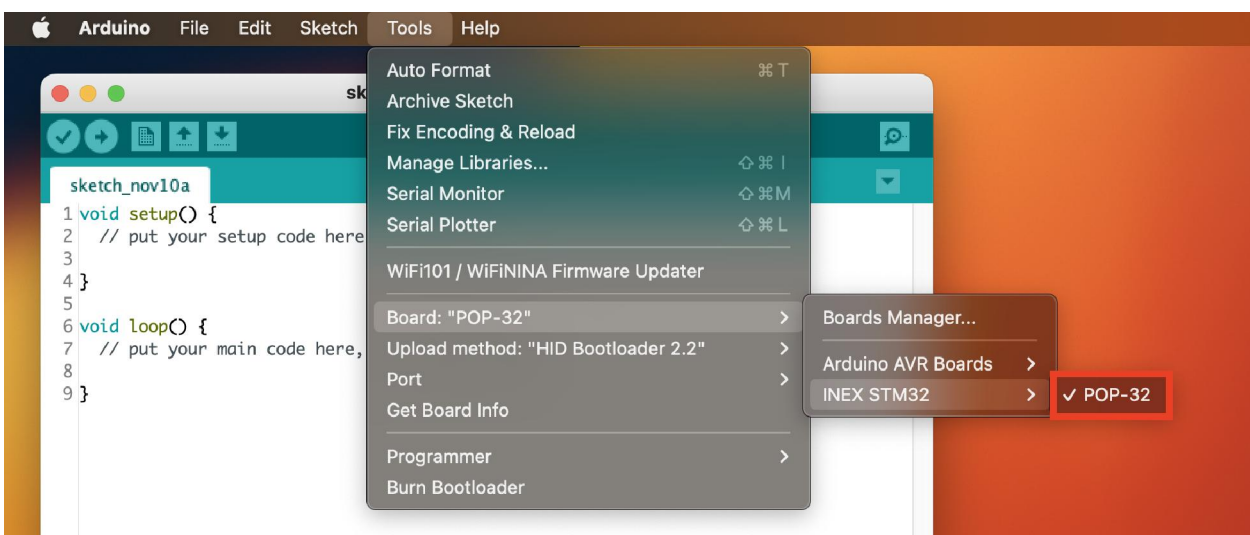
(7) จากนั้นเข้าสู่กระบวนการติดตั้งไลบรารีตามรูปที่ 2-27 รอจนกระทั่งการติดตั้งเสร็จสมบูรณ์ ขั้นตอนถัดไปเป็นทดสอบอัปโหลดโปรแกรมไปยังบอร์ด **POP-32** เพื่อยืนยันว่าการติดตั้งข้อมูลฮาร์ดแวร์และไลบรารีของบอร์ด **POP-32** ให้กับโปรแกรม Arduino IDE ถูกต้องและพร้อมใช้งาน

(8) ต่อสาย USB เพื่อเชื่อมต่อบอร์ด **POP-32** กับคอมพิวเตอร์

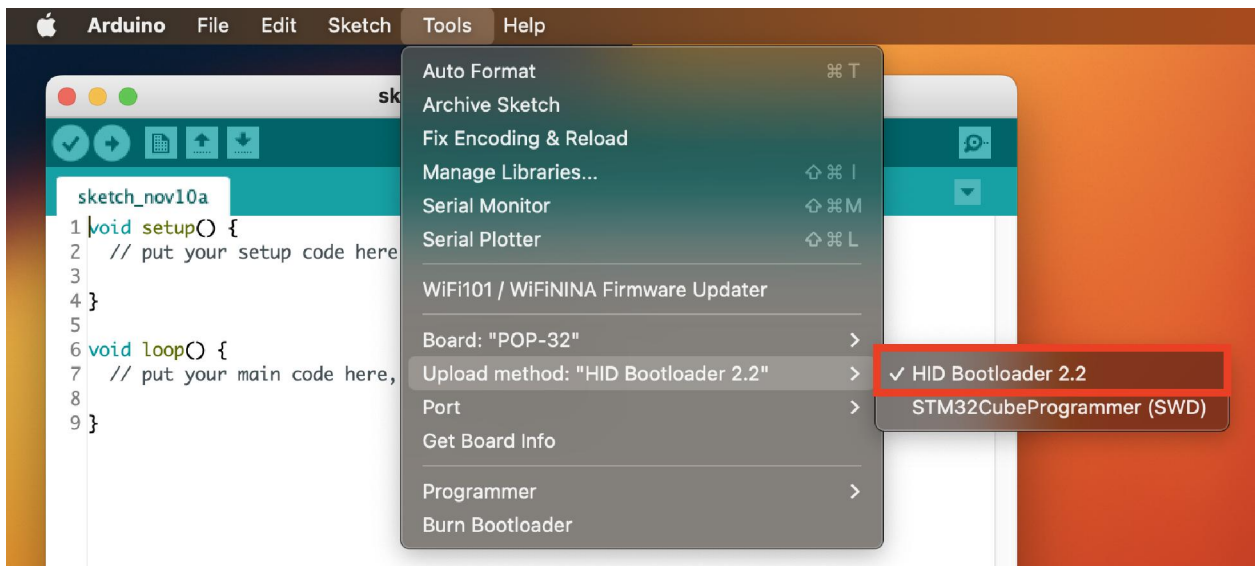
(9) จ่ายไฟเลี้ยงให้บอร์ด **POP-32** เปิดสวิตช์ POWER

(10) ในขณะที่โปรแกรม Arduino IDE แอคทีฟ ที่แถบเมนูด้านบนให้คลิกเลือกเมนู **Tools > Board:xxx > INEX STEM32 > POP-32** ตามรูปที่ 2-28

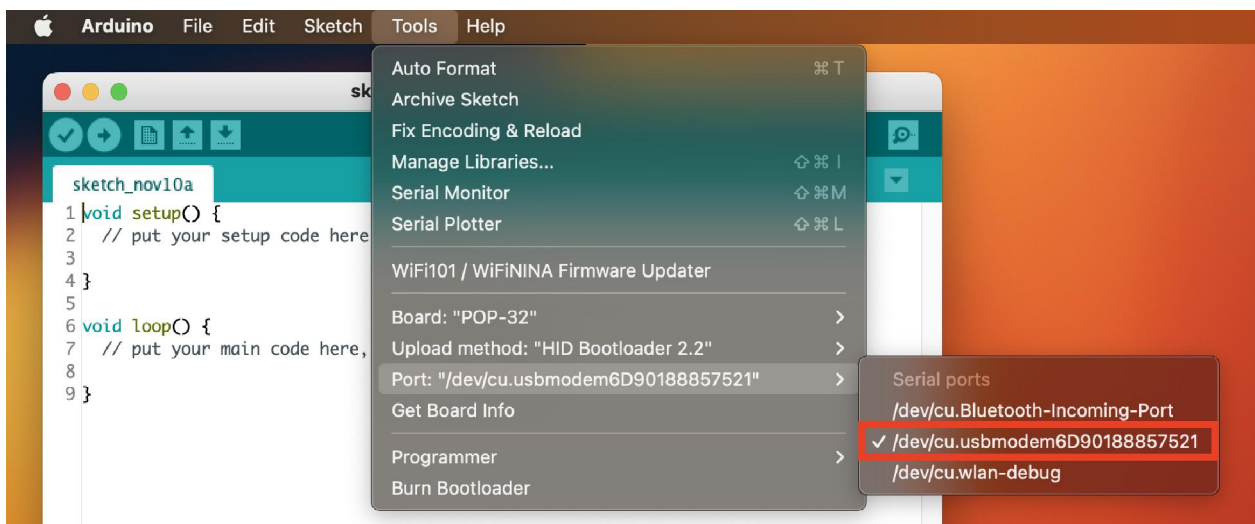
(11) เลือกวิธีการอัปโหลดโค้ด โดยเลือกเมนู **Tools > Upload method: “HID Bootloader 2.2”>HID Bootloader 2.2** ตามรูปที่ 2-29



รูปที่ 2-28 แสดงการเลือกบอร์ด POP-32



รูปที่ 2-29 แสดงการเลือกวิธีการอัปโหลดสำหรับบอร์ด POP-32 ของ Arduino IDE



รูปที่ 2-30 การเลือกหมายเลขพอร์ตสำหรับการสื่อสารข้อมูลอนุกรมกับบอร์ด POP-32

(12) เลือกพอร์ตอนุกรมสำหรับติดต่อกับโปรแกรม Arduino IDE โดยเลือกเมนู **Tools > Port:xxx > /dev/cu.usbmodemXXXXX** ตามรูปที่ 2-30 โดยในส่วนนี้หมายเลขพอร์ตเชื่อมต่อ USB ระบบปฏิบัติการจะเป็นตัวกำหนดขึ้นมาอีกทอดหนึ่ง

(13) ที่หน้าต่างหลักของโปรแกรม Arduino IDE พิมพ์โค้ดตามโปรแกรมที่ 2-1 สำหรับทดสอบการแสดงผลข้อความที่หน้าจอแสดงผล OLED จากนั้นคลิกปุ่ม **Upload**

(14) เมื่อการอัปโหลดโปรแกรมสิ้นสุดลง บอร์ด **POP-32** จะทำงานทันที

ที่จอแสดงผล **OLED** ของบอร์ด **POP-32** แสดงข้อความ **Hello POP-32**

บทที่ 3

โครงสร้างโปรแกรมของ Arduino

ในการเขียนโปรแกรมสำหรับบอร์ด **POP-32** จะต้องเขียนโปรแกรมโดยใช้ภาษา C/C++ ของ Arduino (Arduino programming language) เวอร์ชัน 1.8 ขึ้นไป ภาษาของ Arduino แบ่งได้เป็น 2 ส่วนหลักคือ

1. โครงสร้างภาษา (structure) ตัวแปรและค่าคงที่
2. ฟังก์ชัน (function)

ภาษาของ Arduino จะอ้างอิงตามภาษา C/C++ จึงอาจกล่าวได้ว่าการเขียนโปรแกรมสำหรับ Arduino (รวมถึงบอร์ด **POP-32**) ก็คือการเขียนโปรแกรมภาษา C โดยเรียกใช้งานฟังก์ชันและไลบรารีที่ทาง Arduino ได้เตรียมไว้ให้แล้ว ซึ่งสะดวกทำให้ผู้ที่ไม่มีความรู้ด้านไมโครคอนโทรลเลอร์อย่างลึกซึ้งสามารถเขียนโปรแกรมสั่งงานได้

3.1 โครงสร้างโปรแกรมของ Arduino

โปรแกรมของ Arduino แบ่งได้เป็นสองส่วนคือ

```
void setup()
```

และ

```
void loop()
```

โดยฟังก์ชัน `setup()` เมื่อโปรแกรมทำงานจะทำคำสั่งของฟังก์ชันนี้เพียงครั้งเดียว ใช้ในการกำหนดค่าเริ่มต้นของการทำงาน ส่วนฟังก์ชัน `loop()` เป็นส่วนทำงาน โปรแกรมจะกระทำคำสั่งในฟังก์ชันนี้ต่อเนื่องกันตลอดเวลา โดยโค้ดโปรแกรมที่ทำงานใน `loop()` มักเป็นคำสั่งอ่านค่าอินพุตประมวลผล สั่งงานเอาต์พุต ฯลฯ

ส่วนกำหนดค่าเริ่มต้น เช่น ตัวแปร จะต้องเขียนไว้ที่ส่วนหัวของโปรแกรม ก่อนถึงตัวฟังก์ชัน นอกจากนั้นยังต้องคำนึงถึงตัวพิมพ์เล็ก-ใหญ่ของตัวแปรและชื่อฟังก์ชันให้ถูกต้องด้วย

3.1.1 ส่วนของฟังก์ชัน `setup()`

ฟังก์ชันนี้จะเขียนที่ส่วนต้นของโปรแกรม ทำงานเมื่อโปรแกรมเริ่มต้นเพียงครั้งเดียว ใช้เพื่อกำหนดค่าของตัวแปร, โหมดการทำงานของขาต่างๆ หรือเริ่มต้นเรียกใช้ไลบรารี ฯลฯ

ตัวอย่างที่ 3-1

```
int buttonPin = 3;
void setup()
{
  Serial.begin(9600);
  pinMode(buttonPin, INPUT);
}
void loop()
{
  if (digitalRead(buttonPin) == HIGH)
    Serial.write('H');
  else
    Serial.write('L');
  delay(1000);
}
```

ในขณะที่โปรแกรมภาษา C มาตรฐานที่เขียนบน AVR GCC (เป็นโปรแกรมภาษา C ที่ใช้คอมไพเลอร์แบบ GCC สำหรับไมโครคอนโทรลเลอร์ AVR) จะเขียนได้ดังนี้

```
int main(void)
{
  init();
  setup();
  for (;;)
  {
    loop();
  }
  return ;
}
```

ตรงกับ void setup()

ตรงกับ void loop()

3.1.2 ส่วนของฟังก์ชัน loop ()

หลังจากที่เขียนฟังก์ชัน `setup()` ที่กำหนดค่าเริ่มต้นของโปรแกรมแล้ว ส่วนถัดมาคือฟังก์ชัน `loop()` ซึ่งมีการทำงานตรงตามชื่อคือ จะทำงานตามฟังก์ชันนี้วนต่อเนื่องตลอดเวลา ภายในฟังก์ชันจะมีโปรแกรมของผู้ใช้ เพื่อรับค่าจากพอร์ต ประมวลผล แล้วส่งเอาต์พุตเพื่อควบคุมการทำงาน

ตัวอย่างที่ 3-2

```
int buttonPin = 3;           // กำหนดชื่อตัวแปรให้ขาพอร์ต 3 และชนิดของตัวแปร
void setup()
{
  Serial.begin(9600);
  pinMode(buttonPin, INPUT);
}
// ดูปตรวจสอบการกดสวิตซ์ที่ขาพอร์ตซึ่งถูกประกาศด้วยตัวแปร buttonPin
void loop()
{
  if (digitalRead(buttonPin) == HIGH)
    serial.Write('H');
  else
    Serial.write('L');
  delay(1000);
}
```

3.2 คำสั่งควบคุมการทำงาน

3.2.1 คำสั่ง if

ใช้ทดสอบเพื่อกำหนดเงื่อนไขการทำงานของโปรแกรม เช่น ถ้าอินพุตมีค่ามากกว่าค่าที่กำหนดไว้จะให้ทำอะไร โดยมีรูปแบบการเขียนดังนี้

```
if (someVariable > 50)
{
    // do something here
}
```

ตัวโปรแกรมจะทดสอบว่า ตัวแปร someVariable มีค่ามากกว่า 50 หรือไม่ ถ้าใช่ ให้ทำอะไร ถ้าไม่ใช่ให้ข้ามการทำงานส่วนนี้

การทำงานของคำสั่งนี้จะทดสอบเงื่อนไข ที่เขียนในเครื่องหมายวงเล็บ ถ้าเงื่อนไขเป็นจริงทำตามคำสั่งที่เขียนในวงเล็บปีกกา ถ้าเงื่อนไขเป็นเท็จ ข้ามการทำงานส่วนนี้ไป

ส่วนของการทดสอบเงื่อนไขที่เขียนอยู่ภายในวงเล็บ จะต้องใช้ตัวกระทำเปรียบเทียบต่างๆ ดังนี้

```
x == y   (x เท่ากับ y)
x != y   (x ไม่เท่ากับ y)
x < y    (x น้อยกว่า y)
x > y    (x มากกว่า y)
x <= y   (x น้อยกว่าหรือเท่ากับ y)
x >= y   (x มากกว่าหรือเท่ากับ y)
```

เทคนิคสำหรับการเขียนโปรแกรม

ในการเปรียบเทียบตัวแปรให้ใช้ตัวกระทำ == เช่น if (x==10) ห้ามเขียนผิดเป็น = เช่น if (x=10) คำสั่งที่เขียนผิดในแบบนี้ ผลการทดสอบจะเป็นจริงเสมอ เมื่อผ่านคำสั่งนี้แล้ว x จะมีค่าเท่ากับ 10 ทำให้การทำงานของโปรแกรมผิดเพี้ยนไป ไม่เป็นตามที่กำหนดไว้

นอกจากนั้นยังใช้คำสั่ง if ควบคุมการแยกเส้นทางของโปรแกรม โดยใช้คำสั่ง if...else ได้ด้วย

3.2.2 คำสั่ง `if...else`

ใช้ทดสอบเพื่อกำหนดเงื่อนไขการทำงานของโปรแกรมได้มากกว่าคำสั่ง `if` ธรรมดา โดยกำหนดได้ว่า ถ้าเงื่อนไขเป็นจริงให้ทำอะไร ถ้าเป็นเท็จให้ทำอะไร เช่น ถ้าค่าอินพุตแอนะล็อกที่อ่านได้น้อยกว่า 500 ให้ทำอะไร ถ้าค่ามากกว่าหรือเท่ากับ 500 ให้ทำอีกอย่าง เขียนคำสั่งได้ดังนี้

ตัวอย่างที่ 3-3

```
if (pinFiveInput < 500)
{
    // คำสั่งเพื่อทำงานอย่างหนึ่ง เนื่องจาก pinFiveInput มีค่าน้อยกว่า 500
}
else
{
    // คำสั่งเพื่อทำงานอีกอย่างหนึ่ง เนื่องจาก pinFiveInput มีค่ามากกว่าหรือเท่ากับ 500
}
```

หลังคำสั่ง `else` สามารถตามด้วยคำสั่ง `if` ทำให้รูปแบบคำสั่งกลายเป็น `if...else...if` เป็นการทดสอบเงื่อนไขต่างๆ เมื่อเป็นจริงให้ทำตามที่ต้องการ ดังตัวอย่างต่อไปนี้

ตัวอย่างที่ 3-4

```
if (pinFiveInput < 500)
{
    // คำสั่งเพื่อทำงานอย่างหนึ่ง เนื่องจาก pinFiveInput มีค่าน้อยกว่า 500
}
else if (pinFiveInput >= 1000)
{
    // คำสั่งเพื่อทำงานอีกอย่างหนึ่ง เนื่องจาก pinFiveInput มีค่ามากกว่าหรือเท่ากับ 1000
}
else
{
    // คำสั่งเพื่อกำหนดให้ทำงานต่อไปในกรณีที่ pinFiveInput ไม่ได้มีค่าน้อยกว่า 500
    // และมากกว่าหรือเท่ากับ 1000 (นั่นคือ จะมีการกระทำคำสั่งในโปรแกรมน้อยนี้เมื่อตัวแปรมีค่าอยู่
    // ระหว่าง 501 ถึง 999 (ฐานสิบ)
}
```

หลังคำสั่ง `else` สามารถตามด้วยคำสั่ง `if` ได้ไม่จำกัด (หรือใช้คำสั่ง `switch case` แทนคำสั่ง `if...else...if` สำหรับการทดสอบเงื่อนไขจำนวนมากๆ ได้)

เมื่อใช้คำสั่ง `if...else` แล้ว ต้องกำหนดด้วยว่า ถ้าทดสอบไม่ตรงกับเงื่อนไขใดๆ เลย ให้ทำอะไร โดยให้กำหนดที่คำสั่ง `else` ตัวสุดท้าย

3.2.3 คำสั่ง `for()`

คำสั่งนี้ใช้เพื่อสั่งให้คำสั่งที่อยู่ภายในวงเล็บปีกกาหลัง `for` มีการทำงานซ้ำกันตามจำนวนรอบที่ต้องการ คำสั่งนี้มีประโยชน์มากสำหรับการทำงานที่ต้องทำซ้ำกันและทราบจำนวนรอบของการทำซ้ำที่แน่นอน มักใช้คู่กับตัวแปรอะไรก็ได้ในการเก็บสะสมค่าที่อ่านได้จากขาอินพุตแอนะล็อกหลายๆ ขาที่มีหมายเลขขาต่อเนื่องกัน

รูปแบบของคำสั่ง `for()` แบ่งได้ 3 ส่วนดังนี้

```
for (initialization; condition; increment)
{
    //statement(s);
}
```

เริ่มต้นด้วย `initialization` ใช้กำหนดค่าเริ่มต้นของตัวแปรควบคุมการวนรอบ ในการทำงานแต่ละรอบจะทดสอบ `condition` ถ้าเงื่อนไขเป็นจริงจะกระทำคำสั่งในวงเล็บปีกกา แล้วมาเพิ่มหรือลดค่าตัวแปรตามที่สั่งใน `increment` แล้วทดสอบเงื่อนไขอีก ทำซ้ำจนกว่าเงื่อนไขเป็นเท็จ

ตัวอย่างที่ 3-5

```
for (int i=1; i <= 8; i++)
{
    // คำสั่งเพื่อทำงานโดยใช้ค่าของตัวแปร i และวนทำงานจนกระทั่งค่าของตัวแปร i มากกว่า 8;
}
```

คำสั่ง `for` ของภาษา C จะยืดหยุ่นกว่าคำสั่ง `for` ของภาษาคอมไพเลอร์อื่นๆ โดยสามารถละเว้นบางส่วนหรือทั้งสามส่วนของคำสั่ง `for` ได้ อย่างไรก็ตามยังคงต้องมีเซมิโคลอน

3.2.4 คำสั่ง switch-case

ใช้ทดสอบเงื่อนไขเพื่อกำหนดการทำงานของโปรแกรม ถ้าตัวแปรที่ทดสอบตรงกับเงื่อนไขใดก็ให้ทำงานตามที่กำหนดไว้

พารามิเตอร์

var ตัวแปรที่ต้องการทดสอบว่าตรงกับเงื่อนไขใด

default ถ้าไม่ตรงกับเงื่อนไขใดๆ เลยให้ทำคำสั่งต่อท้ายนี้

break คำสั่งหยุดการทำงาน ใช้เขียนต่อท้าย case ต่างๆ ถ้าไม่ได้เขียน โปรแกรมจะวนทำงานตามเงื่อนไขต่อไปเรื่อยๆ

ตัวอย่างที่ 3-6

```
switch (var)
{
    case 1:
        // คำสั่งเพื่อทำงาน เมื่อค่าของตัวแปรเท่ากับ 1
        break;
    case 2:
        // คำสั่งเพื่อทำงาน เมื่อค่าของตัวแปรเท่ากับ 2
        break;
    default:
        // ถ้าหากค่าของตัวแปรไม่ใช่ 1 และ 2 ให้กระทำคำสั่งในส่วนนี้
}
```

3.2.5 คำสั่ง `while`

เป็นคำสั่งวนรอบ โดยจะทำคำสั่งที่เขียนในวงเล็บปีกกาอย่างต่อเนื่อง จนกว่าเงื่อนไขในวงเล็บของคำสั่ง `while()` จะเป็นเท็จ คำสั่งที่ทำให้ซ้ำจะต้องมีการเปลี่ยนแปลงค่าตัวแปรที่ใช้ทดสอบ เช่น มีการเพิ่มค่าตัวแปร หรือมีเงื่อนไขภายนอกเช่นอ่านค่าจากเซ็นเซอร์ได้เรียบร้อยแล้วให้หยุดการอ่านค่า มิฉะนั้นเงื่อนไขในวงเล็บของ `while()` เป็นจริงตลอดเวลา ทำให้คำสั่ง `while` ทำงานวนไม่รู้จบ

รูปแบบคำสั่ง

```
while (expression)
{
    // statement(s);
}
```

พารามิเตอร์

`expression` เป็นคำสั่งทดสอบเงื่อนไข (ถูกหรือผิด)

ตัวอย่างที่ 3-7

```
var = 0;
while(var < 200)
{
    // คำสั่งเพื่อทำงาน โดยวนทำงานทั้งสิ้น 200 รอบ
    var++;
}
```


3.3 ตัวกระทำทางคณิตศาสตร์

ประกอบด้วยตัวกระทำ 5 ตัวคือ + (บวก), - (ลบ), * (คูณ), / (หาร) และ % (หารเอาเศษ)

3.3.1 ตัวกระทำทางคณิตศาสตร์ บวก ลบ คูณ และหาร

ใช้หาค่าผลรวม ผลต่าง ผลคูณ และผลหารค่าของตัวถูกกระทำสองตัว โดยให้คำตอบมีประเภทตรงกับตัวถูกกระทำทั้งสองตัว เช่น $9/4$ ให้คำตอบเท่ากับ 2 เนื่องจากทั้ง 9 และ 4 เป็นตัวแปรเลขจำนวนเต็ม (int)

นอกจากนี้ตัวกระทำทางคณิตศาสตร์อาจทำให้เกิดโอเวอร์โฟลว (overflow) หากผลลัพธ์ที่ได้มีขนาดใหญ่เกินกว่าจะเก็บในตัวแปรประเภทนั้น ถ้าตัวที่ถูกกระทำต่างประเภทกัน ผลลัพธ์ได้เป็นจะมีขนาดใหญ่ขึ้นเท่ากับประเภทของตัวแปรที่ใหญ่ที่สุด (เช่น $9/4=2$ หรือ $9/3.0=2.25$)

รูปแบบคำสั่ง

```
result = value1 + value2;
```

```
result = value1 - value2;
```

```
result = value1 * value2;
```

```
result = value1 / value2;
```

พารามิเตอร์

value1 : เป็นค่าของตัวแปรหรือค่าคงที่ใดๆ

value2: เป็นค่าของตัวแปรหรือค่าคงที่ใดๆ

ตัวอย่างที่ 3-8

```
y = y + 3;
```

```
x = x - 7;
```

```
i = j * 6;
```

```
r = r / 5;
```

เทคนิคสำหรับการเขียนโปรแกรม

- เลือกขนาดของตัวแปรให้ใหญ่พอสำหรับเก็บค่าผลลัพธ์ที่มากที่สุดของการคำนวณ
- ต้องทราบว่าที่ค่าใดตัวแปรที่เก็บจะมีการวนซ้ำค่ากลับ และวนกลับอย่างไร เช่น 0 ไป 1 หรือ 0 ไป -32768
- สำหรับการคำนวณที่ต้องการเศษส่วนให้ใช้ตัวแปรประเภท float แต่ให้ระวังผลลบ เช่น ตัวแปรมีขนาดใหญ่ คำนวณได้ซ้ำ
- ใช้ตัวกระทำ cast เช่น (int)myfloat ในการเปลี่ยนประเภทของตัวแปรชั่วคราว ขณะที่โปรแกรมทำงาน

3.3.2 ตัวกระทำ % หารเอาเศษ

ใช้หาค่าเศษที่ได้ของการหารเลขจำนวนเต็ม 2 ตัว ตัวกระทำหารเอาเศษไม่สามารถใช้งานกับตัวแปรเลขทศนิยม (float)

รูปแบบคำสั่ง

```
result = value1 % value2;
```

พารามิเตอร์

value1 - เป็นตัวแปรประเภท byte,char,int หรือ long

value2 - เป็นตัวแปรประเภท byte,char,int หรือ long

ผลที่ได้

เศษจากการหารค่าเลขจำนวนเต็ม เป็นข้อมูลชนิดเลขจำนวนเต็ม

ตัวอย่างที่ 3-9

```
x = 7 % 5;           // x now contains 2
x = 9 % 5;           // x now contains 4
x = 5 % 5;           // x now contains 0
x = 4 % 5;           // x now contains 4
```

ตัวกระทำหารเอาเศษนี้มักใช้ในงานที่ต้องการให้เหตุการณ์เกิดขึ้นด้วยช่วงเวลาที่เหมาะสมหรือใช้ทำให้หน่วยความที่เก็บตัวแปรอะเรย์เกิดการล้นค่ากลับ (roll over)

ตัวอย่างที่ 3-10

```
// ตรวจสอบค่าของตัวตรวจจับ 10 ครั้งต่อการทำงาน 1 รอบ
void loop()
{
  i++;
  if ((i % 10) == 0)           // หาค่าของ i ด้วย 10 แล้วตรวจสอบเศษการหารเป็น 0 หรือไม่
  {
    x = analogRead(sensPin);   // อ่านค่าจากตัวตรวจจับ 10 ครั้ง
  }
}
```

ในตัวอย่างนี้เป็นการนำคำสั่ง % มาใช้กำหนดรอบของการทำงาน โดยโปรแกรมวนทำงานเพื่ออ่านค่าจนกว่าผลการหารเอาเศษของคำสั่ง $i \% 10$ จะเท่ากับ 0 ซึ่งจะเกิดขึ้นเมื่อ $i = 10$ เท่านั้น

3.4 ตัวกระทำเปรียบเทียบ

ใช้ประกอบกับคำสั่ง `if()` เพื่อทดสอบเงื่อนไขหรือเปรียบเทียบค่าตัวแปรต่าง โดยจะเขียนเป็นนิพจน์อยู่ภายในเครื่องหมาย `()`

```
x == y (x เท่ากับ y)
x != y (x ไม่เท่ากับ y)
x < y (x น้อยกว่า y)
x > y (x มากกว่า y)
x <= y (x น้อยกว่าหรือเท่ากับ y)
x >= y (x มากกว่าหรือเท่ากับ y)
```

3.5 ตัวกระทำทางตรรกะ

ใช้ในการเปรียบเทียบของคำสั่ง `if()` มี 3 ตัวคือ `&&`, `||` และ `!`

3.5.1 `&&` (ตรรกะ และ)

ให้ค่าเป็นจริงเมื่อผลการเปรียบเทียบทั้งสองข้างเป็นจริงทั้งคู่

ตัวอย่างที่ 3-11

```
if (x > 0 && x < 5)
{
    // ...
}
```

ให้ค่าเป็นจริงเมื่อ `x` มากกว่า 0 และน้อยกว่า 5 (มีค่า 1 ถึง 4)

3.5.2 `&&` (ตรรกะ หรือ)

ให้ค่าเป็นจริง เมื่อผลการเปรียบเทียบพบว่า มีตัวแปรตัวใดตัวหนึ่งเป็นจริงหรือเป็นจริงทั้งคู่

ตัวอย่างที่ 3-12

```
if (x > 0 || y > 0)
{
    // ...
}
```

ให้ผลเป็นจริงเมื่อ `x` หรือ `y` มีค่ามากกว่า 0

3.5.3 ! (ใช้กลับผลเป็นตรงกันข้าม)

ให้ค่าเป็นจริง เมื่อผลการเปรียบเทียบเป็นเท็จ

ตัวอย่างที่ 3-13

```
if (!x)
{
    // ...
}
```

ให้ผลเป็นจริงถ้า x เป็นเท็จ (เช่น ถ้า $x = 0$ ให้ผลเป็นจริง)

3.5.4 ข้อควรระวัง

ระวังเรื่องการเขียนโปรแกรม ถ้าต้องการใช้ตัวกระทำตรรกะและ ต้องเขียนเครื่องหมาย && ถ้าลืมเขียนเป็น & จะเป็นตัวกระทำและระดับบิตกับตัวแปร ซึ่งให้ผลที่แตกต่าง

ในการใช้ตรรกะหรือให้เขียนเป็น || (จัดตั้งสองตัวติดกัน) ถ้าเขียนเป็น | (จัดตั้งตัวเดียว) จะหมายถึงตัวกระทำหรือระดับบิตกับตัวแปร

ตัวกระทำ NOT ระดับบิต (~) จะแตกต่างจากตัวกลับผลให้เป็นตรงข้าม (!) ต้องเลือกใช้ให้ถูกต้อง

ตัวอย่างที่ 3-14

```
if (a >= 10 && a <= 20){}
```

// ให้ผลการทำงานเป็นจริงเมื่อ a มีค่าอยู่ระหว่าง 10 ถึง 20

3.6 ตัวกระทำระดับบิต

ตัวกระทำระดับจะนำบิตของตัวแปรมาประมวลผล ใช้ประโยชน์ในการแก้ปัญหาด้านการเขียนโปรแกรมได้หลากหลาย ตัวกระทำระดับของภาษาซี (ซึ่งรวมถึง Arduino) มี 6 ตัวได้แก่ & (bitwise AND), | (OR), ^ (Exclusive OR), ~ (NOT), << (เลื่อนบิตไปทางขวา) และ >> (เลื่อนบิตไปทางซ้าย)

3.6.1 ตัวกระทำระดับบิต AND (&)

คำสั่ง AND ในระดับบิตของภาษา C เขียนได้โดยใช้ & หนึ่งตัว โดยต้องเขียนระหว่างนิพจน์หรือตัวแปรที่เป็นเลขจำนวนเต็ม การทำงานจะนำข้อมูลแต่ละบิตของตัวแปรทั้งสองตัวมากระทำทางตรรกะ AND โดยมีกฎดังนี้

ถ้าอินพุตทั้งสองตัวเป็น “1” ทั้งคู่เอาต์พุตเป็น “1” กรณีอื่นๆ เอาต์พุตเป็น “0” ดังตัวอย่างต่อไปนี้ ในการดูให้ดูของตัวกระทำตามแนวดัง

0	0	1	1	Operand1
0	1	0	1	Operand2
0	0	0	1	Returned result

ใน Arduino ตัวแปรประเภท int จะมีขนาด 16 บิต ดังนั้นเมื่อใช้ตัวกระทำระดับบิต AND จะมีการกระทำตรรกะและพร้อมกันกับข้อมูลทั้ง 16 บิต ดังตัวอย่างในส่วนของโปรแกรมต่อไปนี้

ตัวอย่างที่ 3-15

```
int a = 92;    // เท่ากับ 0000000001011100 ฐานสอง
int b = 101;   // เท่ากับ 0000000001100101 ฐานสอง
int c = a & b; // ผลลัพธ์คือ 0000000001000100 ฐานสองหรือ 68 ฐานสิบ
```

ในตัวอย่างนี้จะนำข้อมูลทั้ง 16 บิตของตัวแปร a และ b มากระทำทางตรรกะ AND แล้วนำผลลัพธ์ที่ได้ทั้ง 16 บิตไปเก็บที่ตัวแปร c ซึ่งได้ค่าเป็น 01000100 ในเลขฐานสองหรือเท่ากับ 68 ฐานสิบ

นิยมใช้ตัวกระทำระดับบิต AND เพื่อใช้เลือกข้อมูลบิตที่ต้องการ (อาจเป็นหนึ่งบิตหรือหลายบิต) จากตัวแปร int ซึ่งการเลือกเพียงบางบิตนี้จะเรียกว่า **masking**

3.6.2 ตัวกระทำระดับบิต OR (|)

คำสั่งระดับบิต OR ของภาษาซีเขียนได้โดยใช้เครื่องหมาย | หนึ่งตัว โดยต้องเขียนระหว่างนิพจน์หรือตัวแปรที่เป็นเลขจำนวนเต็ม สำหรับการทำงานให้นำข้อมูลแต่ละบิตของตัวแปรทั้งสองตัวมากระทำทางตรรกะ OR โดยมีกฎดังนี้

ถ้าอินพุตตัวใดตัวหนึ่งหรือทั้งสองตัวเป็น “1” เอาต์พุตเป็น “1” กรณีที่อินพุตเป็น “0” ทั้งคู่เอาต์พุตจะเป็น “0” ดังตัวอย่างต่อไปนี้

0	0	1	1	Operand1
0	1	0	1	Operand2
0	1	1	1	Returned result

ตัวอย่างที่ 3-16

ส่วนของโปรแกรมแสดงการใช้ตัวกระทำระดับบิต OR

```
int a = 92;    // เท่ากับ 0000000001011100 ฐานสอง
int b = 101;   // เท่ากับ 0000000001100101 ฐานสอง
int c = a | b; // ผลลัพธ์คือ 0000000001111101 ฐานสอง หรือ 125 ฐานสิบ
```

ตัวอย่างที่ 3-17

โปรแกรมแสดงการใช้ตัวกระทำระดับบิต AND และ OR

ตัวอย่างงานที่ใช้ตัวกระทำระดับบิต AND และ OR เป็นงานที่โปรแกรมเมอร์เรียกว่า **Read-Modify-Write on a port** สำหรับไมโครคอนโทรลเลอร์ 8 บิต ค่าที่อ่านหรือเขียนไปยังพอร์ตมีขนาด 8 บิต ซึ่งแสดงค่าอินพุตที่ขาทั้ง 8 ขา การเขียนค่าไปยังพอร์ตจะเขียนค่าครั้งเดียวได้ทั้ง 8 บิต

ตัวแปรชื่อ PORTD เป็นค่าที่ใช้แทนสถานะของขาดิจิตอลหมายเลข 0,1,2,3,4,5,6,7 ถ้าบิตใดมีค่าเป็น 1 ทำให้ขานั้นมีค่าลอจิกเป็น HIGH (อย่าลืมกำหนดให้ขาพอร์ตนั้นๆ ทำงานเป็นเอาต์พุตด้วยคำสั่ง pinMode() ก่อน) ดังนั้น ถ้ากำหนดค่าให้ PORTD = B00110001; ก็คือต้องการให้ขา 2,3 และ 7 เป็น HIGH ในกรณีนี้ไม่ต้องเปลี่ยนค่าสถานะของขา 0 และ 1 ซึ่งปกติแล้วฮาร์ดแวร์ของ Arduino ใช้ในการสื่อสารแบบอนุกรม ถ้าไปเปลี่ยนค่าแล้วจะกระทบต่อการสื่อสารแบบอนุกรม

อัลกอริทึมสำหรับโปรแกรมเป็นดังนี้

- อ่านค่าจาก PORTD แล้วล้างค่าเฉพาะบิตที่ต้องการควบคุม (ใช้ตัวกระทำแบบบิต AND)
- นำค่า PORTD ที่แก้ไขจากข้างต้นมารวมกับค่าบิตที่ต้องการควบคุม (ใช้ตัวกระทำแบบบิต OR)

เขียนเป็นโปรแกรมได้ดังนี้

```

int i;    // counter variable
int j;
void setup()
{
    DDRD = DDRD | B11111100;    // กำหนดทิศทางของขาพอร์ต 2 ถึง 7 ด้วยค่า 11111100
    Serial.begin(9600);
}
void loop()
{
    for (i=0; i<64; i++)
    {
        PORTD = PORTD & B00000011;    // กำหนดข้อมูลไปยังขาพอร์ต 2 ถึง 7
        j = (i << 2);
        PORTD = PORTD | j;
        Serial.println(PORTD, BIN);    // แสดงค่าของ PORTD ที่หน้าต่าง Serial monitor
        delay(100);
    }
}

```

3.6.3 คำสั่งระดับบิต Exclusive OR (^)

เป็นโอเปอเรเตอร์พิเศษที่ไม่ค่อยได้ใช้ในภาษา C/C++ ตัวกระทำระดับบิต exclusive OR (หรือ XOR) จะเขียนโดยใช้สัญลักษณ์เครื่องหมาย ^ ตัวกระทำนี้มีการทำงานใกล้เคียงกับตัวกระทำระดับบิต OR แตต่างกันเมื่ออินพุตเป็น “1” ทั้งคู่จะให้เอาต์พุตเป็น “0” แสดงการทำงานได้ดังนี้

0	0	1	1	Operand1
0	1	0	1	Operand2
0	1	1	0	Returned result

หรือกล่าวได้อีกอย่างว่า ตัวกระทำระดับบิต XOR จะให้เอาต์พุตเป็น “0” เมื่ออินพุตทั้งสองตัวมีค่าเหมือนกัน และให้เอาต์พุตเป็น “1” เมื่ออินพุตทั้งสองมีค่าต่างกัน

ตัวอย่างที่ 3-18

```

int x = 12;    // ค่าเลขฐานสองเท่ากับ 1100
int y = 10;    // ค่าเลขฐานสองเท่ากับ 1010
int z = x ^ y;    // ผลลัพธ์เท่ากับ 0110 ฐานสองหรือ 6 ฐานสิบ

```

ตัวกระทำระดับบิต XOR จะใช้มากในการสลับค่าบางบิตของตัวแปร int เช่น กลับจาก “0” เป็น “1” หรือกลับจาก “1” เป็น “0”

เมื่อใช้ตัวกระทำระดับบิต XOR ถ้าบิตของ mask เป็น “1” ทำให้บิตนั้นถูกสลับค่า ถ้า mask มีค่าเป็น “1” บิตนั้นมีค่าคงเดิม ตัวอย่างต่อไปนี้เป็นโปรแกรมแสดงการสั่งให้ขาพอร์ตดิจิทัล 5 มีการกลับลอจิกตลอดเวลา

ตัวอย่างที่ 3-19

```

void setup()
{
  DDRD = DDRD | B00100000;    // กำหนดขา 5 เป็นเอาต์พุต
}
void loop()
{
  PORTD = PORTD ^ B00100000;   // กลับลอจิกที่ขา 5
  delay(100);
}

```

3.6.4 ตัวกระทำระดับบิต NOT (~)

ตัวกระทำระดับบิต NOT จะเขียนโดยใช้สัญลักษณ์เครื่องหมาย ~ ตัวกระทำนี้จะใช้งานกับตัวถูกกระทำเพียงตัวเดียวที่อยู่ขวามือ โดยทำการสลับบิตทุกบิตให้มีค่าตรงกันข้ามคือ จาก “0” เป็น “1” และจาก “1” เป็น “0” ดังตัวอย่าง

0	1	Operand1
1	0	~ Operand1

```

int a = 103;    // binary: 0000000001100111
int b = ~a;     // binary: 1111111110011000

```

เมื่อกระทำแล้ว ทำให้ตัวแปร b มีค่า -104 (ฐานสิบ) ซึ่งคำตอบที่ได้เกิดเนื่องจากบิตที่มีความสำคัญสูงสุด (บิตซ้ายมือสุด) ของตัวแปร int อันเป็นบิตแจ้งว่าตัวเลขเป็นบวกหรือลบ มีค่าเป็น “1” แสดงว่าค่าที่ได้นี้คิดลบ โดยในคอมพิวเตอร์จะเก็บค่าตัวเลขทั้งบวกและลบตามระบบทวิคอมพลีเมนต์ (2’s complement)

การประกาศตัวแปร int ซึ่งมีความหมายเหมือนกับการประกาศตัวแปรเป็น signed int ต้องระวังค่าของตัวแปรจะติดลบได้

3.6.5 คำสั่งเลื่อนบิตไปทางซ้าย (<<) และเลื่อนบิตไปทางขวา (>>)

ในภาษา C/C++ มีตัวกระทำเลื่อนบิตไปทางซ้าย << และเลื่อนบิตไปทางขวา >> ตัวกระทำนี้จะสั่งเลื่อนบิตของตัวถูกกระทำที่เขียนด้านซ้ายมือไปทางซ้ายหรือไปทางขวาตามจำนวนบิตที่ระบุไว้ในด้านขวามือของตัวกระทำ

รูปแบบคำสั่ง

```
variable << number_of_bits
```

```
variable >> number_of_bits
```

พารามิเตอร์

variable เป็นตัวแปรเลขจำนวนเต็มที่มีจำนวนบิตน้อยกว่าหรือเท่ากับ 32 บิต (หรือตัวแปรประเภท byte, int หรือ long)

ตัวอย่างที่ 3-20

```
int a = 5;           // เท่ากับ 00000000000000101 ฐานสอง
int b = a << 3;      // ได้ผลลัพธ์เป็น 0000000000101000 ฐานสองหรือ 40
int c = b >> 3;      // ได้ผลลัพธ์เป็น 000000000000101 ฐานสองหรือ 5 ฐานสิบ
```

ตัวอย่างที่ 3-21

เมื่อสั่งเลื่อนค่าตัวแปร x ไปทางซ้ายจำนวน y บิต ($x \ll y$) บิตข้อมูลที่อยู่ด้านซ้ายสุดของ x จำนวน y ตัวจะหายไปเนื่องจากถูกเลื่อนหายไปทางซ้ายมือ

```
int a = 5;           // เท่ากับ 00000000000000101 ฐานสอง
int b = a << 14;      // ได้ผลลัพธ์เป็น 0100000000000000 ฐานสอง
```

การเลื่อนบิตไปทางซ้าย จะทำให้ค่าของตัวแปรด้านซ้ายมือของตัวกระทำจะถูกคูณด้วยค่าสองยกกำลังบิตที่เลื่อนไปทางซ้ายมือ ดังนี้

```
1 << 0  ==  1
1 << 1  ==  2
1 << 2  ==  4
1 << 3  ==  8
...
1 << 8  == 256
1 << 9  == 512
1 <<10  ==1024
...
```

เมื่อสับเปลี่ยนตัวแปร x ไปทางขวามือจำนวน y บิต ($x \gg y$) จะมีผลแตกต่างกันขึ้นกับประเภทของตัวแปร ถ้า x เป็นตัวแปรประเภท `int` ค่าที่เก็บได้มีทั้งค่าบวกและลบ โดยบิตซ้ายมือสุดจะเป็น **sign bit** หรือบิตเครื่องหมาย ถ้าเป็นค่าลบ ค่าบิตซ้ายมือสุดจะมีค่าเป็น 1 กรณีนี้เมื่อสับเปลี่ยนบิตไปทางขวามือแล้ว โปรแกรมจะนำค่าของบิตเครื่องหมายมาเติมให้กับบิตทางซ้ายมือสุด ปรากฏการณ์นี้เรียกว่า **sign extension** มีตัวอย่างดังนี้

ตัวอย่างที่ 3-22

```
int x = -16;           // เท่ากับ 1111111111100000 ฐานสอง
int y = x >> 3;        // เลื่อนบิตของตัวแปร x ไปทางขวา 3 ครั้ง
                        // ได้ผลลัพธ์เป็น 111111111111110 ฐานสอง
```

ถ้าต้องการเลื่อนบิตไปทางขวามือแล้วให้ค่า 0 มาเติมยังบิตซ้ายมือสุด (ซึ่งเกิดกับกรณีที่ตัวแปรเป็นประเภท `unsigned int`) ทำได้โดยใช้การเปลี่ยนประเภทตัวแปรชั่วคราว (typecast) เพื่อเปลี่ยนให้ตัวแปร x เป็น `unsigned int` ชั่วคราวดังตัวอย่างต่อไปนี้

ตัวอย่างที่ 3-23

```
int x = -16;           // เท่ากับ 1111111111100000 ฐานสอง
int y = unsigned(x) >> 3; // เลื่อนบิตของตัวแปร x (แบบไม่คิดเครื่องหมาย) ไปทางขวา 3 ครั้ง
                        // ได้ผลลัพธ์เป็น 000111111111110 ฐานสอง
```

ถ้าหากระมัดระวังเรื่อง **sign extension** แล้ว ก็จะใช้ตัวกระทำเลื่อนบิตไปทางขวามือสำหรับหาค่าตัวแปรด้วย 2 ยกกำลังต่างๆ ได้ดังตัวอย่าง

ตัวอย่างที่ 3-24

```
int x = 1000;
int y = x >> 3;        // หาค่าของ 1000 ด้วย 8 (มาจาก  $2^3$ ) ทำให้  $y = 125$ 
```

3.7 ไวยากรณ์ภาษาของ Arduino

3.7.1 ; (เซมิโคลอน - semicolon)

ใช้เขียนแจ้งว่า จบคำสั่ง

ตัวอย่างที่ 3-25

```
int a = 13;
```

บรรทัดคำสั่งที่ลึ้มเขียนปิดท้ายด้วยเซมิโคลอน จะทำให้แปลโปรแกรมไม่ผ่าน โดยตัวแปลภาษาอาจจะแจ้งให้ทราบว่า ไม่พบเครื่องหมายเซมิโคลอน หรือแจ้งเป็นการผิดพลาดอื่นๆ บางกรณี ที่ตรวจสอบบรรทัดที่แจ้งว่าเกิดการผิดพลาดแล้วไม่พบที่ผิด ให้ตรวจสอบบรรทัดก่อนหน้านั้น

3.7.2 { } (วงเล็บปีกกา - curly brace)

เครื่องหมายวงเล็บปีกกา เป็นส่วนสำคัญของภาษาซี ใช้ในการกำหนดขอบเขตการทำงานในแต่ละช่วง

วงเล็บปีกกาเปิด { จะต้องเขียนตามด้วยวงเล็บปีกกาปิด } ด้วยเสมอ หรือเรียกว่า วงเล็บต้องครบคู่ ในซอฟต์แวร์ Arduino IDE ที่ใช้เขียนโปรแกรมจะมีความสามารถในการตรวจสอบการครบคู่ของเครื่องหมายวงเล็บ ผู้ใช้งานเพียงแค่คลิกที่วงเล็บ มันจะแสดงวงเล็บที่เหลือซึ่งเป็นคู่ของมัน

สำหรับโปรแกรมเมอร์มือใหม่และโปรแกรมเมอร์ที่ย้ายจากภาษา BASIC มาเป็นภาษา C มักจะสับสนกับการใช้เครื่องหมายวงเล็บ แท้ที่จริงแล้วเครื่องหมายปีกกาปิดนี้เทียบได้กับคำสั่ง RETURN ของ subroutine (function) หรือแทนคำสั่ง ENDIF ในการเปรียบเทียบ และแทนคำสั่ง NEXT ของคำสั่งวนรอบ FOR

เนื่องจากการใช้วงเล็บปีกกาได้หลากหลาย ดังนั้นเมื่อต้องการเขียนคำสั่งที่ต้องใช้เครื่องหมายวงเล็บ เมื่อเขียนวงเล็บเปิดแล้วให้เขียนเครื่องหมายวงเล็บปิดทันที ถัดมาจึงค่อยเคาะปุ่ม Enter ในระหว่างเครื่องหมายวงเล็บเพื่อขึ้นบรรทัดใหม่ แล้วเขียนคำสั่งที่ต้องการ ถ้าทำได้ตามนี้วงเล็บจะครบคู่แน่นอน

สำหรับวงเล็บที่ไม่ครบคู่ ทำให้เกิดความผิดพลาดในขณะคอมไพล์โปรแกรม ถ้าเป็นโปรแกรมขนาดใหญ่จะหาที่ผิดได้ยาก ตำแหน่งที่อยู่ของเครื่องหมายวงเล็บแต่ละตัวจะมีผลอย่างมากต่อไวยากรณ์ของภาษาคอมพิวเตอร์ การย้ายตำแหน่งวงเล็บไปเพียงหนึ่งหรือสองบรรทัด ทำให้โปรแกรมทำงานผิดไป

ตำแหน่งที่ใช้วงเล็บปีกกา

ฟังก์ชัน (Function)

```
void myfunction(datatype argument)
{
    statements(s)
}
```

คำสั่งวนรอบ (Loops)

```
while (boolean expression)
{
    statement(s)
}
do
{
    statement(s)
} while (boolean expression);
for (initialisation; termination condition; incrementing expr)
{
    statement(s)
}
```

คำสั่งทดสอบเงื่อนไข (condition)

```
if (boolean expression)
{
    statement(s)
}
else if (boolean expression)
{
    statement(s)
}
else
{
    statement(s)
}
```

3.7.3 // และ /* ... * หมายเหตุบรรทัดเดียวและหลายบรรทัด

เป็นส่วนของผู้ใช้เขียนเพิ่มเติมว่าโปรแกรมทำงานอย่างไร โดยส่วนที่เป็นหมายเหตุจะไม่ถูกคอมไพล์ ไม่นำไปประมวลผล มีประโยชน์มากสำหรับการตรวจสอบโปรแกรมภายหลังหรือใช้แจ้งให้บุคคลอื่นทราบว่า บรรทัดนี้ใช้ทำอะไร ตัวหมายเหตุภาษา C มี 2 ประเภทคือ

(1) หมายเหตุบรรทัดเดียว เขียนเครื่องหมาย // 2 ตัวหน้าบรรทัด

(2) หมายเหตุหลายบรรทัด เขียนเครื่องหมาย /* ... * คู่กับดอกจัน * คร่อมข้อความที่เป็น

หมายเหตุ เช่น /* blabla */

3.7.4 #define

เป็นคำสั่งที่ใช้งานมากในการกำหนดค่าคงที่ให้กับโปรแกรม เนื่องจากการกำหนดค่าที่ไม่ใช่พื้นที่หน่วยความจำของไมโครคอนโทรลเลอร์แต่อย่างใด เมื่อถึงขั้นตอนแปลภาษาคอมไพเลอร์จะแทนที่ตัวอักษรด้วยค่าที่กำหนดไว้ ใน Arduino จะใช้ คำสั่ง #define ตรงกับภาษา C

รูปแบบ

```
#define constantName value
```

อย่าลืมเครื่องหมาย #

ตัวอย่างที่ 3-26

```
#define ledPin 3 // เป็นการกำหนดให้ตัวแปร ledPin เท่ากับค่าคงที่ 3
```

ท้ายคำสั่ง #define **ไม่ต้องมีเครื่องหมายเซมิโคลอน**

3.7.5 #include

ใช้สั่งให้รวมไฟล์อื่นๆ เข้ากับไฟล์โปรแกรมของเราก่อน แล้วจึงทำการคอมไพล์โปรแกรม

รูปแบบคำสั่ง

```
#include <file>
```

```
#include "file"
```

ตัวอย่างที่ 3-27

```
#include <stdio.h>
```

```
#include "POP32.h"
```

บรรทัดแรกจะสั่งให้เรียกไฟล์ **stdio.h** มารวมกับไฟล์โปรแกรมที่กำลังพัฒนา โดยค้นหาไฟล์จากตำแหน่งที่เก็บไฟล์ระบบของ Arduino โดยปกติเป็นไฟล์มาตรฐานที่มาพร้อมกับ Arduino

บรรทัดที่ 2 สั่งให้รวมไฟล์ **POP32.h** มารวมกับไฟล์โปรแกรมที่กำลังพัฒนา โดยหาไฟล์จากโฟลเดอร์ที่อยู่ของไฟล์ภาษา C ก่อน ปกติเป็นไฟล์ที่ผู้ใช้สร้างขึ้นเอง

3.8 ตัวแปร

ตัวแปรเป็นตัวอักษรหลายตัวๆ ที่กำหนดขึ้นในโปรแกรมเพื่อใช้ในการเก็บค่าข้อมูลต่างๆ เช่น ค่าที่อ่านได้จากตัวตรวจจับที่ต่ออยู่กับขาพอร์ตแอนะล็อกของ Arduino ตัวแปรมีหลายประเภทดังนี้

3.8.1 char : ตัวแปรประเภทตัวอักษร

เป็นตัวแปรที่มีขนาด 1 ไบต์ (8 บิต) มีไว้เพื่อเก็บค่าตัวอักษร ตัวอักษรในภาษา C จะเขียนอยู่ในเครื่องหมายคำพูดขีดเดี่ยว เช่น 'A' (สำหรับข้อความที่ประกอบจากตัวอักษรหลายตัวเขียนต่อกันจะเขียนอยู่ในเครื่องหมายคำพูดปกติ เช่น "ABC") ผู้พัฒนาโปรแกรมสามารถสั่งกระทำทางคณิตศาสตร์กับตัวอักษรได้ ในกรณีจะนำค่ารหัส ASCII ของตัวอักษรมาใช้ เช่น 'A' + 1 มีค่าเท่ากับ 66 เนื่องจากค่ารหัส ASCII ของตัวอักษร A เท่ากับ 65

รูปแบบคำสั่ง

```
char sign = ' ';
```

ตัวอย่างที่ 3-28

```
char var = 'x';
```

var คือชื่อของตัวแปรประเภท char ที่ต้องการ

x คือค่าที่ต้องการกำหนดให้กับตัวแปร ในที่นี้เป็นตัวอักษรหนึ่งตัว

3.8.2 byte : ตัวแปรประเภทตัวเลข 8 บิตหรือ 1 ไบต์

ตัวแปร byte ใช้เก็บค่าตัวเลขขนาด 8 บิต มีค่าได้จาก 0 - 255

ตัวอย่างที่ 3-29

```
byte b = B10010; // แสดงค่าของ b ในรูปของเลขฐานสอง (เท่ากับ 18 เลขฐานสิบ)
```

3.8.3 int : ตัวแปรประเภทตัวเลขจำนวนเต็ม

ย่อมาจาก interger แปลว่า เลขจำนวนเต็ม **int** เป็นตัวแปรพื้นฐานสำหรับเก็บตัวเลข ตัวแปรหนึ่งตัวมีขนาด 4 ไบต์ เก็บค่าจาก -2,147,483,648 ถึง 2,147,483,647 ซึ่งมาจาก -2^{31} (ค่าต่ำสุด) และ $2^{31} - 1$ (ค่าสูงสุด) ในการเก็บค่าตัวเลขติดลบใช้เทคนิคที่เรียกว่า **ทวูคอมพลีเมนต์ (2's complement)** บิตสูงสุดบางครั้งเรียกว่า **บิตเครื่องหมาย** หรือ **sign bit** ถ้ามีค่าเป็น "1" แสดงว่า เป็นค่าติดลบ

รูปแบบคำสั่ง

```
int var = val;
```

พารามิเตอร์

var คือชื่อของตัวแปรประเภท int ที่ต้องการ

val คือค่าที่ต้องการกำหนดให้กับตัวแปร

ตัวอย่างที่ 3-30

```
int ledPin = 13;    // กำหนดให้ตัวแปร ledPin มีค่าเท่ากับ 13
```

เมื่อตัวแปรมีค่ามากกว่าค่าสูงสุดที่เก็บได้ จะเกิดการ “ล้นกลับ” (roll over) ไปยังค่าต่ำสุดที่เก็บได้ และเมื่อมีค่าน้อยกว่าค่าต่ำสุดที่เก็บได้จะล้นกลับไปยังค่าสูงสุด ดังตัวอย่างต่อไปนี้

ตัวอย่างที่ 3-31

```
int x
x = -2,147,483,648;
x = x - 1;    // เมื่อกระทำคำสั่งแล้ว ค่าของ x จะเปลี่ยนจาก -2,147,483,648 เป็น 2,147,483,647
x = 2,147,483,647;
x = x + 1;    // เมื่อกระทำคำสั่งแล้ว ค่าของ x จะเปลี่ยนจาก 2,147,483,647 เป็น -2,147,483,648
```

3.8.4 unsigned int : ตัวแปรประเภทเลขจำนวนเต็มไม่คิดเครื่องหมาย

ตัวแปรประเภทนี้คล้ายกับตัวแปร int แต่จะเก็บเลขจำนวนเต็มบวกเท่านั้น โดยเก็บค่า 0 ถึง 4,294,967,295 ($2^{32}-1$)

รูปแบบคำสั่ง

```
unsigned int var = val;
```

พารามิเตอร์

var คือชื่อของตัวแปร int ที่ต้องการ

val คือค่าที่ต้องการกำหนดให้กับตัวแปร

ตัวอย่างที่ 3-32

```
unsigned int ledPin = 13;    // กำหนดให้ตัวแปร ledPin มีค่าเท่ากับ 13 แบบไม่คิดเครื่องหมาย
```

เมื่อตัวแปรมีค่าสูงสุดจะล้นกลับไปค่าต่ำสุดหากมีการเพิ่มค่าต่อไป และเมื่อมีค่าต่ำสุดจะล้นกลับเป็นค่าสูงสุดเมื่อมีการลดค่าต่อไปอีก ดังตัวอย่าง

ตัวอย่างที่ 3-33

```
unsigned int x
x = 0;
x = x - 1;    // เมื่อกระทำคำสั่งแล้ว ค่าของ x จะเปลี่ยนจาก 0 เป็น 4,294,967,295
x = x + 1;    // เมื่อกระทำคำสั่งแล้ว ค่าของ x จะเปลี่ยนจาก 4,294,967,295 กลับไปเป็น 0
```

3.8.5 long : ตัวแปรประเภทเลขจำนวนเต็ม 32 บิต

เป็นตัวแปรเก็บค่าเลขจำนวนเต็มที่ขยายความจุเพิ่มจากตัวแปร int โดยตัวแปร long หนึ่งตัวกินพื้นที่หน่วยความจำ 32 บิต (4 ไบต์) เก็บค่าได้จาก -2,147,483,648 ถึง 2,147,483,647

รูปแบบคำสั่ง

```
long var = val;
```

พารามิเตอร์

var คือชื่อของตัวแปร long ที่ต้องการ

val คือค่าที่ต้องการกำหนดให้กับตัวแปร

ตัวอย่างที่ 3-34

```
long time;           // กำหนดให้ตัวแปร time เป็นแบบ long
```

3.8.6 unsigned long : ตัวแปรประเภทเลขจำนวนเต็ม 32 บิต แบบไม่คิดเครื่องหมาย

เป็นตัวแปรเก็บค่าเลขจำนวนเต็มบวก ตัวแปรหนึ่งตัวกินพื้นที่หน่วยความจำ 32 บิต (4 ไบต์) เก็บค่าได้จาก 0 ถึง 4,294,967,295 หรือ $2^{32}-1$

รูปแบบคำสั่ง

```
unsigned long var = val;
```

พารามิเตอร์

var คือชื่อของตัวแปร unsigned long ที่ต้องการ

val คือค่าที่ต้องการกำหนดให้กับตัวแปร

ตัวอย่างที่ 3-35

```
unsigned long time;   // กำหนดให้ตัวแปร time เป็นแบบ undigned long (ไม่คิดเครื่องหมาย)
```


3.8.7 float : ตัวแปรประเภทเลขทศนิยม

เป็นตัวแปรสำหรับเก็บค่าเลขทศนิยม ซึ่งนิยมใช้ในการเก็บค่าสัญญาณแอนะล็อก เนื่องจากเก็บค่าได้ละเอียดกว่าตัวแปร int ตัวแปร float เก็บค่าได้จาก $-4.4028235 \times 10^{38}$ ถึง 4.4028235×10^{38} โดยหนึ่งตัวจะกินพื้นที่หน่วยความจำ 32 บิต (4 ไบต์)

ในการคำนวณคณิตศาสตร์กับตัวแปร float จะช้ากว่าการคำนวณของตัวแปร int ดังนั้นจึงพยายามหลีกเลี่ยงการคำนวณกับตัวแปร float ในกรณีกระทำคำสั่งวนรอบที่ต้องทำงานด้วยความเร็วสูงสุดของฟังก์ชันทางเวลาที่ต้องแม่นยำอย่างมาก โปรแกรมเมอร์บางคนจะแปลงตัวเลขทศนิยมให้เป็นเลขจำนวนเต็มก่อนแล้วจึงคำนวณเพื่อให้ทำงานได้เร็วขึ้น

รูปแบบคำสั่ง

```
float var = val;
```

พารามิเตอร์

var คือชื่อของตัวแปร float ที่ต้องการ

val คือค่าที่ต้องการกำหนดให้กับตัวแปร

ตัวอย่างที่ 3-36

```
float myfloat;
float sensorCalbrate = 1.117;
```

ตัวอย่างที่ 3-37

```
int x;
int y;
float z;
x = 1;
y = x / 2;           // y เท่ากับ 0 ไม่มีการเก็บค่าของเศษที่ได้จากการหาร
z = (float)x / 2.0;   // z เท่ากับ 0.5
```

เมื่อมีการใช้ตัวแปรแบบ float ตัวเลขที่นำมากระทำกับตัวแปรแบบ float นี้จะต้องเป็นเลขทศนิยมด้วย จากตัวอย่างคือ เลข 2 เมื่อนำมาทำงานกับตัวแปร x ที่เป็นแบบ float เลข 2 จึงต้องเขียนเป็น 2.0

3.8.8 double : ตัวแปรประเภทเลขทศนิยมความละเอียดสองเท่า

เป็นตัวแปรทศนิยมความละเอียดสองเท่า มีขนาด 8 ไบต์ ค่าสูงสุดที่เก็บได้คือ $1.7976931348623157 \times 10^{308}$ ใน Arduino มีหน่วยความจำจำกัด จึงไม่ใช่ตัวแปรประเภทนี้

 POP-Note

การกำหนดค่าคงที่เลขจำนวนเต็มเป็นเลขฐานต่าง ๆ ของ Arduino

ค่าคงที่เลขจำนวนเต็มก็คือตัวเลขที่คุณเขียนในโปรแกรมของ Arduino โดยตรงเช่น 123 โดยปกติแล้วตัวเลขเหล่านี้จะเป็นเลขฐานสิบ (decimal) ถ้าต้องการกำหนดเป็นเลขฐานอื่นจะต้องใช้เครื่องหมายพิเศษระบุ เช่น

ฐาน	ตัวอย่าง
10 (decimal)	123
2 (binary)	B1111011
8 (octal)	0173
16 (hexadecimal)	0x7B

Decimal ก็คือเลขฐานสิบ ซึ่งใช้ในชีวิตประจำวัน

ตัวอย่าง $101 = 101$ มาจาก $(1 * 10^2) + (0 * 10^1) + (1 * 10^0) = 100 + 0 + 1 = 101$

Binary เป็นเลขฐานสอง ตัวเลขแต่ละหลักเป็นได้แค่ 0 หรือ 1

ตัวอย่าง $B101 = 5$ ฐานสิบ มาจาก $(1 * 2^2) + (0 * 2^1) + (1 * 2^0) = 4 + 0 + 1 = 5$

เลขฐานสองจะใช้งานได้ไม่เกิน 8 บิต (ไม่เกิน 1 ไบต์) มีค่าจาก 0 (B0) ถึง 255 (B11111111)

Octal เป็นเลขฐานแปด ตัวเลขแต่ละหลักมีค่าจาก 0 ถึง 7 เท่านั้น

ตัวอย่าง $0101 = 65$ ฐานสิบ มาจาก $(1 * 8^2) + (0 * 8^1) + (1 * 8^0) = 64 + 0 + 1 = 65$

ข้อควรระวังในการกำหนดค่าคงที่ **อย่าเผลอใส่เลข 0 นำหน้า** มิฉะนั้นตัวคอมไพเลอร์จะแปลความหมายผิดไปว่าค่าตัวเลขเป็นเลขฐาน 8

Hexadecimal (hex) เป็นเลขฐานสิบหก ตัวเลขแต่ละหลักมีค่าจาก 0 ถึง 9 และตัวอักษร A คือ 10, B คือ 11 ไปจนถึง F ซึ่งเท่ากับ 15

ตัวอย่าง $0x101 = 257$ ฐานสิบ มาจาก $(1 * 16^2) + (0 * 16^1) + (1 * 16^0) = 256 + 0 + 1 = 257$

3.8.9 string : ตัวแปรประเภทข้อความ

เป็นตัวแปรเก็บข้อความ ซึ่งในภาษา C จะนิยามเป็นอะเรย์ของตัวแปรประเภท char

ตัวอย่างที่ 3-38 ตัวอย่างการประกาศตัวแปรสตริง

```
char Str1[15];
char Str2[8] = {'a','r','d','u','i','n','o'};
char Str3[8] = {'a','r','d','u','i','n','o','\0'};
char Str4[] = "arduino";
char Str5[8] = "arduino";
char Str6[15] = "arduino";
```

- Str1 เป็นการประกาศตัวแปรสตริงโดยไม่ได้กำหนดค่าเริ่มต้น
- Str2 ประกาศตัวแปรสตริงพร้อมกำหนดค่าให้กับข้อความที่ละตัวอักษร หากไม่ครบตามจำนวนที่ประกาศ คอมไพเลอร์จะเพิ่ม null string ให้เองจนครบ (จากตัวอย่างประกาศไว้ 8 ตัว แต่ข้อความมี 7 ตัวอักษร จึงมีการเติม null string ให้อีก 1 ตัว
- Str3 ประกาศตัวแปรสตริงพร้อมกำหนดค่าให้กับข้อความ แล้วปิดท้ายด้วยตัวอักษรปิด นั่นคือ \0
- Str4 ประกาศตัวแปรสตริงพร้อมกำหนดค่าตัวแปรในเครื่องหมายคำพูด จากตัวอย่าง ไม่ได้กำหนดขนาดตัวแปร คอมไพเลอร์จะกำหนดขนาดให้เองตามจำนวนตัวอักษร
- Str5 ประกาศตัวแปรสตริงพร้อมกำหนดค่าตัวแปรในเครื่องหมายคำพูด และขนาดของตัวแปร จากตัวอย่าง ประกาศไว้ 8 ตัว
- Str6 ประกาศตัวแปรสตริง โดยกำหนดขนาดเผื่อไว้สำหรับข้อความอื่นที่ยาวมากกว่านี้

3.8.9.1 การเพิ่มตัวอักษรแจ้งว่าจบข้อความ (null termination)

ในตัวแปรสตริงของภาษา C กำหนดให้ตัวอักษรสุดท้ายเป็นตัวแจ้งการจบข้อความ (null string) ซึ่งก็คือตัวอักษร \0 ในการกำหนดขนาดของตัวแปร (ค่าในวงเล็บเหลี่ยม) จะต้องกำหนดให้เท่ากับจำนวนตัวอักษร + 1 ดังในตัวแปร Str2 และ Str3 ในตัวอย่างที่ 3-38 ที่ข้อความ **Arduino** มีอักษร 7 ตัว ในการประกาศตัวแปรต้องระบุเป็น [8]

ในการประกาศตัวแปรสตริง ต้องเผื่อพื้นที่สำหรับเก็บตัวอักษรแจ้งว่าจบข้อความ มิฉะนั้น คอมไพเลอร์จะแจ้งเตือนว่าเกิดการผิดพลาด ในตัวอย่างที่ 3-38 ตัวแปร Str1 และ Str6 เก็บข้อความ ได้สูงสุด 14 ตัวอักษร

3.8.9.2 เครื่องหมายคำพูดขีดเดียวและสองขีด

ปกติแล้วจะกำหนดค่าตัวแปรสตริงภายในเครื่องหมายคำพูด เช่น "Abc" สำหรับตัวแปรตัวอักษร (char) จะกำหนดค่าภายในเครื่องหมายคำพูดขีดเดียว 'A'

3.8.10 ตัวแปรอะเรย์ (array)

ตัวแปรอะเรย์เป็นตัวแปรหลายตัวที่ถูกเก็บรวมอยู่ในตัวแปรชื่อเดียวกัน โดยอ้างถึงตัวแปรแต่ละตัวด้วยหมายเลขดัชนีที่เขียนอยู่ในวงเล็บสี่เหลี่ยม ตัวแปรอะเรย์ของ Arduino จะอ้างอิงตามภาษา C ตัวแปรอะเรย์อาจดูซับซ้อน แต่ถ้าใช้ตัวแปรอะเรย์อย่างตรงไปตรงมาจะง่ายต่อการทำความเข้าใจ

ตัวอย่างที่ 3-39 ตัวอย่างการประกาศตัวแปรอะเรย์

```
int myInts[6];
int myPins[] = {2, 4, 8, 3, 6};
int mySensVals[6] = {2, 4, -8, 3, 2};
char message[6] = "hello";
```

- ผู้พัฒนาโปรแกรมสามารถประกาศตัวแปรอะเรย์ได้โดยยังไม่กำหนดค่าของตัวแปร myInts
- การประกาศตัวแปร myPins เป็นการประกาศตัวแปรอะเรย์โดยไม่ระบุขนาด เมื่อประกาศตัวแปรแล้วต้องกำหนดค่าทันที เพื่อให้คอมไพเลอร์นับว่า ตัวแปรมีสมาชิกกี่ตัวและกำหนดค่าได้ถูกต้อง จากตัวอย่างมีทั้งสิ้น 5 ตัว
- ในการประกาศตัวแปรอะเรย์ ผู้พัฒนาโปรแกรมสามารถประกาศและกำหนดขนาดของตัวแปรอะเรย์ได้ ในพร้อมกันดังตัวอย่างการประกาศตัวแปร mySensVals ที่ประกาศทั้งขนาดและกำหนดค่า
- ตัวอย่างสุดท้ายเป็นการประกาศอะเรย์ของตัวแปร message ที่เป็นแบบ char มีตัวอักษร 5 ตัวคือ hello แต่การกำหนดขนาดของตัวแปรจะต้องเผื่อที่สำหรับเก็บตัวอักษรแจ้งจบข้อความด้วย จึงทำให้ค่าดัชนีต้องกำหนดเป็น 6

3.8.10.1 การใช้งานตัวแปรอะเรย์

พิมพ์ชื่อตัวแปรพร้อมกับระบุค่าดัชนีภายในเครื่องหมายวงเล็บสี่เหลี่ยม ค่าดัชนีของตัวแปรอะเรย์เริ่มต้นด้วย 0 ดังนั้นค่าของตัวแปร mySensVals มีค่าดังนี้

```
mySensVals[0] == 2, mySensVals[1] == 4
```

การกำหนดค่าให้กับตัวแปรอะเรย์ ทำได้ดังนี้

```
mySensVals[0] = 10;
```

การเรียกค่าสมาชิกของตัวแปรอะเรย์ ทำได้ดังนี้

```
x = mySensVals[4];
```

3.8.10.2 อะเรย์และคำสั่งวนรอบ for

โดยทั่วไปจะพบการใช้งานตัวแปรอะเรย์ภายในคำสั่ง for โดยใช้ค่าตัวแปรนับรอบของคำสั่ง for เป็นค่าดัชนีของตัวแปรอะเรย์ ดังตัวอย่างต่อไปนี้

ตัวอย่างที่ 3-40

```
int i;
for (i = 0; i < 5; i = i + 1)
{
  Serial.println(myPins[i]); // แสดงค่าสมาชิกของตัวแปรอะเรย์ที่หน้าต่าง Serial monitor
}
```

3.9 ขอบเขตของตัวแปร

ตัวแปรในภาษา C ที่ใช้ใน Arduino จะมีคุณสมบัติที่เรียกว่า “ขอบเขตของตัวแปร” (scope) ซึ่งแตกต่างจากภาษา BASIC ซึ่งตัวแปรทุกตัวมีสถานะเท่าเทียมกันหมดคือ เป็นแบบ global

3.9.1 ตัวแปรโลคอลและโกลบอล

ตัวแปรแบบโกลบอล (global variable) เป็นตัวแปรที่ทุกฟังก์ชันในโปรแกรมรู้จัก โดยต้องประกาศตัวแปร นอกฟังก์ชัน สำหรับตัวแปรแบบโลคอลหรือตัวแปรท้องถิ่นเป็นตัวแปรที่ประกาศตัวแปรอยู่ภายในเครื่องหมายวงเล็บปีกกาของฟังก์ชัน และรู้จักเฉพาะภายในฟังก์ชันนั้น

เมื่อโปรแกรมเริ่มมีขนาดใหญ่และซับซ้อนมากขึ้น การใช้ตัวแปรโลคอลจะมีประโยชน์มาก เนื่องจากแน่ใจได้ว่ามีแค่ฟังก์ชันนั้นเท่านั้นที่สามารถใช้งานตัวแปร ช่วยป้องกันการเกิดการผิดพลาดเมื่อฟังก์ชันทำการแก้ไขค่าตัวแปรที่ใช้งานโดยฟังก์ชันอื่น

ตัวอย่างที่ 3-41

```
int gPWMval;      // ทุกฟังก์ชันมองเห็นตัวแปรนี้
void setup()
{
  void loop()
  {
    int i;          // ตัวแปร i จะถูกมองเห็นและใช้งานภายในฟังก์ชัน loop เท่านั้น
    float f;        // ตัวแปร f จะถูกมองเห็นและใช้งานภายในฟังก์ชัน loop เท่านั้น
  }
}
```

3.9.2 ตัวแปรสแตติก (static)

เป็นคำสงวน (keyword) ที่ใช้ตอนประกาศตัวแปรที่มีขอบเขตใช้งานแค่ภายในฟังก์ชันเท่านั้น โดยต่างจากตัวแปรโลคอลตรงที่ตัวแปรแบบโลคอลจะถูกสร้างและลบทิ้งทุกครั้งที่เราเรียกใช้ฟังก์ชัน สำหรับตัวแปรสแตติกเมื่อจบการทำงานของฟังก์ชันค่าตัวแปรจะยังคงอยู่ (ไม่ถูกลบทิ้ง) เป็นการรักษาค่าตัวแปรไว้ระหว่างการเรียกใช้งานฟังก์ชัน

ตัวแปรที่ประกาศเป็น static จะถูกสร้างและกำหนดค่าในครั้งแรกที่เราเรียกใช้ฟังก์ชัน

3.10 ค่าคงที่ (constants)

ค่าคงที่เป็น กลุ่มตัวอักษรหรือข้อความที่ได้กำหนดค่าไว้ล่วงหน้าแล้ว ตัวคอมไพเลอร์ของ Arduino จะรู้จักกับค่าคงที่เหล่านี้แล้ว ไม่จำเป็นต้องประกาศหรือกำหนดค่าคงที่

3.10.1 HIGH, LOW : ใช้กำหนดค่าทางตรรกะ

ในการอ่านหรือเขียนค่าให้กับขาพอร์ตดิจิตอล ค่าที่เป็นได้มี 2 ค่าคือ HIGH หรือ LOW

HIGH เป็นการกำหนดค่าให้ขาดิจิตอลนั้นมีแรงดันเท่ากับ +5V ส่วนการอ่านค่า ถ้าอ่านได้ +3V หรือมากกว่า ไมโครคอนโทรลเลอร์จะอ่านค่าได้เป็น HIGH ค่าคงที่ของ HIGH คือ “1” หรือเทียบเป็นตรรกะคือ จริง (true)

LOW เป็นการกำหนดค่าให้ขาดิจิตอลนั้นมีแรงดันเท่ากับ 0V ส่วนการอ่านค่า ถ้าอ่านได้ +2V หรือน้อยกว่า ไมโครคอนโทรลเลอร์จะอ่านค่าได้เป็น LOW ค่าคงที่ของ LOW คือ “0” หรือเทียบเป็นตรรกะคือ เท็จ (false)

3.10.2 INPUT, OUTPUT : กำหนดทิศทางของขาพอร์ตดิจิตอล

ขาพอร์ตดิจิตอลทำหน้าที่ได้ 2 อย่างคือ เป็นอินพุตและเอาต์พุต

เมื่อกำหนดเป็น **INPUT** หมายถึง กำหนดให้ขาพอร์ตนั้นๆ เป็นขาอินพุต

เมื่อกำหนดเป็น **OUTPUT** หมายถึง กำหนดให้ขาพอร์ตนั้นๆ เป็นขาเอาต์พุต

3.11 ตัวกระทำอื่นๆ ที่เกี่ยวข้องกับตัวแปร

3.11.1 cast : การเปลี่ยนประเภทตัวแปรชั่วคราว

cast เป็นตัวกระทำที่ใช้สั่งให้เปลี่ยนประเภทของตัวแปรไปเป็นประเภทอื่น และบังคับให้คำนวณค่าตัวแปรเป็นประเภทใหม่

รูปแบบคำสั่ง

(type) variable

เมื่อ Type เป็นประเภทของตัวแปรใดๆ (เช่น int, float, long)

Variable เป็นตัวแปรหรือค่าคงที่ใดๆ

ตัวอย่างที่ 3-42

```
int i;
float f;
f = 4.6;
i = (int) f;
```

ในการเปลี่ยนประเภทตัวแปรจาก float เป็น int ค่าที่ได้จะถูกตัดเศษออก ดังนั้น (int)4.6 จึงกลายเป็น 4

3.11.2 sizeof : แจ้งขนาดของตัวแปร

ใช้แจ้งบอกจำนวนไบนารีของตัวแปรที่ต้องการทราบค่า ซึ่งเป็นทั้งตัวแปรปกติและตัวแปรอะเรย์

รูปแบบคำสั่ง

เขียนได้สองแบบดังนี้

sizeof(variable)

sizeof variable

เมื่อ Variable คือตัวแปรปกติหรือตัวแปรอะเรย์ (int, float, long) ที่ต้องการทราบขนาด

ตัวอย่างที่ 3-43

ตัวกระทำ sizeof มีประโยชน์อย่างมากในการจัดการกับตัวแปรอะเรย์ (รวมถึงตัวแปรสตริง)

ตัวอย่างต่อไปนี้จะพิมพ์ข้อความออกทางพอร์ตอนุกรมครั้งละหนึ่งตัวอักษร

```
char myStr[] = "this is a test";
int i;
void setup()
{
    Serial.begin(9600);
}
void loop()
{
    for (i = 0; i < sizeof(myStr) - 1; i++)
    {
        Serial.print(i, DEC);
        Serial.print(" = ");
        Serial.println(myStr[i], BYTE);
    }
}
```

3.12 คำสงวนของ Arduino

คำสงวนคือ คำคงที่ ตัวแปร และฟังก์ชันที่ได้กำหนดไว้เป็นส่วนหนึ่งของภาษา C ของ Arduino ห้ามนำคำเหล่านี้ไปตั้งชื่อตัวแปร สามารถแสดงได้ดังนี้

# Constants	# Datatypes	glcdFillScreen	# Other
HIGH	boolean	glcdClear	abs
LOW	byte	glcdPixel	acos
INPUT	char	glcdRect	+=
OUTPUT	class	glcdFillRect	+
SERIAL	default	glcdLine	[]
DISPLAY	do	glcdCircle	asin
PI	double	glcdFillCircle	=
HALF_PI	int	glcdArc	atan
TWO_PI	long	getdist	atan2
LSBFIRST	private	in	&
MSBFIRST	protected	oled	&=
CHANGE	public	out	
FALLING	return	motor	=
RISING	short	motor_stop	boolean
false	signed	fd	byte
true	static	bk	case
null	switch	fd2	ceil
	throw	tl	char
# Literal Constants	try	tr	char
GLCD_RED	unsigned	sl	class
GLCD_GREEN	void	sr	,
GLCD_BLUE	# Methods/Fucntions	servo	//
GLCD_YELLOW	sw_ok	sound	?:
GLCD_BLACK	sw_ok_press	beep	constrain
GLCD_WHITE	analog	uart_set_baud	cos
GLCD_SKY	knob	uart_get_baud	{}
GLCD_MAGENTA	glcd	uart_putc	—
# Port Constants	glcdChar	uart_puts	default
DDRB	glcdString	uart	delay
PINB	glcdMode	uart_available	delayMicroseconds
PORTB	glcdGetMode	uart_getkey	/
DDRC	glcdFlip	uart1_set_baud	/**
PINC	glcdGetFlip	uart1_get_baud	.
PORTC	colorRGB	uart1_putc	else
DDRD	setTextColor	uart1_puts	==
PIND	setTextBackgroundColor	uart1	exp
PORTD	setTextSize	uart1_available	false
# Names	getTextColor	uart1_getkey	float
popxt	getTextBackgroundColor	uart1_flush	float
	getTextSize		floor
			for
			<
			<=
			HALF_PI
			if
			++
			!=

int	new	while	printString
<<	null	Serial	printInteger
<	()	begin	printByte
<=	PII	read	printHex
log	return	print	printOctal
&&	>>	write	printBinary
!	;	println	printNewline
	Serial	available	pulseIn
^	Setup	digitalWrite	shiftOut
^=	sin	digitalRead	compass_read
loop	sq	pinMode	compass_set_heading
max	sqrt	analogRead	compass_read_heading
millis	=	analogWrite	compass_writeConfig
min	switch	attachInterrupts	compass_readConfig
-	tan	detachInterrupts	compass_scan
%	this	beginSerial	compass_scan
/*	true	serialWrite	encoder
*	TWO_PI	serialRead	popx2
	void	serialAvailable	POP32



บทที่ 4

ฟังก์ชันพื้นฐานของ Arduino และตัวอย่างคำสั่ง
สำหรับการพัฒนาโปรแกรม

ทางผู้จัดทำซอฟต์แวร์ Arduino IDE ได้จัดเตรียมฟังก์ชันพื้นฐาน เช่น ฟังก์ชันเกี่ยวกับขาพอร์ต อินพุตเอาต์พุตดิจิทัล, อินพุตเอาต์พุตแอนะล็อก เป็นต้น ดังนั้นในการเขียนโปรแกรมจึงเรียกใช้ฟังก์ชันเหล่านี้ได้ทันที

นอกจากฟังก์ชันพื้นฐานเหล่านี้แล้ว นักพัฒนาท่านอื่นๆ ที่ร่วมในโครงการ Arduino นี้ก็ได้เพิ่มไลบรารีอื่นๆ เช่น ไลบรารีควบคุมมอเตอร์, การติดต่อกับอุปกรณ์บัส I²C ฯลฯ ในการเรียกใช้งานต้องเพิ่มบรรทัด `#include` เพื่อผนวกไฟล์ที่เหมาะสมก่อน จึงจะเรียกใช้ฟังก์ชันได้

ในบทนี้จะอธิบายถึงการเรียกใช้ฟังก์ชันและตัวอย่างโปรแกรมสำหรับทำการทดลอง โดยใช้บอร์ด POP-32 เป็นอุปกรณ์หลักในการทดลอง

4.1 ฟังก์ชันอินพุตเอาต์พุตดิจิทัล (Digital I/O)

4.1.1 คำอธิบายและการเรียกใช้ฟังก์ชัน

4.1.1.1 `pinMode (pin, mode)`

ใช้กำหนดขาพอร์ตใดๆ ให้เป็นพอร์ตดิจิทัล

พารามิเตอร์

pin - หมายเลขขาพอร์ตของบอร์ด POP-32 (ค่าเป็น int)

mode - โหมดการทำงานเป็น INPUT (อินพุต) หรือ OUTPUT (เอาต์พุต) (ค่าเป็น int) ต้องใช้ตัวพิมพ์ใหญ่

ตัวอย่างที่ 4-1

```
int ledPin = PC13;           // กำหนดชื่อตัวแปรให้แก่ LED ที่พอร์ต PC13
void setup()
{
  pinMode(ledPin, OUTPUT);    // กำหนดเป็นเอาต์พุต
}
void loop()
{
  digitalWrite(ledPin, HIGH); // LED ติดสว่าง
  delay(1000);                // หน่วงเวลา 1 วินาที
  digitalWrite(ledPin, LOW);  // LED ดับ
  delay(1000);                // หน่วงเวลา 1 วินาที
}
```

4.1.1.2 digitalWrite(pin, value)

สั่งงานให้ขาพอร์ตรับค่าสถานะเป็นลอจิกสูง (HIGH หรือ “1”) หรือลอจิกต่ำ (LOW หรือ “0”)

พารามิเตอร์

pin - หมายเลขขาพอร์ตของบอร์ด POP-32 (ค่าเป็น int)

value - มีค่า HIGH หรือ LOW

ตัวอย่างที่ 4-2

```
int ledPin = PC13;           // กำหนดชื่อตัวแปรแก่ LED ที่พอร์ต PC13
void setup()
{
  pinMode(ledPin, OUTPUT);    // กำหนดเป็นเอาต์พุต
}
void loop()
{
  digitalWrite(ledPin, HIGH); // LED ติดสว่าง
  delay(500);                 // หน่วงเวลา 0.5 วินาที
  digitalWrite(ledPin, LOW);  // LED ดับ
  delay(500);                 // หน่วงเวลา 0.5 วินาที
}
```

กำหนดให้พอร์ต PC13 เป็น HIGH (มีลอจิกเป็น “1”) หน่วงเวลา 0.5 วินาที แล้วจึงสั่งให้กลับเป็น LOW (มีลอจิกเป็น “0”) อีกครั้ง วนเช่นนี้ไปตลอด

4.1.1.3 int digitalRead(pin)

อ่านค่าสถานะของขาที่ระบุไว้ว่ามีค่าเป็น HIGH หรือ LOW

พารามิเตอร์

pin - ขาพอร์ตที่ต้องการอ่านค่า ซึ่งต้องเป็นขาพอร์ตดิจิทัล

ค่าที่ส่งกลับ

เป็น HIGH หรือ LOW

ตัวอย่างที่ 4-3

```
int ledPin = PBD;           // กำหนดชื่อตัวแปรแก่ LED ที่พอร์ต PBD
int inPin = PAB;            // กำหนดชื่อตัวแปรแก่สวิตช์ที่พอร์ต PAB
int val = 0;                // กำหนดตัวแปรสำหรับเก็บค่าที่อ่านได้จากอินพุต
void setup()
{
  pinMode(ledPin, OUTPUT);   // กำหนดให้พอร์ต PBD เป็นเอาต์พุต
  pinMode(inPin, INPUT);     // กำหนดให้พอร์ต PAB เป็นอินพุต
}
```

```
void loop()
{
    val = digitalRead(inPin);           // อ่านค่าจากพอร์ตอินพุต
    digitalWrite(ledPin, val);         // แสดงค่าที่อ่านได้ที่พอร์ตเอาต์พุต ในที่นี้คือ พอร์ต AG
}
```

เมื่อไม่กดสวิตช์ สถานะที่พอร์ต AG เป็น “1” ทำให้พอร์ต PC13 มีสถานะเป็น “1” เช่นกัน LED ที่ต่อกับพอร์ต PBO จึงติดสว่าง เมื่อกดสวิตช์ สถานะลอจิกที่พอร์ต PA6 จะเป็น “0” พอร์ต PBO จึงมีสถานะเป็น “0” เช่นกัน LED ที่ต่ออยู่จึงดับ

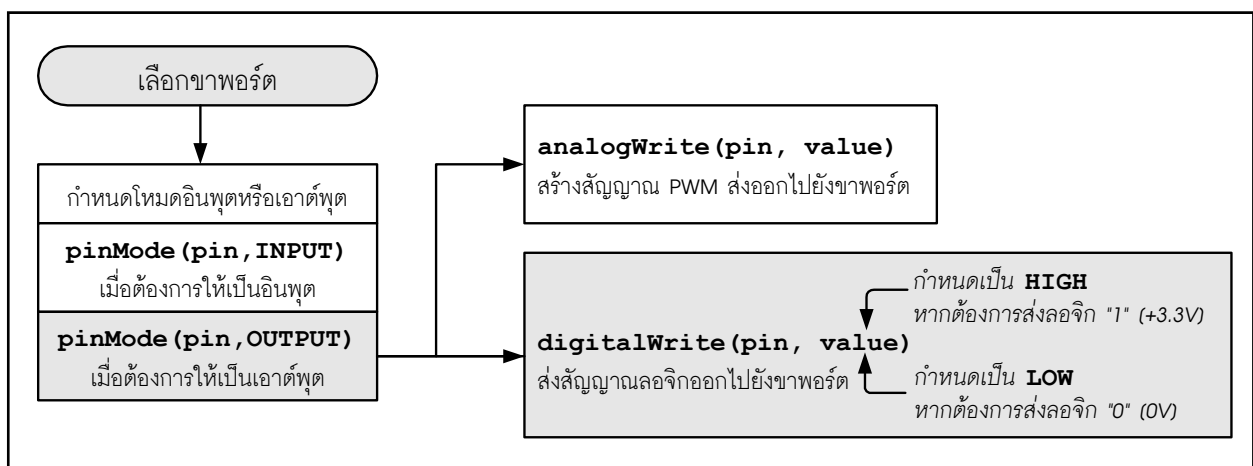
หมายเหตุ : ขาพอร์ตในโปรแกรมนี้อ้างอิงถึงจุดต่อพอร์ตของบอร์ด POP-32

4.1.2 การใช้งานพอร์ตเอาต์พุตดิจิทัลของบอร์ด POP-32

4.1.2.1 คุณสมบัติของขาพอร์ตเอาต์พุต

ขาพอร์ตที่กำหนดให้เป็นเอาต์พุตผ่านทางฟังก์ชัน `pinMode()` จะมีสถานะเป็นอิมพีแดนซ์ต่ำ ทำให้จ่ายกระแสไฟฟ้าให้กับวงจรภายนอกได้สูงถึง 20mA ซึ่งเพียงพอสำหรับขับกระแสไฟฟ้าให้ LED สว่าง (ต้องต่อตัวต้านทานอนุกรมด้วย) หรือใช้กับตัวตรวจจับต่างๆ ได้ แต่ไม่เพียงพอสำหรับขับรีเลย์ โซลินอยด์ หรือมอเตอร์

ก่อนที่จะใช้งานขาดิจิทัลของบอร์ด POP-32 จะต้องสังเกตรหัสขาพอร์ตนี้ทำหน้าที่เป็นอินพุตหรือเอาต์พุต ในหัวข้อนี้จะทดลองต่อเป็นเอาต์พุต ถ้ากำหนดให้เป็นเอาต์พุตสามารถรับหรือจ่ายกระแสไฟฟ้าได้สูงสุดถึง 20mA ซึ่งเพียงพอสำหรับขับ LED



รูปที่ 4-1 ไดอะแกรมแสดงกลไกการกำหนดให้ขาพอร์ตของ Arduino และบอร์ด POP-32 เป็นพอร์ตเอาต์พุตดิจิทัล (บล็อกสีเทาแทนการทำงานหรือเงื่อนไขที่ต้องการ)

4.1.2.2 การกำหนดโหมดของขาพอร์ต

ก่อนใช้งานต้องกำหนดโหมดการทำงานของขาพอร์ตดิจิทัลให้เป็นอินพุตหรือเอาต์พุต กำหนดได้จากฟังก์ชัน `pinMode()` มีรูปแบบดังนี้

```
pinmode (pin, mode) ;
```

เมื่อ `pin` คือ หมายเลขขาที่ต้องการ

`Mode` คือ โหมดการทำงาน (INPUT หรือ OUTPUT)

หลังจากที่กำหนดให้เป็นเอาต์พุตแล้วเมื่อต้องการเขียนค่าไปยังขาอื่นๆ ให้เรียกใช้ฟังก์ชัน `digitalWrite()` โดยมีรูปแบบดังนี้

```
digitalWrite (pin, value) ;
```

เมื่อ `pin` คือหมายเลขขาที่ต้องการ

`value` สถานะลอจิกที่ต้องการ (HIGH หรือ LOW)

4.1.3 การทดลองอินพุตดิจิทัลของบอร์ด POP-32

4.1.5.1 คุณสมบัติของขาพอร์ตอินพุต

ขาพอร์ตของ Arduino และบอร์ด **POP-32** จะถูกกำหนดเป็นอินพุตตั้งแต่เริ่มต้น จึงไม่จำเป็นต้องใช้ฟังก์ชัน `pinMode()` ในการกำหนดให้เป็นอินพุต ขาพอร์ตที่ถูกกำหนดเป็นอินพุตจะมีสถานะเป็นอิมพีแดนซ์สูง ทำให้มีความต้องการกระแสไฟฟ้าจากอุปกรณ์ที่ต้องการอ่านค่าอินพุตน้อยมาก ทำให้ไม่สามารถรับหรือจ่ายกระแสไฟฟ้าให้กับวงจรภายนอก จึงนำขาที่เป็นอินพุตนี้ไปใช้งานบางประเภท เช่น สร้างตัวตรวจจับการสัมผัสที่อาศัยการวัดค่าความจุไฟฟ้า

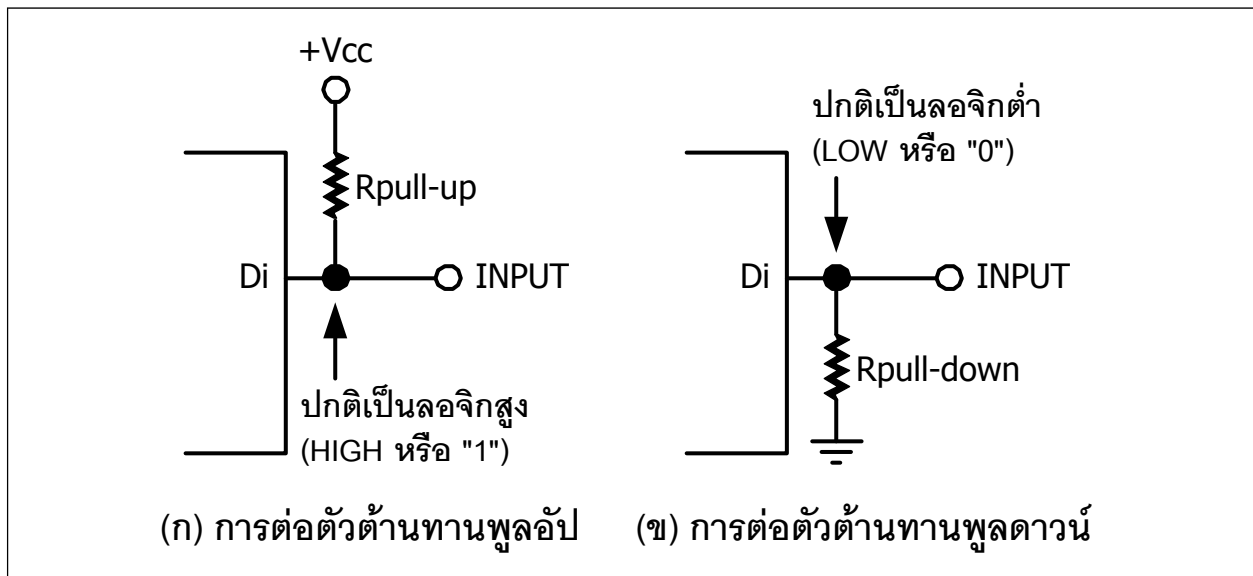
สำหรับขาอินพุตดิจิทัลเมื่อไม่มีอินพุตป้อนให้จะต้องกำหนดค่าแรงดันไฟฟ้าให้แน่นอนทำได้โดยต่อตัวต้านทานพูลอัพ (pull-up resistor) โดยต่อขาของตัวต้านทานขาหนึ่งไปยังไฟเลี้ยง หรือต่อพูลดาวน์ (pull-down) ซึ่งต่อขาหนึ่งของตัวต้านทานจากขาพอร์ตลงกราวด์ ค่าของตัวต้านทานที่ใช้ทั่วไปคือ $10k\Omega$ ดังรูปที่ 4-2

บอร์ด **POP-32** มีจุดต่อพอร์ตดิจิทัลที่กำหนดให้เป็นอินพุตหรือเอาต์พุตจำนวน 11 จุดต่อหรือ 11 ขา ถ้าต้องการกำหนดเป็นอินพุตดิจิทัลต้องกำหนดด้วยฟังก์ชัน `pinMode` และอ่านค่าอินพุตได้จากฟังก์ชัน `digitalRead` ซึ่งมีรูปแบบดังนี้

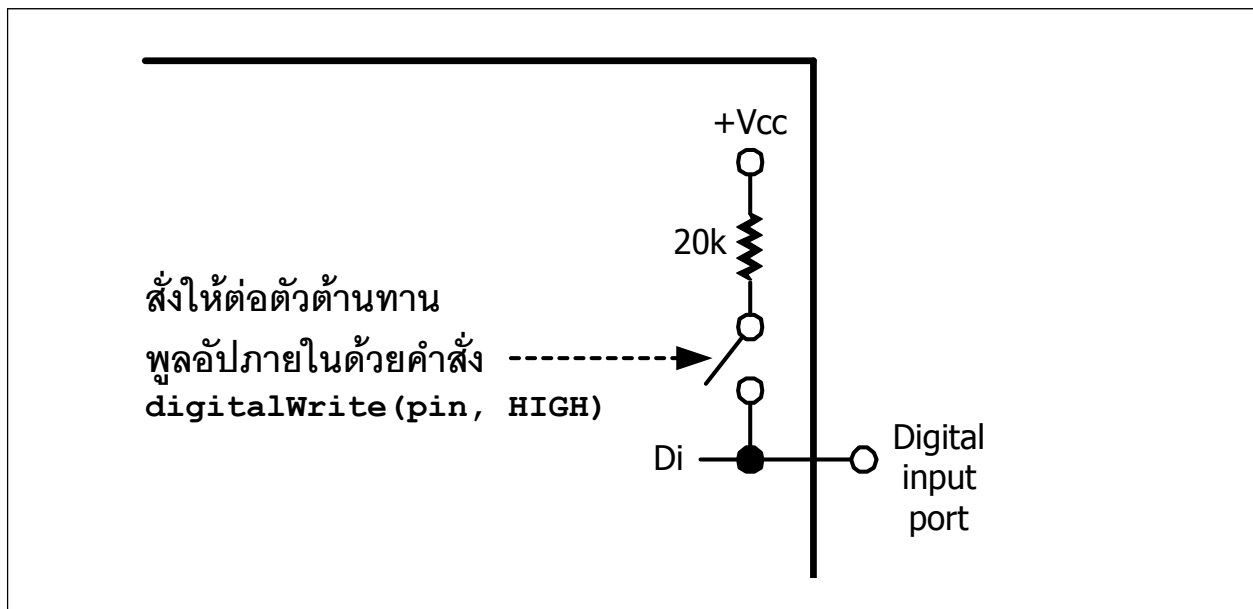
```
digitalRead (pin) ;
```

เมื่อ `pin` คือหมายเลขขาที่ต้องการอ่านค่าสถานะ

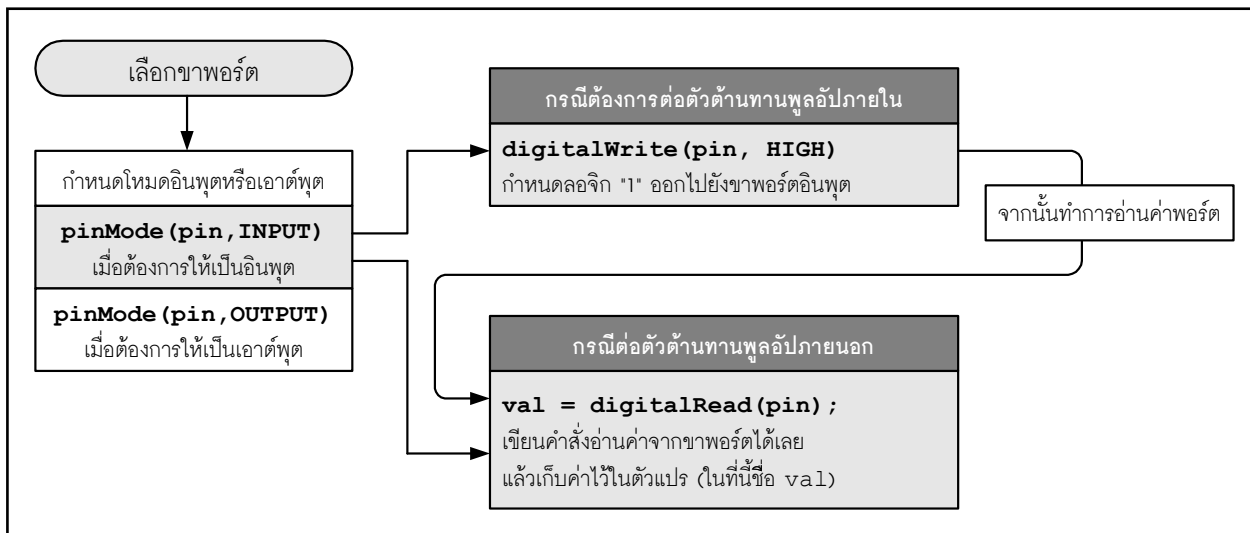
เมื่อฟังก์ชันทำงาน จะคืนค่าเป็นสถานะของขาที่ต้องการอ่านค่า โดยคืนค่าเป็น LOW (ค่าเป็น “0”) หรือ HIGH (ค่าเป็น “1”)



รูปที่ 4-2 แสดงการต่อตัวต้านทานเพื่อกำหนดสถานะของขาพอร์ตอินพุตของไมโครคอนโทรลเลอร์ ในขณะที่ยังไม่มีอินพุตส่งเข้ามา



รูปที่ 4-3 แสดงการต่อตัวต้านทานพูลอัปภายในที่พอร์ตอินพุตดิจิทัลของไมโครคอนโทรลเลอร์ซึ่งควบคุมได้ด้วยกระบวนการทางซอฟต์แวร์



รูปที่ 4-4 ไดอะแกรมแสดงกลไกการกำหนดให้ขาพอร์ตของ Arduino และบอร์ด POP-32 เป็นพอร์ตอินพุตดิจิทัล (บล็อกสีเทาแทนการทำงานหรือเงื่อนไขที่ต้องการ)

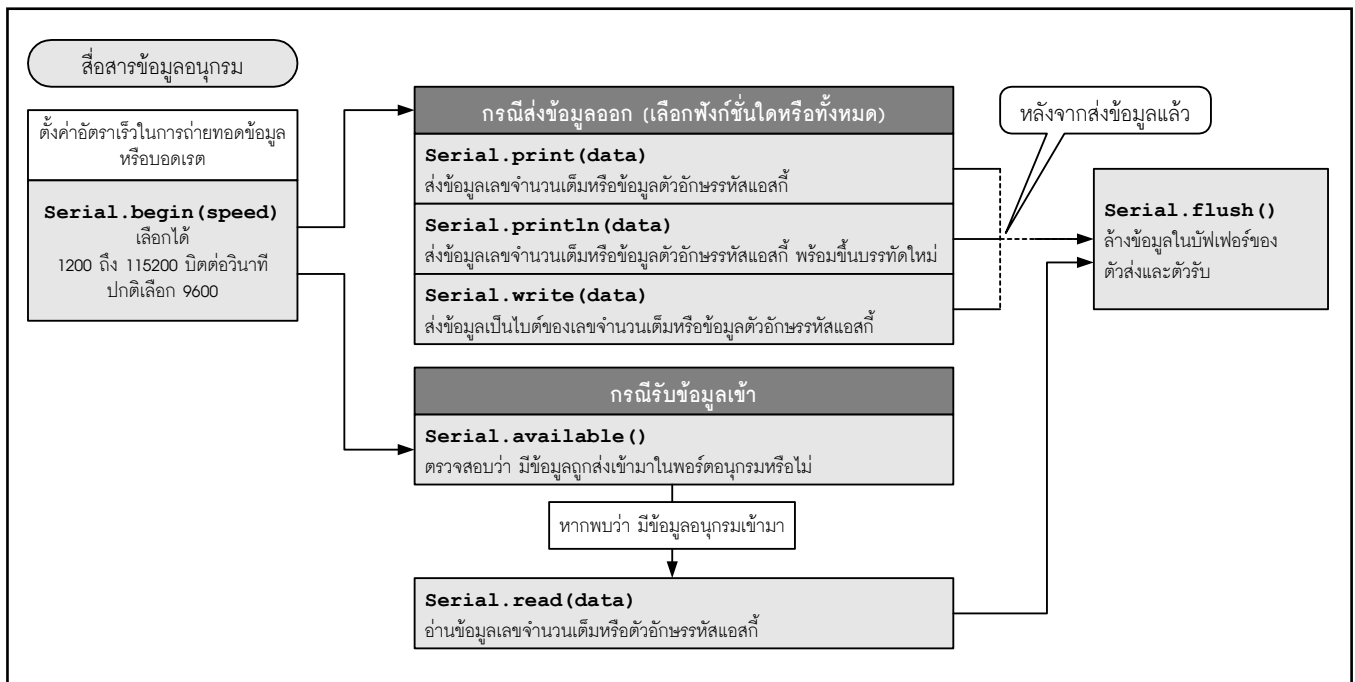
ภายในขาพอร์ตของไมโครคอนโทรลเลอร์ STM32F103CBT6 ซึ่งเป็นไมโครคอนโทรลเลอร์หลักของบอร์ด POP-32 จะมีการต่อตัวต้านทานพูลอัพค่า 20kΩ เตรียมไว้ให้ สำหรับการเพื่อต่อใช้งานผ่านทางซอฟต์แวร์ ดังรูปที่ 4-3

ในรูปที่ 4-4 แสดงไดอะแกรมการเขียนคำสั่งเพื่อกำหนดให้ใช้งานขาพอร์ตของไมโครคอนโทรลเลอร์หลักของบอร์ด POP-32 ทำงานเป็นขาพอร์ตอินพุตดิจิทัล

4.2 ฟังก์ชันเกี่ยวกับการสื่อสารผ่านพอร์ตอนุกรม

ใช้สื่อสารข้อมูลระหว่างฮาร์ดแวร์ Arduino (ในที่นี้คือบอร์ด POP-32) กับคอมพิวเตอร์ โดยกระทำผ่านพอร์ต USB ในรูปที่ 4-5 แสดงกลไกการทำงานของฟังก์ชัน Serial

นอกจากนี้ บอร์ด POP-32 ยังมีพอร์ตสำหรับสื่อสารข้อมูลอนุกรมหรือ UART อีกหนึ่งชุด นั่นคือ UART1 ซึ่งใช้พอร์ตหมายเลข 23 เป็นขา RxD สำหรับรับข้อมูลอนุกรม และพอร์ตหมายเลข 22 เป็นขา TxD สำหรับส่งข้อมูลอนุกรม โดย UART1 นี้ใช้สำหรับเชื่อมต่ออุปกรณ์สื่อสารข้อมูลอนุกรมทั้งแบบมีสายหรือไร้สายอย่างบลูทูธ หรือ XBEE ด้วยระดับสัญญาณ TTL (0 และ +5.3V หรือ +5V) รวมถึงโมดูล Serial WiFi เพื่อช่วยให้บอร์ด POP-32 เชื่อมต่อกับเครือข่ายอินเทอร์เน็ตได้ อันนำไปสู่การพัฒนาโครงงานควบคุมอุปกรณ์ผ่านโครงข่ายข้อมูลในแบบต่างๆ รวมถึงการควบคุมข้ามโลกผ่านเว็บเบราว์เซอร์



รูปที่ 4-5 กลไกการสื่อสารข้อมูลอนุกรมของฟังก์ชัน Serial ใน Arduino

4.2.1 คำอธิบายและการเรียกใช้ฟังก์ชัน

4.2.1.1 Serial.begin(int datarate)

กำหนดค่าอัตราบอดของการรับส่งข้อมูลอนุกรม ในหน่วยบิตต่อวินาที (bits per second : bps หรือ baud) ในกรณีที่ติดต่อกับคอมพิวเตอร์ให้ใช้ค่าต่อไปนี้ 300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600 หรือ 115200

พารามิเตอร์

Int datarate ในหน่วยบิตต่อวินาที (baud หรือ bps)

ตัวอย่างที่ 4-4

```

void setup()
{
    Serial.begin(9600);    // เปิดพอร์ตอนุกรม กำหนดอัตราบอดเป็น 9600 บิตต่อวินาที
}
  
```

4.2.1.2 int Serial.available()

ใช้แจ้งว่าได้รับข้อมูลตัวอักษร (characters) แล้ว และพร้อมสำหรับการอ่านไปใช้งาน

ค่าที่ส่งกลับจากฟังก์ชัน

จำนวนไบต์ที่พร้อมสำหรับการอ่านค่า โดยเก็บข้อมูลในบัฟเฟอร์ตัวรับ

ถ้าไม่มีข้อมูล จะมีค่าเป็น 0

ถ้ามีข้อมูล ฟังก์ชันจะคืนค่าที่มากกว่า 0 โดยบัฟเฟอร์เก็บข้อมูลได้สูงสุด 128 ไบต์

4.2.1.3 int Serial.read()

ใช้อ่านค่าข้อมูลที่ได้รับจากพอร์ตอนุกรม

ค่าที่ส่งกลับจากฟังก์ชัน

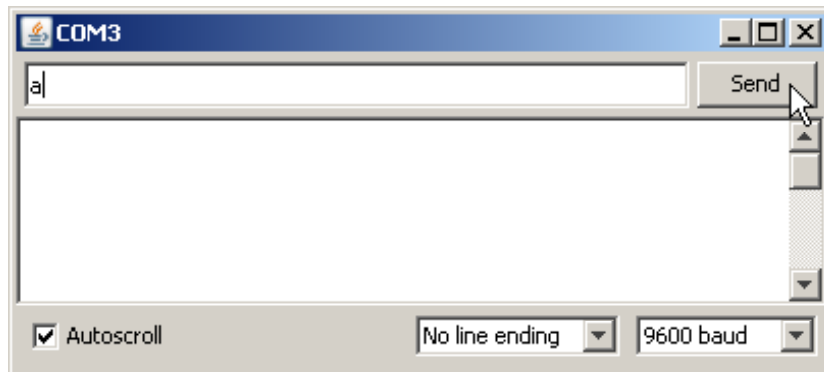
เป็นเลข int ที่เป็นไบต์แรกของข้อมูลที่ได้รับ (หรือเป็น -1 ถ้าไม่มีข้อมูล)

ตัวอย่างที่ 4-5

```
int incomingByte = 0;
void setup()
{
  Serial.begin(9600); // เปิดพอร์ตสื่อสาร กำหนดอัตราบอด 9600 บิตต่อวินาที
}
void loop() // วงส่งข้อมูลเมื่อได้รับข้อมูลเข้ามา
{
  if (Serial.available() > 0) // อ่านข้อมูลอนุกรม
  {
    incomingByte = Serial.read(); // เก็บข้อมูลที่อ่านได้ไว้ในตัวแปร
    Serial.print("I received: "); // พิมพ์ข้อความออกไปยังหน้าต่าง Serial Monitor
    Serial.println(incomingByte, DEC); // พิมพ์ข้อมูลที่รับได้ออกไปยังหน้าต่าง Serial Monitor
  }
}
```

ในตัวอย่างนี้ เลือกใช้อัตราบอด 9600 บิตต่อวินาที ถ้ามีข้อมูลเข้ามาจะเก็บไว้ในตัวแปร incomingByte แล้วนำไปแสดงผลที่หน้าต่าง Serial Monitor โดยต่อท้ายข้อความ I received :.....

เมื่อรันโปรแกรม ให้เปิดหน้าต่าง Serial Monitor โดยคลิกที่ปุ่ม  ซึ่งอยู่ที่มุมขวาของหน้าต่างหลัก

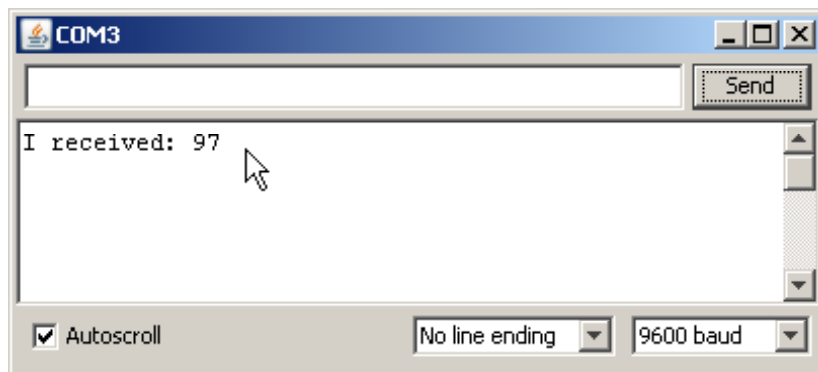


เลือกอัตราบอดของหน้าต่าง Serial Monitor เป็น 9600baud,

เลือก No line Ending

คลิกทำเครื่องหมายที่ช่อง Autoscroll

จากนั้นป้อนอักษร a แล้วคลิกปุ่ม Send



เมื่อบอร์ด **POP-32** ได้รับข้อมูล จะส่งค่าของตัวอักษรในรูปของเลขฐานสิบกลับมา
อักษร a มีรหัสแอสกีเท่ากับ 61 เมื่อแปลงเป็นเลขฐานสิบ จะได้ค่าเป็น 97

4.2.1.4 Serial.flush()

ใช้ล้างบัฟเฟอร์ตัวรับข้อมูลของพอร์ตอนุกรมให้ว่าง

4.2.1.5 Serial.print(data)

ใช้ส่งข้อมูลออกทางพอร์ตอนุกรม

พารามิเตอร์

Data - เป็นข้อมูลเลขจำนวนเต็ม ได้แก่ char, int หรือเลขทศนิยมที่ตัดเศษออกเป็นเลขจำนวนเต็ม

รูปแบบฟังก์ชัน

คำสั่งนี้สามารถเขียนได้หลายรูปแบบ

Serial.print(b)

เป็นการเขียนคำสั่งแบบไม่ระบุรูปแบบ จะพิมพ์ค่าตัวแปร b เป็นเลขฐานสิบ โดยพิมพ์ตัวอักษรรหัส ASCII

ดังตัวอย่าง

```
int b = 79;
Serial.print(b);
พิมพ์ข้อความ 79
```

Serial.print(b, DEC)

เป็นคำสั่งพิมพ์ค่าตัวแปร b เป็นตัวเลขฐานสิบ โดยพิมพ์ตัวอักษรตามรหัส ASCII ดังตัวอย่าง

```
int b = 79;
Serial.print(b);
พิมพ์ข้อความ 79
```

Serial.print(b, HEX)

เป็นคำสั่งพิมพ์ค่าตัวแปร b เป็นตัวเลขฐานสิบหก โดยพิมพ์ตัวอักษรตามรหัส ASCII ดังตัวอย่าง

```
int b = 79;
Serial.print(b, HEX);
พิมพ์ข้อความ 4F
```

Serial.print(b, OCT)

เป็นคำสั่งพิมพ์ค่าตัวแปร b เป็นตัวเลขฐานแปด โดยพิมพ์ตัวอักษรตามรหัส ASCII ดังตัวอย่าง

```
int b = 79;
Serial.print(b, OCT);
พิมพ์ข้อความ 117
```

Serial.print(b, BIN)

เป็นคำสั่งพิมพ์ค่าตัวแปร b เป็นตัวเลขฐานสอง โดยพิมพ์ตัวอักษรตามรหัส ASCII ดังตัวอย่าง

```
int b = 79;
Serial.print(b, BIN);
พิมพ์ข้อความ 1001111
```

Serial.write(b)

เป็นคำสั่งพิมพ์ค่าตัวแปร b ขนาด 1 ไบต์ เดิมทีคำสั่งนี้ จะเป็น Serial.print(b,BYTE) ตั้งแต่ Arduino 1.0 ขึ้นมา ได้ยกเลิกคำสั่ง Serial.print(b,BYTE) และให้ใช้คำสั่ง Serial.write(b) แทน ดังตัวอย่าง

```
int b = 79;
```

```
Serial.write(b);
```

พิมพ์ตัวอักษร 0 ซึ่งมีค่าตามตาราง ASCII เท่ากับ 79 ให้ดูข้อมูลจากตารางรหัสแอสกีเพิ่มเติม

Serial.print(str)

เป็นคำสั่งพิมพ์ค่าข้อความในวงเล็บ หรือข้อความที่เก็บในตัวแปร str ดังตัวอย่าง

```
Serial.print("Hello World!"); // พิมพ์ข้อความ Hello World
```

พารามิเตอร์

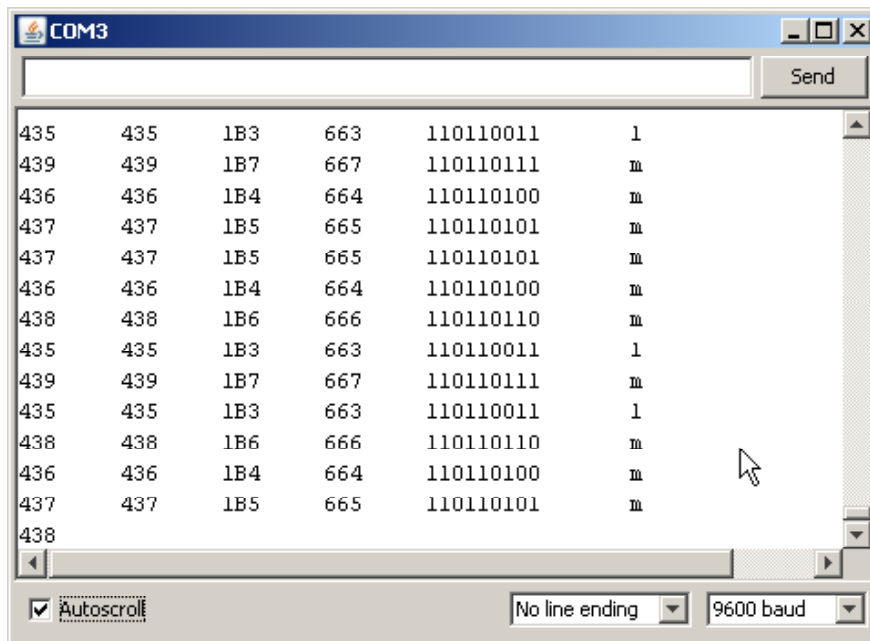
b - ไบต์ข้อมูลที่ต้องการพิมพ์ออกทางพอร์ตอนุกรม

str - ตัวแปรสตริงที่เก็บข้อความสำหรับส่งออกพอร์ตอนุกรม

ตัวอย่างที่ 4-6

```
int analogValue = 0; // กำหนดตัวแปรเก็บค่าสัญญาณแอนะล็อก
void setup()
{
  Serial.begin(9600); // เปิดพอร์ตอนุกรม กำหนดอัตราบอดเป็น 9600 บิตต่อวินาที
}
void loop()
{
  analogValue = analogRead(0); // อ่านค่าสัญญาณแอนะล็อกช่อง 0
  // แสดงผลที่หน้าต่าง Serial Monitor ในหลายรูปแบบ
  Serial.print(analogValue); // แสดงเป็นรหัสแอสกีของเลขฐานสิบ
  Serial.print("\t"); // คั่นด้วยแท็บ (tab)
  Serial.print(analogValue, DEC); // แสดงเป็นรหัสแอสกีของเลขฐานสิบ
  Serial.print("\t"); // คั่นด้วยแท็บ (tab)
  Serial.print(analogValue, HEX); // แสดงเป็นรหัสแอสกีของเลขฐานสิบหก
  Serial.print("\t"); // คั่นด้วยแท็บ (tab)
  Serial.print(analogValue, OCT); // แสดงเป็นรหัสแอสกีของเลขฐานแปด
  Serial.print("\t"); // คั่นด้วยแท็บ (tab)
  Serial.print(analogValue, BIN); // แสดงเป็นรหัสแอสกีของเลขฐานสอง
  Serial.print("\t"); // คั่นด้วยแท็บ (tab)
  Serial.write(analogValue/4);
  // แสดงข้อมูลดิบ ซึ่งต้องผ่านการหารด้วย 4 เนื่องจากฟังก์ชัน analogRead() คืนค่า 0 ถึง 1023
  // แต่ตัวแปรแบบไบต์รับค่าได้เพียง 255
  Serial.print("\t"); // คั่นด้วยแท็บ (tab)
  Serial.println(); // ขึ้นบรรทัดใหม่
  delay(10); // หน่วงเวลา 10 มิลลิวินาที
}
```

ตัวอย่างนี้แสดงการพิมพ์ข้อมูลจากฟังก์ชัน `Serial.Print()` ในรูปแบบต่างๆ แสดงผ่านหน้าต่าง Serial Monitor



ต้องเลือกอัตราบอดของหน้าต่าง Serial Monitor เป็น 9600baud, เลือก No line Ending และคลิกทำเครื่องหมายที่ช่อง Autoscroll ด้วย จึงจะได้ผลการทำงานที่ถูกต้อง

เทคนิคสำหรับการเขียนโปรแกรม

`Serial.print()` จะตัดเศษเลขทศนิยมเหลือเป็นเลขจำนวนเต็ม ทำให้ส่วนทศนิยมหายไปทางแก้ไขทางหนึ่งคือ คูณเลขทศนิยมด้วย 10, 100, 1000 ฯลฯ ขึ้นอยู่กับจำนวนหลักของเลขทศนิยมเพื่อแปลงเลขทศนิยมเป็นจำนวนเต็มก่อนแล้วจึงส่งออกพอร์ตอนุกรม จากนั้นที่ฝั่งภาครับให้ทำการหารค่าที่รับได้เพื่อแปลงกลับเป็นเลขทศนิยม

4.2.1.6 Serial.println(data)

เป็นฟังก์ชันพิมพ์ (หรือส่ง) ข้อมูลออกทางพอร์ตอนุกรมตามด้วยรหัส **carriage return** (รหัส ASCII หมายเลข 13 หรือ `\r`) และ **linefeed** (รหัส ASCII หมายเลข 10 หรือ `\n`) เพื่อให้เกิดการเลื่อนบรรทัดและขึ้นบรรทัดใหม่หลังจากพิมพ์ข้อความ มีรูปแบบเหมือนคำสั่ง `Serial.print()`

รูปแบบฟังก์ชัน

Serial.println(b)

เป็นคำสั่งพิมพ์ข้อมูลแบบไม่ระบุรูปแบบ จะพิมพ์ค่าตัวแปร b เป็นเลขฐานสิบ ตามด้วยรหัสอักขร carriage return และ linefeed ดังตัวอย่างต่อไปนี้

Serial.println(b, DEC)

เป็นคำสั่งพิมพ์ค่าตัวแปร b เป็นตัวเลขฐานสิบ ตามด้วยรหัสอักขร carriage return และ linefeed

Serial.println(b, HEX)

เป็นคำสั่งพิมพ์ค่าตัวแปร b เป็นตัวเลขฐานสิบหก ตามด้วยรหัสอักขร carriage return และ linefeed

Serial.println(b, OCT)

เป็นคำสั่งพิมพ์ค่าตัวแปร b เป็นตัวเลขฐานแปด ตามด้วยรหัสอักขร carriage return และ linefeed

Serial.println(b, BIN)

เป็นคำสั่งพิมพ์ค่าตัวแปร b เป็นตัวเลขฐานสอง ตามด้วยรหัสอักขร carriage return และ linefeed

Serial.println(str)

เป็นคำสั่งพิมพ์ค่าข้อความในวงเล็บหรือข้อความ ที่เก็บในตัวแปร str ตามด้วยรหัสอักขร carriage return และ linefeed

Serial.println()

เป็นคำสั่งพิมพ์รหัส carriage return และ linefeed

พารามิเตอร์

b - ไบต์ข้อมูลที่ต้องการพิมพ์ออกทางพอร์ตอนุกรม

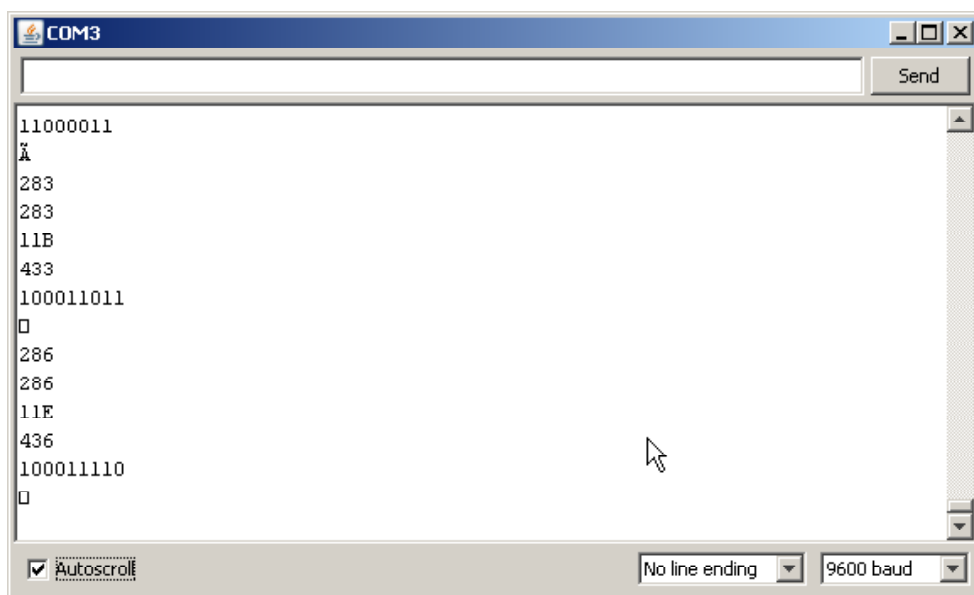
str - ตัวแปรสตริงที่เก็บข้อความสำหรับส่งออกพอร์ตอนุกรม

ตัวอย่างที่ 4-7

```

int analogValue = 0;           // ประกาศตัวแปรสำหรับเก็บค่าแอนะล็อก
void setup()
{
  Serial.begin(9600);           // เปิดพอร์ตอนุกรมและเลือกอัตราบอดเป็น 9600 บิตต่อวินาที
}
void loop()
{
  analogValue = analogRead(0);  // อ่านค่าอินพุตแอนะล็อกช่อง 0
  // แสดงผลในรูปแบบต่างๆ
  Serial.println(analogValue);  // แสดงเป็นรหัสแอสกีของเลขฐานสิบ
  Serial.println(analogValue, DEC); // แสดงเป็นรหัสแอสกีของเลขฐานสิบ
  Serial.println(analogValue, HEX); // แสดงเป็นรหัสแอสกีของเลขฐานสิบหก
  Serial.println(analogValue, OCT); // แสดงเป็นรหัสแอสกีของเลขฐานแปด
  Serial.println(analogValue, BIN); // แสดงเป็นรหัสแอสกีของเลขฐานสอง
  Serial.write(analogValue);     // แสดงค่าในรูปแบบข้อมูลระดับไบต์
  Serial.println();
  Serial.flush();
  delay(500);                   // หน่วงเวลา 500 มิลลิวินาที
}

```



ต้องเลือกอัตราบอดของหน้าต่าง Serial Monitor เป็น 9600baud, เลือก No line Ending และคลิกทำเครื่องหมายที่ช่อง Autoscroll ด้วย จึงจะได้ผลการทำงานที่ถูกต้อง

4.3 ฟังก์ชันอินพุตเอาต์พุตแอนะล็อก

4.5.1 คำอธิบายและการเรียกใช้ฟังก์ชัน

4.5.1.1 `int analogRead(pin)`

อ่านค่าจากขาพอร์ตที่กำหนดให้เป็นอินพุตแอนะล็อกของบอร์ด **POP-32** โดย **POP-32** มีวงจรแปลงสัญญาณแอนะล็อกเป็นดิจิทัล ความละเอียด 10 บิต จึงแปลงค่าแรงดันไฟฟ้าอินพุต 0 ถึง +5.3V ให้เป็นข้อมูลตัวเลขจำนวนเต็มระหว่าง 0 ถึง 1023

พารามิเตอร์

pin - หมายเลขของขาอินพุตแอนะล็อก มีค่า 0 ถึง 5 หรือเป็นตัวแปรที่ใช้แทนค่า 0 ถึง 5

ค่าที่ส่งกลับ

เลขจำนวนเต็มจาก 0 ถึง 1023

หมายเหตุ

สำหรับขาที่เป็นอินพุตแอนะล็อกไม่จำเป็นต้องประกาศแจ้งว่าเป็นอินพุตหรือเอาต์พุต

ตัวอย่างที่ 4-8

```
int ledPin = PBO;           // ต่อแผงวงจร ZX-LED เข้าที่พอร์ต PBO/16
int analogPin = AO;         // ต่อแผงวงจร ZX-POT เข้าที่พอร์ตอินพุต AO/0
int val = 0;                // กำหนดตัวแปรรับค่าแอนะล็อก
int threshold = 512;        // กำหนดค่าอ้างอิง
void setup()
{
  pinMode(ledPin, OUTPUT);  // กำหนดให้พอร์ต PBO/16 เป็นเอาต์พุต
}
void loop()
{
  val = analogRead(analogPin); // อ่านค่าอินพุตแอนะล็อก AO
  if (val >= threshold)
  {
    digitalWrite(ledPin, HIGH); // จับ LED
  }
  else
  {
    digitalWrite(ledPin, LOW);  // ดับ LED
  }
}
```

ตัวอย่างนี้จะสั่งให้พอร์ต PBO/16 เป็นลอจิก “1” เมื่ออ่านค่าจากพอร์ตอินพุต AO แล้วมีค่ามากกว่าหรือเท่ากับค่าอ้างอิงที่กำหนดไว้ (ในตัวอย่าง ค่าอ้างอิงหรือ threshold = 255) แต่ถ้าน้อยกว่า สถานะที่พอร์ต PBO/16 จะกลายเป็นลอจิก “0” ทำให้ LED ดับ

4.5.1.2 analogWrite (pin, value)

ใช้เขียนค่าแอนะล็อกไปยังพอร์ตที่กำหนดไว้ เพื่อสร้างสัญญาณ PWM

พารามิเตอร์

pin - หมายเลขขาพอร์ตของบอร์ด POP-32

value - เป็นค่าตัวเลขไบนารีที่มีค่าระหว่าง 0 ถึง 255

เมื่อค่าเป็น 0 แรงดันของขาพอร์ตที่กำหนดจะเป็น 0V เมื่อมีค่าเป็น 255 แรงดันที่ขาพอร์ตจะเป็น +5V สำหรับค่าระหว่าง 0 ถึง 255 จะทำให้ขาพอร์ตที่กำหนดได้มีค่าแรงดันเปลี่ยนแปลงในย่าน 0 ถึง +5V

ค่าที่ส่งกลับจากฟังก์ชัน

เลขจำนวนเต็มจาก 0 ถึง 255

ผู้ใช้งานสามารถนำสัญญาณที่ได้จากคำสั่งนี้ไปใช้ในการปรับความสว่างของ LED หรือต่อขยายกระแสเพื่อต่อปรับความเร็วของมอเตอร์ได้ หลังจากเรียกใช้คำสั่งนี้แล้วที่ขาพอร์ตที่กำหนดจะมีสัญญาณ PWM ส่งออกมาอย่างต่อเนื่อง จนกว่าจะมีการเรียกคำสั่ง analogWrite (หรือเรียกคำสั่ง digitalWrite หรือ digitalWrite ที่ขาเดียวกัน)

ตัวอย่างที่ 4-9

```
int ledPin = PB0;           // ต่อมินิบอร์ด ZX-LED เข้าที่พอร์ต PB0/16
int analogPin = A0;         // ต่อมินิบอร์ด ZX-POT เข้าที่พอร์ตอินพุต A0
int val = 0;                // กำหนดตัวแปรรับค่าแอนะล็อก
int threshold = 512;        // กำหนดค่าอ้างอิง
void setup()
{
  pinMode(ledPin, OUTPUT);  // กำหนดให้พอร์ต PB0/16 เป็นเอาต์พุต
}
void loop()
{
  val = analogRead(analogPin); // เก็บข้อมูลที่อ่านได้จากอินพุตแอนะล็อกลงในตัวแปร
  analogWrite(ledPin, val / 4); // ปรับค่าที่ได้จากการแปลงสัญญาณ 0 ถึง 1023
                                // เป็นค่า 0 ถึง 255 แล้วส่งไปยังพอร์ตเอาต์พุตแอนะล็อก
}
```

ตัวอย่างนี้เป็นการควบคุมความสว่างของ LED ที่ต่อกับพอร์ต PB0/16 ให้เป็นไปตามค่าที่อ่านได้จากมินิบอร์ด ZX-POT (มินิบอร์ดวงจรตัวต้านทานปรับค่าได้) ที่ต่อกับพอร์ต A0

4.4 ฟังก์ชันพอร์ตอินพุตเอาต์พุตขั้นสูง (Enhanced I/O)

4.4.1 `shiftOut(dataPin, clockPin, bitOrder, value)`

เป็นฟังก์ชันสำหรับเลื่อนข้อมูลขนาด 1 ไบต์ (8 บิต) ออกจากขาพอร์ตที่กำหนดไว้ สามารถกำหนดได้ว่า จะให้เริ่มเลื่อนบิตข้อมูลจากบิตที่มีความสำคัญสูงสุด (MSB หรือบิตซ้ายสุด) หรือบิตที่มีความสำคัญต่ำสุด (LSB - บิตขวาสุด) โดยบิตของข้อมูลจะออกที่ขา `dataPin` และใช้อีกหนึ่งขาชื่อ `clockPin` เป็นตัวกำหนดจังหวะการเลื่อนข้อมูล

การส่งข้อมูลวิธีที่วนนี้เรียกว่า การส่งข้อมูลอนุกรมแบบโปรโตคอลซิงโครนัส ซึ่งเป็นวิธีทั่วไปที่ไมโครคอนโทรลเลอร์ใช้ติดต่อกับตัวตรวจนับหรือไมโครคอนโทรลเลอร์ตัวอื่น อุปกรณ์ทั้งสองตัวต้องทำงานประสานกัน (ซิงโครไนซ์กัน) และติดต่อกันที่ความเร็วสูงสุด เนื่องจากอุปกรณ์ทั้งสองตัวจะใช้สัญญาณนาฬิกาเดียวกัน จะเรียกชื่ออีกอย่างว่าเป็นการสื่อสารแบบ SPI (synchronous peripheral interface)

พารามิเตอร์

`dataPin` - พอร์ตที่กำหนดให้เป็นเอาต์พุตเพื่อส่งข้อมูลออก (ตัวแปรชนิด `int`)

`clockPin` - พอร์ตที่กำหนดให้ทำหน้าที่ส่งสัญญาณนาฬิกาเพื่อกำหนดจังหวะการเลื่อนข้อมูลของ `dataPin`

`bitOrder` - กำหนดลำดับการเลื่อนบิตข้อมูล เลือกได้ว่าเป็น `MSBFIRST` หรือ `LSBFIRST`

`value` - ไบต์ข้อมูลที่ต้องการส่งแบบอนุกรม

พอร์ตที่เป็น `dataPin` และ `clockPin` จะต้องกำหนดให้เป็นเอาต์พุตก่อน ด้วยการเรียกใช้ฟังก์ชัน `pinMode`

4.4.1.1 การเขียนโปรแกรมที่ผิดพลาดที่พบบ่อยของฟังก์ชัน `shiftOut`

เนื่องจากฟังก์ชันนี้สามารถส่งข้อมูลได้ครั้งละ 1 ไบต์ (8 บิต) เท่านั้น ถ้าเป็นตัวแปร `int` หนึ่งตัวเก็บข้อมูลขนาด 2 ไบต์ (16 บิต) ทำให้ไม่สามารถส่งค่าได้โดยใช้คำสั่งนี้เพียงครั้งเดียว ถ้าส่งเพียงครั้งเดียว ค่าที่ได้จะผิดพลาด ดังตัวอย่างโปรแกรมที่ผิดพลาดต่อไปนี้

ตัวอย่างที่ 4-10

```
int data;
int clock;
int cs;
...
digitalWrite(cs, LOW);
data = 500;
shiftOut(data, clock, MSBFIRST, data)
digitalWrite(cs, HIGH);
```

ข้อมูลที่ต้องการเลื่อนเท่ากับ 500 ดังนั้นจึงมีขนาด 2 ไบต์ (เพราะ 1 ไบต์เก็บค่าได้สูงสุด 255 ค่า) แต่เนื่องจากในโปรแกรมเลื่อนเพียง 1 ไบต์ ค่าที่ได้จึงเท่ากับ 244 เนื่องจากข้อมูล 8 บิตบนของตัวแปร `data` มีค่าเท่ากับ 244

ในตัวอย่างที่ 4-11 เป็นการแก้ไขโปรแกรมเพื่อให้สามารถเลื่อนข้อมูลได้อย่างถูกต้อง

ตัวอย่างที่ 4-11

ในกรณีที่ต้องการเลื่อนบิต MSB ก่อน เขียนโปรแกรมได้ดังนี้

```
data = 500;
shiftOut(data, clock, MSBFIRST, (data >> 8)); // เลื่อนข้อมูลไบต์สูง
shiftOut(data, clock, MSBFIRST, data);          // เลื่อนข้อมูลไบต์ต่ำ
```

ในกรณีที่ต้องการเลื่อนบิต LSB ก่อน เขียนโปรแกรมได้ดังนี้

```
data = 500;
shiftOut(data, clock, MSBFIRST, data);          // เลื่อนข้อมูลไบต์ต่ำ
shiftOut(data, clock, MSBFIRST, (data >> 8)); // เลื่อนข้อมูลไบต์สูง
```

4.4.2 unsigned long pulseIn(pin, value)

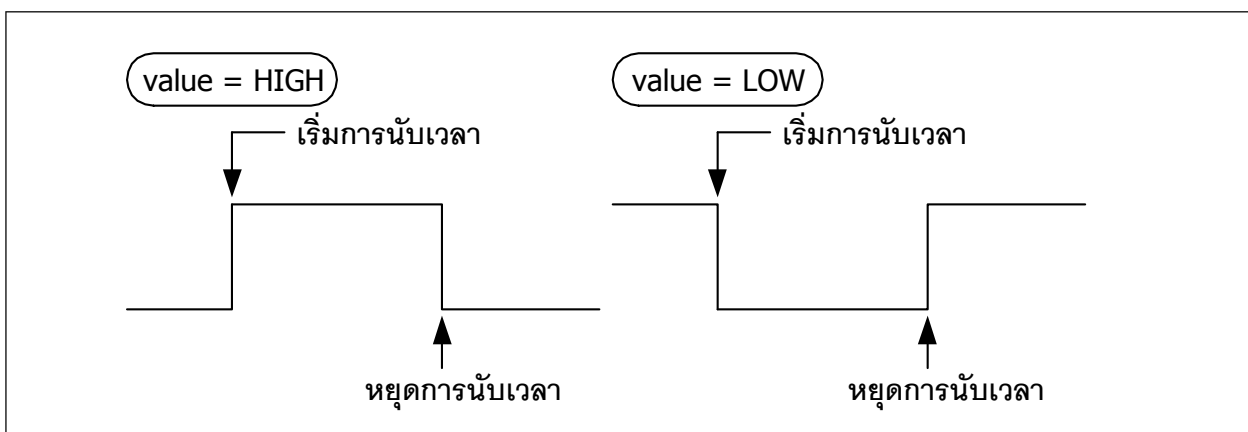
ใช้อ่านคาบเวลาของพัลส์ (ไม่ว่าเป็น HIGH หรือ LOW) ที่เกิดขึ้นที่ pin (ขาพอร์ตอินพุตดิจิทัล) ที่กำหนด เช่น ถ้ากำหนด value เป็น HIGH คำสั่ง pulseIN() จะรอให้ขาที่กำหนดไว้มีสถานะเป็น HIGH เพื่อเริ่มจับเวลา และจับเวลาต่อไปจนกว่าสถานะของขา นั้นจะเป็น LOW จึงหยุดจับเวลา ค่าที่ฟังก์ชันคืนกลับมาเป็นคาบเวลาของพัลส์ในหน่วยไมโครวินาที แสดงการทำงานในรูปที่ 4-4

ค่าฐานเวลาของฟังก์ชันประมาณค่าได้จากการทดลอง และอาจผิดพลาดได้ในกรณีที่พัลส์มีความกว้างมาก ฟังก์ชันนี้ทำงานได้ดีกับสัญญาณพัลส์ที่มีคาบเวลา 10 ไมโครวินาทีถึง 3 นาที

พารามิเตอร์

pin - ขาพอร์ตที่ต้องการอ่านค่าคาบเวลาพัลส์ (ตัวแปรชนิด int)

value - ระดับสัญญาณที่ต้องการตรวจจับ กำหนดเป็น **HIGH** หรือ **LOW** (ตัวแปรชนิด int) โดย HIGH คือ ลอจิกสูง และ LOW คือ ลอจิกต่ำ



รูปที่ 4-4 การทำงานของฟังก์ชัน pulseIn

ค่าที่ส่งกลับจากฟังก์ชัน

คาบเวลาของพัลส์ในหน่วยไมโครวินาที

ตัวอย่างที่ 4-12

```

int pin = 18;
unsigned long duration;
void setup()
{
    pinMode(pin, INPUT);
}
void loop()
{
    duration = pulseIn(pin, HIGH);
}

```

ฟังก์ชันนี้ไม่มีกลไกการหมดเวลา ถ้าไม่มีพัลส์เกิดขึ้น จะวนรอไปตลอดเวลา ทำให้โปรแกรมค้างได้

4.5 ฟังก์ชันเกี่ยวกับเวลา (Time)

4.5.1 unsigned long millis()

คืนค่าเป็นค่าเวลาในหน่วยมิลลิวินาที นับตั้งแต่ไมโครคอนโทรลเลอร์เริ่มรันโปรแกรมปัจจุบัน

ค่าที่ส่งกลับจากฟังก์ชัน

ค่าเวลาในหน่วยเป็นมิลลิวินาทีตั้งแต่เริ่มรันโปรแกรมปัจจุบัน คืนค่าเป็น unsigned long ค่าตัวเลขจะเกิดการโอเวอร์โฟลว (ค่าเกินแล้วกลับเป็นศูนย์) เมื่อเวลาผ่านไปประมาณ 9 ชั่วโมง

ตัวอย่างที่ 4-13

```

long time;
void setup()
{
    Serial.begin(9600);
}
void loop()
{
    Serial.print("Time: ");
    time = millis();           // อ่านค่าเวลา
    Serial.println(time);      // แสดงค่าเวลา
    Serial.flush();
    delay(1000);
}

```

4.5.2 delay(ms)

เป็นฟังก์ชันชะลอการทำงาน หรือหน่วงเวลาของโปรแกรมตามเวลาที่กำหนดในหน่วยมิลลิวินาที

พารามิเตอร์

ms - ระยะเวลาที่ต้องการหน่วงเวลา หน่วยเป็นมิลลิวินาที (1000 ms เท่ากับ 1 วินาที)

ตัวอย่างที่ 4-14

```
int ledPin = PB0;                // ต่อมินิบอร์ด ZX-LED กับพอร์ต PB0/16
void setup()
{
  pinMode(ledPin, OUTPUT);       // กำหนดเป็นพอร์ตเอาต์พุต
}
void loop()
{
  digitalWrite(ledPin, HIGH);    // ขั้ว LED
  delay(1000);                  // หน่วงเวลา 1 วินาที
  digitalWrite(ledPin, LOW);     // ดับ LED
  delay(1000);                  // หน่วงเวลา 1 วินาที
}
```

4.5.3 delayMicroseconds(us)

เป็นฟังก์ชันชะลอการทำงานหรือหน่วงเวลาของโปรแกรมตามเวลาที่กำหนดในหน่วยไมโครวินาที เพื่อให้หน่วงเวลาได้อย่างแม่นยำ ฟังก์ชันนี้จะหยุดการทำงานของอินเทอร์พรีต ทำให้การทำงานบางอย่าง เช่น การรับข้อมูลจากพอร์ตอนุกรม หรือการเพิ่มค่าที่จะส่งกลับคืนโดยฟังก์ชัน `millis()` จะไม่เกิดขึ้น ดังนั้นควรจะใช้ฟังก์ชันนี้สำหรับการหน่วงเวลาสั้นๆ เท่านั้น

พารามิเตอร์

us - ค่าหน่วงเวลาในหน่วยไมโครวินาที (1000 ไมโครวินาที = 1 มิลลิวินาที และหนึ่งล้านไมโครวินาที = 1 วินาที)

ตัวอย่างที่ 4-15

```
int outPin = PB0;                // ต่อมินิบอร์ด ZX-LED กับพอร์ต PB0/16
void setup()
{
  pinMode(outPin, OUTPUT);       // กำหนดเป็นพอร์ตเอาต์พุต
}
void loop()
{
  digitalWrite(outPin, HIGH);    // ขั้ว LED
  delayMicroseconds(50);         // หน่วงเวลา 50 ไมโครวินาที
  digitalWrite(outPin, LOW);     // ดับ LED
  delayMicroseconds(50);         // หน่วงเวลา 50 ไมโครวินาที
}
```

คำเตือน ฟังก์ชันนี้ทำงานอย่างแม่นยำในช่วงตั้งแต่ 3 ไมโครวินาทีขึ้นไป

4.6 ฟังก์ชันทางคณิตศาสตร์

4.6.1 $\min(x, y)$

หาค่าตัวเลขที่น้อยที่สุดของตัวเลขสองตัว

พารามิเตอร์

x - ตัวเลขตัวแรก เป็นข้อมูลประเภทใดก็ได้

y - ตัวเลขตัวที่สอง เป็นข้อมูลประเภทใดก็ได้

ค่าที่ส่งกลับจากฟังก์ชัน

ค่าที่น้อยที่สุดของตัวเลขสองตัวที่ให้

ตัวอย่างที่ 4-16

```
sensVal = min(sensVal, 100);
```

ตัวอย่างนี้จะได้ค่าของ *sensVal* ที่ไม่เกิน 100 กลับจากฟังก์ชัน

4.6.2 $\max(x, y)$

หาค่าตัวเลขที่มากที่สุดของตัวเลขสองตัว

พารามิเตอร์

x - ตัวเลขตัวแรก เป็นข้อมูลประเภทใดก็ได้

y - ตัวเลขตัวที่สอง เป็นข้อมูลประเภทใดก็ได้

ค่าที่ส่งกลับจากฟังก์ชัน

ค่าที่มากที่สุดของตัวเลขสองตัวที่ให้

ตัวอย่างที่ 4-17

```
sensVal = max(sensVal, 20);
```

จากตัวอย่างนี้ ค่าของ *sensVal* จะมีค่าน้อย 20

4.6.3 $\text{abs}(x)$

หาค่าสัมบูรณ์ (absolute) ของตัวเลข

พารามิเตอร์

x - ตัวเลข

ค่าที่ส่งกลับจากฟังก์ชัน

x เมื่อ x มีค่ามากกว่าหรือเท่ากับศูนย์ (x มีค่าเป็นบวกหรือศูนย์)

-x เมื่อ x มีค่าน้อยกว่าศูนย์ (x มีค่าติดลบ)

4.6.4 constrain(x, a, b)

ปิดค่าตัวเลขที่น้อยกว่าหรือมากกว่าให้อยู่ในช่วงที่กำหนด

พารามิเตอร์

x - ตัวเลขที่ต้องการปิดค่าให้อยู่ในช่วงที่กำหนด สามารถเป็นข้อมูลชนิดใดก็ได้

a - ค่าต่ำสุดของช่วงที่กำหนด

b - ค่าสูงสุดของช่วงที่กำหนด

ค่าที่ส่งกลับจากฟังก์ชัน

x เมื่อ x มีค่าอยู่ระหว่าง a และ b

a เมื่อ x มีค่าน้อยกว่า a

b เมื่อ x มีค่ามากกว่า b

ตัวอย่างที่ 4-18

```
sensVal = constrain(sensVal, 10, 150);
```

จากตัวอย่างนี้ ค่าของ *sensVal* จะอยู่ในช่วง 10 ถึง 150

4.7 ฟังก์ชันเกี่ยวกับตัวเลขสุ่ม

4.7.1 randomSeed (seed)

ใช้กำหนดตัวแปรสำหรับสร้างตัวเลขสุ่ม โดยสามารถใช้ตัวแปรได้หลากหลายรูปแบบ โดยทั่วไปจะใช้ค่าเวลาปัจจุบัน (จากฟังก์ชัน `millis()`) แต่สามารถใช้ค่าอย่างอื่นได้ เช่น ค่าที่ได้เมื่อผู้ใช้กดสวิตช์ หรือค่าสัญญาณรบกวนที่อ่านได้จากขาอินพุตแอนะล็อก

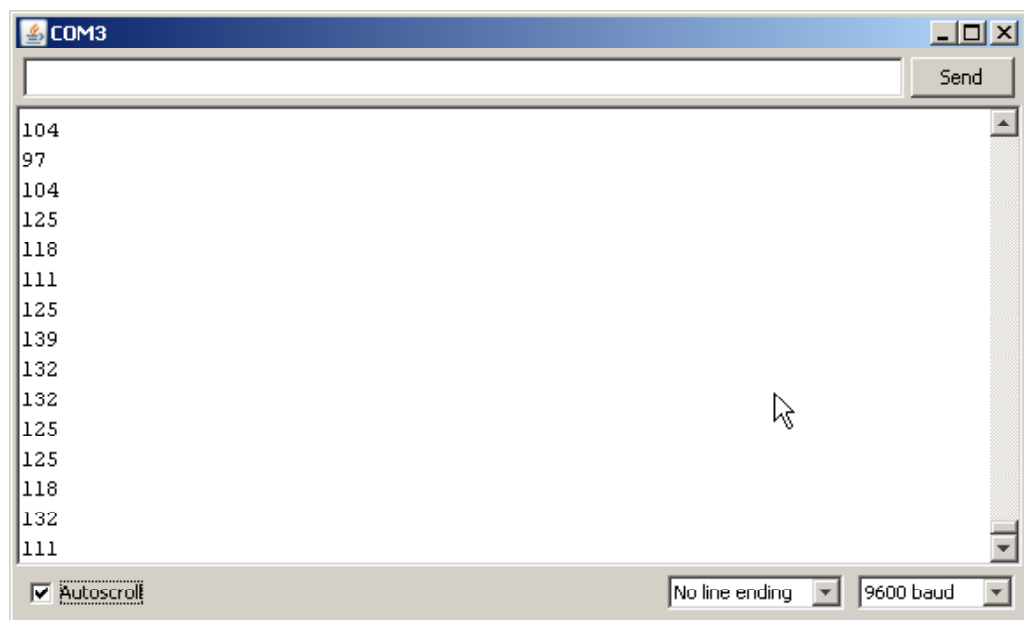
พารามิเตอร์

seed เป็นค่าตัวเลขแบบ long int

ตัวอย่างที่ 4-19

```
long randNumber;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  randomSeed(analogRead(0));
  randNumber = random(300);
  Serial.println(randNumber);
}
```

ในตัวอย่างนี้ กำหนดให้เกิดการสุ่มตัวเลขเมื่ออ่านค่าจากอินพุตแอนะล็อกช่อง 0 ย่านของตัวเลขสุ่มคือ 0 ถึง 300 เมื่อสุ่มตัวเลขแล้ว ให้แสดงค่าที่หน้าต่าง Serial Monitor



4.7.2 long random(max) ,

long random (min,max)

ใช้สร้างตัวเลขสุ่มเทียม (pseudo-random numbers) เพื่อนำไปใช้ใน โปรแกรม ก่อนใช้ฟังก์ชันนี้ จะต้องเรียกใช้ฟังก์ชัน randomSeed() ก่อน

พารามิเตอร์

min กำหนดค่าตัวเลขสุ่มไม่น้อยกว่าค่านี้ (เป็นออฟชั่นเพิ่มเติม)

max กำหนดค่าสูงสุดของตัวเลขสุ่ม

ค่าที่ส่งกลับจากฟังก์ชัน

คืนค่าเป็นตัวเลขสุ่มในช่วงที่กำหนด (เป็นตัวแปร long int)

ตัวอย่างที่ 4-20

```
long randNumber;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  randomSeed(analogRead(0));           // สุ่มค่าจากการอ่านค่าอินพุตแอนะล็อก
  randNumber = random(50,300);         // สุ่มค่าในช่วง 50 ถึง 300
  Serial.println(randNumber);
  Serial.flush();
  delay(50);
}
```

เมื่อรันโปรแกรมนี้นี้ ค่าที่ได้จากจะอยู่ในช่วง 50 ถึง 300 แม้ว่าจะปรับค่าแรงดันอินพุตที่พอร์ต AO เป็น 0 หรือ +5.9V (ซึ่งปกติมีค่า 0 ถึง 1023) ทั้งนี้เป็นผลมาจากฟังก์ชัน random(min,max)

4.8 ฟังก์ชันเกี่ยวกับอินเทอร์รัปต์จากภายนอก

ใช้ระบุว่าเมื่อเกิดการอินเทอร์รัปต์จากภายนอกจะให้โปรแกรมกระโดดไปยังฟังก์ชันใด

4.8.1 การอินเทอร์รัปต์ (interrupt)

การจัดจังหวะการทำงานของซีพียู นับเป็นคุณสมบัติที่ต้องมีในไมโครคอนโทรลเลอร์สมัยใหม่ และเป็นคุณสมบัติที่มีบทบาทสำคัญอย่างมาก ในขณะที่ระบบกำลังทำการลำเลียงขวดไปตามสายพานเพื่อทำการบรรจุน้ำยา แล้วเกิดเหตุการณ์ขวดหมด จึงต้องจัดจังหวะกระบวนการบรรจุน้ำยาชั่วคราว จนกว่าจะจัดหาขวดเข้ามาในระบบเป็นที่เรียบร้อย กระบวนการทำงานก็จะดำเนินต่อไป

จากตัวอย่างดังกล่าว ถ้าเปรียบเทียบกับโปรแกรมควบคุมของไมโครคอนโทรลเลอร์ ระบบลำเลียงขวดเพื่อบรรจุน้ำยาเปรียบได้กับโปรแกรมหลัก เหตุการณ์ขวดหมดคือ เงื่อนไขของการเกิดอินเทอร์รัปต์ที่เป็นจริง ทำให้เกิดการอินเทอร์รัปต์ขึ้น การจัดหาขวดมาเพิ่มเติมเปรียบได้กับซีพียูกระโดดออกจากโปรแกรมหลักไปทำงานที่โปรแกรมย่อยบริการอินเทอร์รัปต์เพื่อจัดหาขวด นั่นคือเสร็จสิ้นการบริการอินเทอร์รัปต์ ซีพียูก็จะกระโดดกลับมาทำงานที่โปรแกรมหลักต่อไป ระบบสายพานลำเลียงก็จะทำงานต่อไปตามปกติ

4.8.2 อินพุตรับสัญญาณอินเทอร์รัปต์

ทุกขาพอร์ตของบอร์ด **POP-32** รองรับการทำงานเป็นอินพุตสำหรับรับสัญญาณอินเทอร์รัปต์ ในแบบเปลี่ยนแปลงสถานะลอจิก (interrupt on-changed)

4.8.3 `attachInterrupt(interrupt, function, mode)`

ใช้ระบุว่า เมื่อขาอินพุตที่รับสัญญาณอินเทอร์รัปต์จากภายนอกมีการเปลี่ยนแปลงจะกำหนดให้ซีพียูกระโดดไปยังฟังก์ชันใด เมื่อเกิดการอินเทอร์รัปต์ขึ้น จะไม่สามารถเรียกใช้ฟังก์ชัน `millis()` และ `delay()` ได้ เมื่อเกิดการตอบสนองอินเทอร์รัปต์แล้ว ดังนั้นหากมีข้อมูลเข้ามาทางพอร์ต UART1 ข้อมูลจึงอาจสูญหายได้

พารามิเตอร์

Interrupt - หมายเลขของช่องอินพุตอินเทอร์รัปต์ (เป็น int)

function - ฟังก์ชันที่จะกระโดดไปทำงานเมื่อเกิดอินเทอร์รัปต์ ไม่รับค่าพารามิเตอร์และไม่มีการคืนค่า

mode - เลือกประเภทสัญญาณที่ใช้กระตุ้นให้เกิดการอินเทอร์รัปต์

LOW เกิดอินเทอร์รัปต์เมื่อขาสัญญาณเป็นลอจิก “0”

CHANGE เกิดอินเทอร์รัปต์เมื่อมีการเปลี่ยนแปลงลอจิก

RISING เกิดอินเทอร์รัปต์เมื่อมีการเปลี่ยนลอจิก “0” เป็น “1”

FALLING เกิดอินเทอร์รัปต์เมื่อมีการเปลี่ยนลอจิก “1” เป็น “0”

ตัวอย่างที่ 4-21

```

int pin = PB0;
volatile int state = LOW;
void setup()
{
    pinMode(pin, OUTPUT);
    attachInterrupt(0, blink, CHANGE);
}
void loop()
{
    digitalWrite(pin, state);
}
void blink()
{
    state = !state;
}

```

ตัวอย่างนี้เลือกอินพุตอินเตอร์รัปต์ช่อง 0 กำหนดให้ไปทำงานที่ฟังก์ชัน blink เมื่อเกิดอินเตอร์รัปต์ขึ้น

4.8.4 detachInterrupt (interrupt)

ยกเลิกการอินเตอร์รัปต์

พารามิเตอร์

Interrupt - หมายเลขของช่องอินพุตอินเตอร์รัปต์ที่ต้องการยกเลิก (ค่าเป็น 0 ถึง 3)



บทที่ 5

ไลบรารีสำหรับพัฒนาโปรแกรมของ บอร์ดไมโครคอนโทรลเลอร์ POP-32

การพัฒนาโปรแกรมภาษา C/C++ ด้วย Arduino สำหรับบอร์ดไมโครคอนโทรลเลอร์ POP-32 ดำเนินการภายใต้การสนับสนุนของไฟล์ไลบรารี POP32.h ทั้งนี้เพื่อช่วยลดขั้นตอนและความซับซ้อนในการเขียนโปรแกรมควบคุมส่วนต่างๆ ของฮาร์ดแวร์ลง เนื่องจากต้องการให้ความสำคัญไปอยู่ที่การเขียนโปรแกรมสำหรับรองรับการเรียนรู้และกิจกรรมการแข่งขัน

5.1 คลาส OLED_I2C_SSD1309

สำหรับควบคุมการทำงานของจอแสดงผล OLED บนบอร์ด POP-32

คลาส OLED_I2C_SSD1309 เป็นคลาสที่รวบรวมเมธอดต่างๆ สำหรับควบคุมการทำงานของจอภาพกราฟิกแบบ OLED ความละเอียด 128 x 64 พิกเซล เพื่อความสะดวกในการเขียนโปรแกรมในคลาส OLED_I2C_SSD1309 จึงสร้างออบเจกต์ที่สืบทอดมาจากคลาส OLED_I2C_SSD1309 โดยตั้งชื่อว่า oled ไว้ให้ใช้งานเรียบร้อยแล้ว โดยออบเจกต์ที่ถูกสร้างจากคลาสนี้มีเมธอด (Public Methods) ให้เรียกใช้งานดังนี้

5.1.1 void show(void)

ทำหน้าที่อัปเดตหรือปรับปรุงการแสดงผลกราฟิกที่จอภาพ ไม่มีการกินค่าใดๆ จากเมธอดนี้

5.1.2 void text(uint8_t row, uint8_t col, char *p)

ทำหน้าที่แสดงข้อความ

พารามิเตอร์

row คือ ตำแหน่งแถวหรือโรว์ กำหนดได้ 0 ถึง 7 (ที่ขนาดตัวอักษรปกติ)

col คือ ตำแหน่งหลักหรือคอลัมน์ กำหนดได้ 0 ถึง 20 (ที่ขนาดตัวอักษรปกติ)

*p คือ ตัวชี้ข้อความที่ต้องการนำมาแสดง รวมถึงรหัสที่ใช้กำหนดรูปแบบพิเศษเพื่อร่วมแสดงผลข้อมูล และตัวเลขในรูปแบบอื่นๆ ประกอบด้วย

%c หรือ %C – รับค่าแสดงผลอักษร 1 ตัวอักษร

%d หรือ %D – รับค่าแสดงผลตัวเลขจำนวนเต็มในช่วง -2,147,483,648 ถึง 2,147,483,647

%l หรือ %L – รับค่าแสดงผลตัวเลขจำนวนเต็มในช่วง -2,147,483,648 ถึง 2,147,483,647

%f หรือ %F – รับค่าแสดงผลตัวเลขจำนวนจริงแสดงทศนิยม 3 หลัก

ตัวอย่างที่ 5-1

```
#include <POP32.h>

void setup()
{
    oled.text(0,0,"Hello World");    // แสดงข้อความ Hello World
                                      // ที่ตำแหน่งซ้ายสุดของบรรทัดแรกสุด (บรรทัดที่ 1)
    oled.show();                     // ปรับปรุงการแสดงผล
}

void loop()
{
}
```

ตัวอย่างที่ 5-2

```
#include <POP32.h>

void setup()
{
    int x=20;
    oled.text(1,2,"Value = %d ",x);  // แสดงตัวอักษรและตัวเลขบรรทัดเดียวกัน
                                      // เริ่มต้นที่คอลัมน์ 2 ของบรรทัด 1 (บรรทัดที่ 2)
    oled.show();                     // ปรับปรุงการแสดงผล
}

void loop()
{
}
```

5.1.3 void mode(uint8_t modeset)

ทำหน้าที่กำหนดทิศทางการแสดงผลหรือสั่งให้หมุนการแสดงผล

พารามิเตอร์

modeset คือ ค่าทิศทางการหมุนมีค่า 0 ถึง 3 ใช้แทน 0 , 90 , 180 และ 270 องศา

ตัวอย่างที่ 5-3

```
#include <POP32.h>

void setup()
{
    oled.mode(1);                     // หมุนหน้าจอจากปกติ 90 องศา
    oled.text(2,0,"Hello");          // แสดงข้อความ Hello ที่ตำแหน่งซ้ายสุดของบรรทัด 2
    oled.show();                     // ปรับปรุงการแสดงผล
}

void loop()
{
}
```

5.1.4 void dim(boolean dim)

ปรับความสว่างของจอแสดงผล

พารามิเตอร์

dim ทำหน้าที่กำหนดคุณสมบัติการปรับความสว่างหน้าจอแสดงผล

- true สำหรับลดความสว่างหน้าจอแสดงผล
- false สำหรับปรับความสว่างหน้าจอแสดงผลแบบปกติ (เป็นค่าตั้งต้น)

ตัวอย่างที่ 5-4

```
#include <POP32.h>
void setup()
{
    oled.mode(1);           // หมุนหน้าจอจากปกติ 90 องศา
    oled.dim(true);         // ปรับความสว่างปกติ
    oled.text(2,0,"HelloWorld"); // แสดงข้อความ HelloWorld ที่ตำแหน่งซ้ายสุดของบรรทัด 2
    oled.show();            // ปรับปรุงการแสดงผล
    oled.dim(false);        // ลดความสว่างลง
    oled.show();            // ปรับปรุงการแสดงผล
    delay (2000);           // หน่วงเวลา 2 วินาที
}
void loop()
{
}
```

5.1.5 void textColor(uint8_t newColor)

ทำหน้าที่กำหนดสีของตัวอักษร ค่าตั้งต้นกำหนดเป็นสีขาว

พารามิเตอร์

newColor คือ สีที่ต้องการกำหนด

- กำหนดเป็น BLACK (หรือเทียบเท่ากับ 0) สำหรับสีดำ
- กำหนดเป็น WHITE (หรือเทียบเท่ากับ 1) สำหรับสีขาว

ตัวอย่างที่ 5-5

```
#include <POP32.h>
void setup()
{
    oled.setTextColor(WHITE); // กำหนดให้ข้อความเป็นสีขาว
    oled.text(2,0,"Hello World"); // แสดงข้อความ Hello World
    oled.show();                // ปรับปรุงการแสดงผล
}
void loop()
{
}
```

5.1.6 void textColor(uint8_t newColor, uint8_t newBColor)

กำหนดสีของตัวอักษรและพื้นหลัง ค่าตั้งต้นกำหนดเป็นตัวอักษรสีขาวบนพื้นดำ
พารามิเตอร์

newColor คือสีของตัวอักษรที่ต้องการกำหนด

- กำหนดเป็น BLACK (หรือเทียบเท่ากับ 0) สำหรับสีดำ
- กำหนดเป็น WHITE (หรือเทียบเท่ากับ 1) สำหรับสีขาว

newBColor คือค่าสีของพื้นหลังที่ต้องการกำหนด

- กำหนดเป็น BLACK (หรือเทียบเท่ากับ 0) สำหรับสีดำ
- กำหนดเป็น WHITE (หรือเทียบเท่ากับ 1) สำหรับสีขาว

ตัวอย่างที่ 5-6

```
#include <POP32.h>
void setup()
{
    oled.textColor(BLACK,WHITE);    // กำหนดข้อความเป็นสีดำ พื้นหลังเป็นสีขาว
    oled.text(2,0,"Hello World");    // แสดงข้อความ Hello World
    oled.show();                    // ปรับปรุงการแสดงผล
}
void loop()
{
}
```

5.1.7 void textBackgroundColor(uint8_t newColor)

กำหนดค่าสีพื้นหลังของตัวอักษร ค่าตั้งต้นเป็นสีดำ

พารามิเตอร์

newColor คือสีที่ต้องการ

- กำหนดเป็น BLACK (หรือเทียบเท่ากับ 0) สำหรับสีดำ
- กำหนดเป็น WHITE (หรือเทียบเท่ากับ 1) สำหรับสีขาว

ตัวอย่างที่ 5-7

```
#include <POP32.h>
void setup()
{
    oled.textBackgroundColor(WHITE); // กำหนดให้พื้นหลังของตัวอักษรเป็นสีขาว
    oled.textColor(BLACK);           // กำหนดให้ข้อความเป็นสีดำ
    oled.text(2,0,"Hello World");    // แสดงข้อความ Hello World
    oled.show();                    // ปรับปรุงการแสดงผล
}
void loop()
{
}
```


5.1.8 void textSize(uint8_t newSize)

กำหนดขนาดตัวอักษร โดยระบุเป็นจำนวนเท่าของขนาดปกติ ค่าตั้งต้นเมื่อเริ่มทำงานทุกครั้ง คือ ขนาดปกติ

พารามิเตอร์

newSize คือค่าขนาดจำนวนเท่าของขนาดปกติ

ตัวอย่างที่ 5-8

```
#include <POP32.h>
void setup()
{
    oled.textSize(1);           // กำหนดขนาดตัวอักษรเป็น 1 เท่า
    oled.text(0,0,"Size1");     // แสดงข้อความ Size1
    oled.textSize(2);           // กำหนดขนาดตัวอักษรเป็น 2 เท่า
    oled.text(1,0,"Size2");     // แสดงข้อความ Size2
    oled.textSize(3);           // กำหนดขนาดตัวอักษรเป็น 3 เท่า
    oled.text(2,0,"Size3");     // แสดงข้อความ Size3
    oled.show();                // ปรับปรุงการแสดงผล
}
void loop()
{
}
```

5.1.9 void fillScreen(uint8_t color)

เป็นการเคลียร์หน้าจอแสดงผล แล้วเปลี่ยนสีพื้นหลังของจอแสดงผลด้วยสีที่ระบุ

พารามิเตอร์

color คือสีที่ต้องการ

- กำหนดเป็น BLACK (หรือเทียบเท่ากับ 0) สำหรับสีดำ
- กำหนดเป็น WHITE (หรือเทียบเท่ากับ 1) สำหรับสีขาว

ตัวอย่างที่ 5-9

```
oled.fillScreen(WHITE);        // กำหนดพื้นหลังเป็นสีขาว
oled.fillScreen(BLACK);        // กำหนดพื้นหลังเป็นสีดำ
```

5.1.10 void clear() ;

เคลียร์หน้าจอแสดงผลด้วยสีดำ

ตัวอย่างที่ 5-10

```
#include <POP32.h>
void setup()
{
  void loop()
  {
    oled.text(0,0,"Hello POP-32");
    oled.show();           // ปรับปรุงการแสดงผล
    delay(2000);           // หน่วงเวลา 2 วินาที
    oled.clear();          // เคลียร์หน้าจอแสดงผล
    oled.show();           // ปรับปรุงการแสดงผล
    delay(2000);           // หน่วงเวลา 2 วินาที
  }
}
```

5.1.11 void drawPixel(int16_t x, int16_t y, uint16_t color) ;

เป็นเมธอดพล็อตจุดบนจอภาพตามพิกัดที่กำหนด โดยอ้างอิงจอแสดงผลขนาด 128x64 พิกเซล

พารามิเตอร์

x คือค่าพิกัดในแนวนอนหรือแกน x มีค่า 0 ถึง 127

y คือค่าพิกัดในแนวตั้งหรือแกน y มีค่า 0 ถึง 63

color คือสีที่ต้องการ

- คีนค่าเป็น BLACK (หรือ 0) สำหรับสีดำ
- คีนค่าเป็น WHITE (หรือ 1) สำหรับสีขาว

ตัวอย่างที่ 5-11

```
#include <POP32.h>
void setup()
{
  for(int i=0;i<128;i+=4)
  {
    oled.drawPixel(i,32,WHITE); // พล็อตจุดทุกๆ 4 พิกเซลในแนวแกน x กลางจอภาพ
  }
  for(int i=0;i<64;i+=4)
  {
    oled.drawPixel(64,i,WHITE); // พล็อตจุดทุกๆ 4 พิกเซลในแนวแกน y กลางจอภาพ
  }
  oled.show();                 // ปรับปรุงการแสดงผล
}
void loop()
{
}
```

5.1.12 void drawRect(int x1,int y1,int width, int height,uint16_t color) ;

เป็นเมธอดลากเส้นจากตำแหน่งที่กำหนดมายังตำแหน่งปลายทาง

พารามิเตอร์

x1 คือค่าตำแหน่งเริ่มต้นของรูปสี่เหลี่ยมในแกน x มีค่า 0 ถึง 127

y1 คือค่าตำแหน่งเริ่มต้นของรูปสี่เหลี่ยมในแกน y มีค่า 0 ถึง 63

width คือค่าความกว้างของรูปสี่เหลี่ยม (แนวนอน) มีค่า 1 ถึง 128

height คือค่าความสูงของรูปสี่เหลี่ยม (แนวตั้ง) มีค่า 1 ถึง 64

color คือค่าสีของเส้น

- คีนค่าเป็น BLACK (หรือ 0) สำหรับสีดำ

- คีนค่าเป็น WHITE (หรือ 1) สำหรับสีขาว

ตัวอย่างที่ 5-12

```
#include <POP32.h>
void setup()
{
    oled.drawRect(10,10,60,30,WHITE); // วาดรูปสี่เหลี่ยมเส้นสีขาว ขนาด 60 x 30 พิกเซล
    oled.show();                       // ปรับปรุงการแสดงผล
}
void loop()
{
}
```

5.1.13 void fillRect(int x1,int y1,int width, int height,uint16_t color);

เป็นการวาดรูปสี่เหลี่ยมทึบ โดยกำหนดจุดเริ่มต้นและความกว้างยาวของรูปสี่เหลี่ยมที่ต้องการ เมธอดนี้เป็นการสร้างรูปสี่เหลี่ยมที่ไม่มีเส้นกรอบ ในขณะที่เมธอด drawRect เป็นการวาดรูปกรอบสี่เหลี่ยมที่กำหนดสีของเส้นกรอบได้แต่ภายในกรอบไม่มีสี

พารามิเตอร์

x1 คือ ค่าตำแหน่งเริ่มต้นของรูปสี่เหลี่ยมในแกน x มีค่า 0 ถึง 127

y1 คือ ค่าตำแหน่งเริ่มต้นของรูปสี่เหลี่ยมในแกน y มีค่า 0 ถึง 63

width คือ ค่าความกว้างของรูปสี่เหลี่ยม (แนวนอน) มีค่า 1 ถึง 128

height คือ ค่าความสูงของรูปสี่เหลี่ยม (แนวตั้ง) มีค่า 1 ถึง 64

color คือ ค่าสีที่ระบาย

- คีนค่าเป็น BLACK (หรือ 0) สำหรับสีดำ
- คีนค่าเป็น WHITE (หรือ 1) สำหรับสีขาว

ตัวอย่างที่ 5-13

```
#include <POP32.h>
void setup()
{
    oled.fillRect(10,10,60,30,WHITE);    // วาดรูปสี่เหลี่ยมพื้นสีขาว ขนาด 60 x 30 พิกเซล
    oled.show();                          // ปรับปรุงการแสดงผล
}
void loop()
{
}
```

5.1.14 void drawLine(int x1, int y1, int x2, int y2, uint16_t color);

เป็นเมธอดลากเส้นจากจุดหนึ่งไปยังอีกจุดหนึ่ง กำหนดเป็นพิกัดในแนวแกนนอน (x) และ แกนตั้ง (y)

พารามิเตอร์

x1 คือ ค่าตำแหน่งเริ่มต้นของเส้นบนแกน x มีค่า 0 ถึง 127

y1 คือ ค่าตำแหน่งเริ่มต้นของเส้นบนแกน y มีค่า 0 ถึง 63

x2 คือ ค่าตำแหน่งสิ้นสุดของเส้นบนแกน x มีค่า 1 ถึง 127

y2 คือ ค่าตำแหน่งสิ้นสุดของเส้นบนแกน y มีค่า 1 ถึง 63

color คือ ค่าสีของเส้น

- คีนค่าเป็น BLACK (หรือ 0) สำหรับสีดำ

- คีนค่าเป็น WHITE (หรือ 1) สำหรับสีขาว

ตัวอย่างที่ 5-14

```
#include <POP32.h>
void setup()
{
    oled.drawLine(0,0,127,63,WHITE);    // วาดเส้นสีขาวทแยงจากมุมซ้ายบนลงมามุมขวาล่าง
    oled.show();                        // ปรับปรุงการแสดงผล
}
void loop()
{
}
```

5.1.15 void drawCircle(int x, int y, int radius, uint16_t color);

เป็นเมธอดวาดรูปเส้นวงกลม โดยกำหนดตำแหน่งของจุดศูนย์กลางของวงกลมและความยาวของรัศมี

พารามิเตอร์

x คือ พิกัดจุดศูนย์กลางของวงกลมบนแกนนอน มีค่าระหว่าง 0 ถึง 127

y คือ พิกัดจุดศูนย์กลางของวงกลมบนแกนตั้ง มีค่าระหว่าง 0 ถึง 63

radius คือ ค่ารัศมีของวงกลม

color คือ ค่าสีของเส้น

- คีนค่าเป็น BLACK (หรือ 0) สำหรับสีดำ
- คีนค่าเป็น WHITE (หรือ 1) สำหรับสีขาว

ตัวอย่างที่ 5-15

```
#include <POP32.h>
void setup()
{
    oled.drawCircle(63,31,26,WHITE);    // วาดเส้นวงกลมสีขาว รัศมี 26 พิกเซล
    oled.show();                        // ปรับปรุงการแสดงผล
}
void loop()
{
}
```

5.1.16 void fillCircle(int x, int y, int radius, uint16_t color);

เป็นเมธอดวาดรูปวงกลมทึบหรือแบบมีสีพื้นหลัง ด้วยการกำหนดตำแหน่งของจุดศูนย์กลางของวงกลมและความยาวของรัศมี

พารามิเตอร์

x คือพิกัดจุดศูนย์กลางของวงกลมบนแกน x มีค่าระหว่าง 0 ถึง 127

y คือพิกัดจุดศูนย์กลางของวงกลมบนแกน y มีค่าระหว่าง 0 ถึง 63

radius คือค่ารัศมีวงกลม

color คือ ค่าสีของเส้น

- คีนค่าเป็น BLACK (หรือ 0) สำหรับสีดำ
- คีนค่าเป็น WHITE (หรือ 1) สำหรับสีขาว

ตัวอย่างที่ 5-16

```
#include <POP32.h>
void setup()
{
    oled.fillCircle(63,31,26,WHITE); // วาดรูปวงกลมพื้นสีขาว รัศมี 26 พิกเซล
    oled.show();                      // ปรับปรุงการแสดงผล
}
void loop()
{
}
```

5.2 ฟังก์ชันกำเนิดเสียง**5.2.1 void beep () ;**

เป็นฟังก์ชันกำเนิดเสียง "ติ๊ด" มีความถี่ 500Hz นาน 100 มิลลิวินาที สำหรับขับออกไปทางลำโพงเปียโซของบอร์ด POP-32

ตัวอย่างที่ 5-17

```
#include <POP32.h>
void setup()
{
}
void loop()
{
    beep();           // กำเนิดเสียงความถี่ 500Hz นาน 100 มิลลิวินาที
    delay(1000);      // หน่วงเวลา 1 วินาที
}
```

5.2.2 void sound(uint16_t freq,uint32_t time);

เป็นฟังก์ชันกำเนิดเสียงที่กำหนดความถี่และระยะเวลาได้

พารามิเตอร์

freq คือ ค่าความถี่เสียงในหน่วยเฮิรตซ์ มีค่า 0 ถึง 32,767

time คือ ค่าเวลาของการกำเนิดเสียงในหน่วยมิลลิวินาที มีค่า 0 ถึง 32,767

ตัวอย่างที่ 5-18

```
#include <POP32.h>
void setup()
{
}
void loop()
{
    sound(1200,200); // กำเนิดเสียงความถี่ 1200Hz นาน 200 มิลลิวินาที
    delay(1000);     // หน่วงเวลา 1 วินาที
}
```

5.3 ฟังก์ชันตรวจสอบการกดสวิตช์ OK และ KNOB

5.5.1 void waitSW_OK() ;

เป็นฟังก์ชันรอกการกดสวิตช์ OK บนบอร์ด POP-32 แบบหนึ่ง เมื่อเรียกใช้งานฟังก์ชันนี้ ระบบจะทำการแสดงข้อความที่หน้าจอ OLED เพื่อรอผู้ใช้งานกดสวิตช์ OK

ตัวอย่างที่ 5-19

```
#include <POP32.h>

void setup()
{
    waitSW_OK(); // รอจนกระทั่งมีการสวิตช์หรือปุ่ม OK จึงย้ายไปทำงานอื่นต่อไปได้
}

void loop()
{
}
```

5.5.2 int SW_OK() ;

เป็นฟังก์ชันตรวจสอบสถานะสวิตช์ OK บนบอร์ด POP-32

การคืนค่า

“1” (เป็นจริง) เมื่อมีการกดสวิตช์

“0” (เป็นเท็จ) เมื่อไม่มีการกดสวิตช์

ตัวอย่างที่ 5-20

```
#include <POP32.h>

void setup()
{
}

void loop()
{
    if(SW_OK()) // ตรวจสอบการกดสวิตช์ OK
    {
        beep(); // เมื่อกดสวิตช์ OK จะมีเสียง "ติ๊ด" ดังออกลำโพง
        delay(200); // หน่วงเวลา 0.2 วินาที
    }
}
```


5.5.3 void waitSW_A() ;

เป็นฟังก์ชันตรวจสอบการกดสวิตช์ A บนบอร์ด **POP-32** ต้องรอนจนกระทั่งก้านสวิตช์ถูกปล่อยหลังจากกดสวิตช์ จึงจะผ่านไปกระทำคำสั่งอื่นต่อไป โดย LED ประจำตัวสวิตช์ A จะพริบต่อเนื่องเพื่อแจ้งผู้ใช้งานในระหว่างรอการกดสวิตช์

ตัวอย่างที่ 5-21

```
#include <POP32.h>

void setup()
{
    waitSW_A();      // รอนจนกระทั่งมีการกดสวิตช์ A
}

void loop()
{
    .....
}
```

5.5.4 int SW_A() ;

เป็นฟังก์ชันตรวจสอบสถานะสวิตช์ A บนบอร์ด **POP-32** โดยให้สถานะ “เป็นจริง” เมื่อมีการกดสวิตช์และ “เป็นเท็จ” เมื่อไม่มีการกดสวิตช์

การคืนค่า

“1” (เป็นจริง) เมื่อมีการกดสวิตช์

“0” (เป็นเท็จ) เมื่อไม่มีการกดสวิตช์

ตัวอย่างที่ 5-21

```
#include <POP32.h>

void setup()
{
}

void loop()
{
    if(SW_A())          // ตรวจสอบการกดสวิตช์ A
    {
        beep();        // เมื่อกดสวิตช์ A จะมีเสียง “ติ๊ด” ดังออกลำโพง
        delay(200);     // หน่วงเวลา 0.2 วินาที
    }
}
```

5.5.5 void waitSW_B() ;

เป็นฟังก์ชันตรวจสอบการกดสวิตช์ B บนบอร์ด **POP-32** โดยต้องรอจนกระทั่งก้านสวิตช์ถูกปล่อยหลังจากกดสวิตช์ จึงจะผ่านไปกระทำคำสั่งอื่นได้ โดยที่ LED ประจำตัวสวิตช์ B จะกะพริบต่อเนื่องเพื่อแจ้งผู้ใช้งานในระหว่างรอการกดสวิตช์

ตัวอย่างที่ 5-22

```
#include <POP32.h>

void setup()
{
    waitSW_B();    // รอจนกระทั่งมีการกดสวิตช์ B
}

void loop()
{
    .....
}
```

5.5.6 int SW_B() ;

เป็นฟังก์ชันตรวจสอบสถานะสวิตช์ B บนบอร์ด **POP-32** โดยให้สถานะ “เป็นจริง” เมื่อมีการกดสวิตช์และ “เป็นเท็จ” เมื่อไม่มีการกดสวิตช์

การคืนค่า

“1” (เป็นจริง) เมื่อมีการกดสวิตช์

“0” (เป็นเท็จ) เมื่อไม่มีการกดสวิตช์

ตัวอย่างที่ 5-23

```
#include <POP32.h>

void setup()
{
}

void loop()
{
    if(SW_B())        // ตรวจสอบการกดสวิตช์ B
    {
        beep();        // เมื่อกดสวิตช์ B จะมีเสียง "ติ๊ด" ดังออกลำโพง
        delay(200);    // หน่วงเวลา 0.2 วินาที
    }
}
```

5.5.7 void waitAnykey () ;

เป็นฟังก์ชันตรวจสอบการกดสวิตช์ใดๆ ไม่ว่าจะเป็นสวิตช์ A, B หรือ OK บนบอร์ด **POP-32** โดยต้องรอจนกระทั่งสวิตช์ถูกปล่อยหลังจากกดสวิตช์ จึงจะผ่านไปกระทำคำสั่งอื่นต่อไปได้ LED ประจำตัวสวิตช์ A และ B จะกะพริบต่อเนื่องเพื่อแจ้งผู้ใช้งานในระหว่างรอการกดสวิตช์

ตัวอย่างที่ 5-24

```
#include <POP32.h>

void setup()
{
    waitAnykey(); // รอจนกระทั่งเกิดการกดสวิตช์ A, B หรือ OK
}

void loop()
{
    .....
}
```

5.5.8 int knob () ;

เป็นฟังก์ชันอ่านค่าข้อมูลจากตัวต้านทานปรับค่าได้ **KNOB** บนบอร์ด **POP-32** ค่าที่อ่านได้มีค่า 0 ถึง 4,000

การคืนค่า

ค่าตัวเลข 0 ถึง 4,000

ตัวอย่างที่ 5-25

```
#include <POP32.h>

void setup()
{
}

void loop()
{
    oled.text(0,0,"Knob: %d  ",knob()); // อ่านค่าข้อมูลจาก KNOB มาแสดงที่จอแสดงผล
    oled.show();                        // ปรับปรุงการแสดงผล
}
```

5.5.9 int knob(int scale) ;

เป็นฟังก์ชันอ่านค่าจากตัวต้านทานปรับค่าได้ KNOB บนบอร์ด POP-32 ค่าที่อ่านได้มีค่า 0 ถึงค่าที่กำหนด

พารามิเตอร์

scale คือค่าสูงสุดที่ต้องการกำหนดจากการอ่านค่า KNOB

การคืนค่า

เป็นค่า 0 ถึง ค่าของ scale

ตัวอย่างที่ 5-26

```
#include <POP32.h>
void setup()
{
  void loop()
  {
    oled.text(0,0,"Knob: %d  ",knob(100)); // อ่านข้อมูลจาก KNOB มาแสดงที่จอแสดงผล
                                              // มีค่า 0 ถึง 100
    oled.show();                          // ปรับปรุงการแสดงผล
  }
}
```

5.5.10 int knob(intscaleCCW,intscaleCW) ;

เป็นฟังก์ชันอ่านค่าข้อมูลจากตัวต้านทานปรับค่าได้ KNOB บนบอร์ด POP-32 โดยค่าที่อ่านได้มีค่าในช่วง scaleCCW ถึงค่า scaleCW ตามที่กำหนด

พารามิเตอร์

scaleCCW คือ ค่าต่ำสุดจากการอ่านค่า KNOB

scaleCW คือ ค่าสูงสุดจากการอ่านค่า KNOB

การคืนค่า

ค่าที่อ่านได้จากฟังก์ชันนี้มีค่าในช่วง scaleCCW ถึง scaleCW

ตัวอย่างที่ 5-27

```
#include <POP32.h>
void setup()
{
  void loop()
  {
    oled.text(0,0,"Knob: %d  ",knob(20,500)); // อ่านค่าจาก KNOB มาแสดงที่จอแสดงผล
                                              // มีค่า 20 ถึง 500
    oled.show();                          // ปรับปรุงการแสดงผล
  }
}
```

5.4 ฟังก์ชันเข้าถึงพอร์ตอินพุต/เอาต์พุต

5.4.1 void out(int num,int dat) ;

เป็นฟังก์ชันส่งข้อมูลออกทางพอร์ตเอาต์พุตที่กำหนด

พารามิเตอร์

num คือ หมายเลขพอร์ตเอาต์พุต กำหนดได้เป็น 0 ถึง 7 และ 16

dat คือ ค่าข้อมูลที่ส่งออก มีค่า 1 หรือ HIGH กับ 0 หรือ LOW

ตัวอย่างที่ 5-28

```
#include <POP32.h>
void setup()
{
  void loop()
  {
    out(3,1);      // กำหนดให้พอร์ต 3 มีสถานะลอจิกเป็น "1" (HIGH)
    delay(500);    // หน่วงเวลา 0.5 วินาที
    out(3,0);      // กำหนดให้พอร์ต 3 มีสถานะลอจิกเป็น "0" (LOW)
    delay(500);    // หน่วงเวลา 0.5 วินาที
  }
}
```

5.4.2 int in(int num) ;

เป็นฟังก์ชันอ่านค่าจากพอร์ตอินพุตที่กำหนด

พารามิเตอร์

num คือ หมายเลขพอร์ตอินพุตที่ต้องการอ่านค่า กำหนดได้เป็น 0 ถึง 7 และ 16

การคืนค่า

คืนค่าเป็น "1" หรือ "0"

ตัวอย่างที่ 5-29

```
#include <POP32.h>
void setup()
{
  void loop()
  {
    if(in(6)==0)      // ตรวจสอบค่าอินพุตที่พอร์ต 6 มีค่าเป็น "0" หรือไม่
    {
      beep();         // ถ้าสถานะลอจิกของพอร์ต 6 เป็น "0" ขั้วเสียง "ดีด" ออกลำโพง
      delay(200);     // หน่วงเวลา 0.2 วินาที
    }
  }
}
```

5.5 ฟังก์ชัน analog

5.5.1 int analog(char ch) ;

เป็นฟังก์ชันอ่านข้อมูลดิจิทัลที่ได้จากการแปลงสัญญาณแอนะล็อกของไมโครคอนโทรลเลอร์ที่ขาพอร์ต A0 ถึง A8 ใช้เชื่อมต่อกับตัวตรวจจับที่ให้ผลการทำงานในรูปแบบแรงดันไฟฟ้าในย่าน 0 ถึง 3.3V

พารามิเตอร์

ch คือหมายเลขอินพุตแอนะล็อกที่ต้องการอ่านค่า มีค่า 0 ถึง 8 ซึ่งตรงกับขาพอร์ต A0 ถึง A8

การคืนค่า

คืนค่าเป็นข้อมูลจากวงจรแปลงสัญญาณแอนะล็อกเป็นดิจิทัลภายในไมโครคอนโทรลเลอร์ จากช่องอินพุตที่กำหนด มีความละเอียดในการแปลงสัญญาณ 12 บิต ค่าที่เป็นไปได้คือ 0 ถึง 4,095

ตัวอย่างที่ 5-30

```
#include <POP32.h>
void setup()
{
void loop()
{
    intadc = analog(8);    // อ่านค่าการแปลงสัญญาณ ADC จากพอร์ต A8 เก็บที่ตัวแปร adc
    oled.text(1,0,"Analog: %d ",adc);    // แสดงค่าข้อมูลที่อ่านได้ที่จอแสดงผล
    oled.show();           // ปรับปรุงการแสดงผล
}
}
```

5.6 ฟังก์ชันควบคุมมอเตอร์

5.6.1 void motor(char ch,int pow) ;

เป็นฟังก์ชันขับมอเตอร์ไฟตรง

พารามิเตอร์

ch คือ ตำแหน่งเอาต์พุตของวงจรขับมอเตอร์ไฟตรงของบอร์ด POP-32 มีค่า 1, 2 , 3 หรือ 4

pow คือ กำลังขับมอเตอร์ มีค่า -100 ถึง 100

ถ้ากำหนดค่า pow เป็นบวก (1 ถึง 100) ทำให้มอเตอร์หมุนไปในทิศทางหนึ่ง

ถ้ากำหนดค่า pow เป็นลบ (-1 ถึง -100) มอเตอร์จะถูกขับไปในทิศทางตรงข้าม

ถ้ากำหนดค่า pow เป็น 0 มอเตอร์จะหยุดหมุน แต่ไม่แนะนำให้กำหนดเป็น 0 หากต้องการให้มอเตอร์หยุดหมุนควรเรียกใช้ฟังก์ชัน motor_stop

ตัวอย่างที่ 5-31

```
motor(1,60); // ขับมอเตอร์ช่อง 1 ด้วยกำลังขับ 60% ของกำลังสูงสุด
motor(1,-60); // ขับมอเตอร์ช่อง 1 ด้วยกำลังขับ 60% ไปในทิศทางตรงข้าม
```

ตัวอย่างที่ 5-32

```
motor(2,100); // ขับมอเตอร์ช่อง 2 ด้วยกำลังขับ 100%
motor(2,-100); // ขับมอเตอร์ช่อง 2 ด้วยกำลังขับ 100% ไปในทิศทางตรงข้าม
```

5.6.2 void motor_stop(char ch) ;

เป็นฟังก์ชันหยุดขับเคลื่อนมอเตอร์

พารามิเตอร์

ch คือ ช่องเอาต์พุตมอเตอร์ไฟตรงมีค่าเป็น 1, 2, 3 และ 4

- ถ้ากำหนดเป็นมาโคร 1234 จะเป็นการเลือกให้มอเตอร์ไฟตรงทั้ง 4 ช่องหยุดทำงานพร้อมกัน

ตัวอย่างที่ 5-33

```
motor_stop(1); // หยุดขับเคลื่อนมอเตอร์ช่อง 1
motor_stop(2); // หยุดขับเคลื่อนมอเตอร์ช่อง 2
```

ตัวอย่างที่ 5-34

```
motor_stop(1234); // หยุดขับเคลื่อนมอเตอร์ทั้ง 4 ช่องพร้อมกัน
```

5.7 กลุ่มฟังก์ชันขับเคลื่อนหุ่นยนต์แบบ 2 ล้อ

โดยอ้างอิงให้ล้อซ้ายถูกขับเคลื่อนด้วยวงจรจับมอเตอร์ช่องที่ 1 ส่วนล้อขวาถูกขับเคลื่อนด้วยวงจรจับมอเตอร์ช่องที่ 2

5.7.1 void fd(int8_t speed) ;

fd ย่อมาจากคำว่า **forward** เป็นฟังก์ชันกำหนดให้หุ่นยนต์เคลื่อนที่ไปข้างหน้า โดย LED แสดงสถานะการทำงานของมอเตอร์ไฟตรงทั้งสองช่องจะติดสว่างเป็นสีเขียว

พารามิเตอร์

speed คือ กำลังขับเคลื่อน มีค่า 0 ถึง 100

ตัวอย่างที่ 5-35

```
fd(80); // กำหนดให้หุ่นยนต์เคลื่อนที่ไปข้างหน้าด้วยกำลังขับ 80%
```

5.7.2 void fd2(int8_t speed1, int8_t speed2) ;

fd2 มาจากคำว่า **forward2** เป็นฟังก์ชันกำหนดให้หุ่นยนต์เคลื่อนที่ไปข้างหน้าด้วยการกำลังขับเคลื่อนไฟตรงทั้งสองช่องแยกกันอิสระ เมื่อหุ่นยนต์ทำงาน LED แสดงสถานะการทำงานของมอเตอร์ไฟตรงทั้งสองช่องจะติดสว่างเป็นสีเขียว

พารามิเตอร์

speed1 คือ ค่ากำลังขับของวงจรจับมอเตอร์ไฟตรงช่อง 1 มีค่า 0 ถึง 100

speed2 คือ ค่ากำลังขับของวงจรจับมอเตอร์ไฟตรงช่อง 2 มีค่า 0 ถึง 100

ตัวอย่างที่ 5-36

```
fd2(30,80);
// ควบคุมให้หุ่นยนต์เคลื่อนที่เป็นวงกลม เนื่องจากมอเตอร์ไฟตรงช่อง 2
// หมุนเร็วกว่ามอเตอร์ไฟตรงช่อง 1 มาก
```

5.7.3 void bk(int8_t speed) ;

bk มาจากคำว่า **backward** เป็นฟังก์ชันกำหนดให้หุ่นยนต์ถอยหลัง เมื่อหุ่นยนต์ทำงาน LED แสดงสถานะการทำงานของมอเตอร์ไฟตรงทั้งสองช่องจะติดสว่างเป็นสีแดง

พารามิเตอร์

speed คือ กำลังขับเคลื่อน มีค่า 0 ถึง 100

ตัวอย่างที่ 5-37

```
bk(90); // ควบคุมให้หุ่นยนต์ถอยหลังด้วยกำลังขับ 90%
```


5.7.4 void bk2(int8_t speed1, int8_t speed2);

bk2 มาจากคำว่า **backward2** เป็นฟังก์ชันกำหนดให้หุ่นยนต์ถอยหลังแบบกำหนดกำลังขับเคลื่อนมอเตอร์ไฟตรงแยกอิสระ เมื่อหุ่นยนต์ทำงาน LED แสดงสถานะการทำงานของมอเตอร์ไฟตรงทั้งสองช่องจะติดสว่างเป็นสีแดง

พารามิเตอร์

speed1 คือ ค่ากำลังขับเคลื่อนมอเตอร์ไฟตรงช่อง 1 มีค่า 0 ถึง 100

speed2 คือ ค่ากำลังขับเคลื่อนมอเตอร์ไฟตรงช่อง 2 มีค่า 0 ถึง 100

ตัวอย่างที่ 5-38

bk2(80,80); // ควบคุมให้หุ่นยนต์ถอยหลังตรงด้วยความเร็วเท่ากัน ทำงานเหมือนกับเมธอด bk

5.7.5 void tl(int8_t speed);

tl มาจากคำว่า **turn left** เป็นฟังก์ชันกำหนดให้หุ่นยนต์เลี้ยวซ้ายแบบล้อเดียว โดยการเลี้ยวของฟังก์ชันนี้จะกำหนดให้มอเตอร์ไฟตรงที่ขับเคลื่อนด้านซ้ายหยุดหมุน ส่วนมอเตอร์ไฟตรงที่ขับเคลื่อนขวาจะถูกขับให้หมุนไปข้างหน้า เมื่อหุ่นยนต์ทำงาน LED แสดงสถานะการขับเคลื่อนมอเตอร์ที่ล้อขวาคิดสว่างเป็นสีเขียว

พารามิเตอร์

speed คือ ค่ากำลังขับเคลื่อนมอเตอร์ไฟตรง มีค่า 0 ถึง 100

ตัวอย่างที่ 5-39

tl(50); // กำหนดให้หุ่นยนต์เลี้ยวซ้ายแบบล้อเดียวด้วยกำลังขับ 50%

5.7.6 void tr(int8_t speed);

tr มาจากคำว่า **turn right** เป็นฟังก์ชันกำหนดให้หุ่นยนต์เลี้ยวขวาแบบล้อเดียว โดยการเลี้ยวของฟังก์ชันนี้จะกำหนดให้มอเตอร์ไฟตรงที่ใช้ขับเคลื่อนด้านขวาหยุดหมุน ส่วนมอเตอร์ไฟตรงที่ล้อซ้ายถูกขับให้หมุนไปข้างหน้า เมื่อหุ่นยนต์ทำงาน LED ที่เอาต์พุตวงจรขับเคลื่อนมอเตอร์ไฟตรงที่ล้อซ้ายจะติดสว่างเป็นสีเขียว

พารามิเตอร์

speed คือ กำลังขับเคลื่อนมอเตอร์ไฟตรง มีค่า 0 ถึง 100

ตัวอย่างที่ 5-40

tr(50); // สั่งให้หุ่นยนต์เลี้ยวขวาแบบล้อเดียวด้วยกำลังขับ 50 เปอร์เซ็นต์

5.7.7 void sl(int8_t speed) ;

sl มาจากคำว่า spin left เป็นฟังก์ชันกำหนดให้หุ่นยนต์หมุนตัวเลี้ยวซ้ายแบบสองล้อ โดยการเลี้ยวของฟังก์ชันนี้จะกำหนดให้ล้อด้านซ้ายหมุนไปข้างหลัง ส่วนล้อด้านขวาหมุนไปข้างหน้า เมื่อหุ่นยนต์ทำงาน LED ที่เอาต์พุตวงจรขับมอเตอร์ไฟตรงที่ล้อซ้ายจะติดสว่างเป็นสีแดง ส่วนที่ล้อขวา LED จะติดสว่างเป็นสีเขียว

พารามิเตอร์

speed คือ ค่ากำลังขับมอเตอร์ไฟตรง มีค่า 0 ถึง 100

ตัวอย่างที่ 5-41

sl(70); // สั่งให้หุ่นยนต์หมุนซ้ายด้วยกำลังขับ 70 เปอร์เซ็นต์

5.7.8 void sr(int8_t speed) ;

sr มาจากคำว่า spin right เป็นฟังก์ชันกำหนดให้หุ่นยนต์หมุนตัวขวาแบบสองล้อ โดยการเลี้ยวของฟังก์ชันนี้จะกำหนดให้ล้อด้านขวาหมุนไปข้างหลัง ส่วนล้อด้านซ้ายหมุนไปข้างหน้า เมื่อหุ่นยนต์ทำงาน LED ที่เอาต์พุตวงจรขับมอเตอร์ไฟตรงที่ล้อขวาจะติดสว่างเป็นสีแดง ส่วนล้อซ้ายจะติดสว่างเป็นสีเขียว

พารามิเตอร์

speed คือ ค่ากำลังขับมอเตอร์ไฟตรง มีค่า 0 ถึง 100

ตัวอย่างที่ 5-42

sr(70); // สั่งให้หุ่นยนต์หมุนขวาดูด้วยกำลังขับ 70 เปอร์เซ็นต์

5.7.9 void ao() ;

เป็นฟังก์ชันหยุดขับเคลื่อนหุ่นยนต์

ตัวอย่างที่ 5-43

ao(); // สั่งให้หยุดขับเคลื่อนหุ่นยนต์

5.7.10 void servo(uint8_t ch,int angle) ;

เป็นฟังก์ชันที่กำหนดตำแหน่งแกนหมุนของเซอร์โวมอเตอร์ โดยควบคุมเซอร์โวมอเตอร์ได้ 6 ตัว

พารามิเตอร์

ch คือ ช่องเอาต์พุตเซอร์โวมอเตอร์ มีค่า 1 ถึง 6

angle คือ ค่าตำแหน่งแกนหมุนเซอร์โวมอเตอร์ มีค่า 0 ถึง 180 และ -1

ถ้ากำหนดเป็น -1 หมายถึง ไม่ใช้งานเซอร์โวมอเตอร์ที่ช่องนั้นๆ

ตัวอย่างที่ 5-44

```

#include <POP32.h>

void setup()
{
}

void loop()
{
    servo(1,60);    // ขับเซอร์โวมอเตอร์ช่อง 1 ให้แกนหมุนไปยังตำแหน่ง 60 องศา
    sleep(2000);    // หน่วงเวลา 2 วินาที
    servo(1,120);   // ขับเซอร์โวมอเตอร์ช่อง 1 ให้แกนหมุนไปยังตำแหน่ง 120 องศา
    sleep(2000);    // หน่วงเวลา 2 วินาที
}

```



